

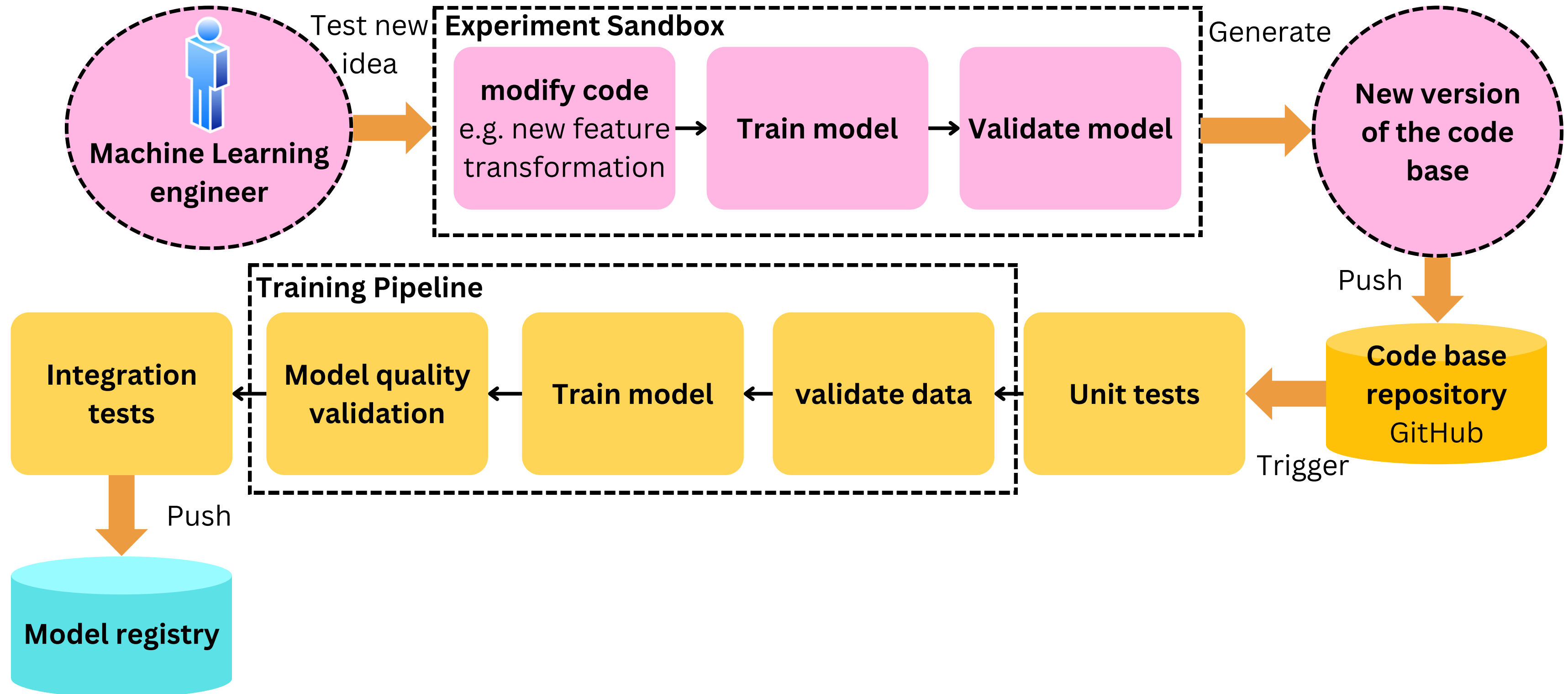


The **AiEdge** ACADEMY

Deploying LLMs

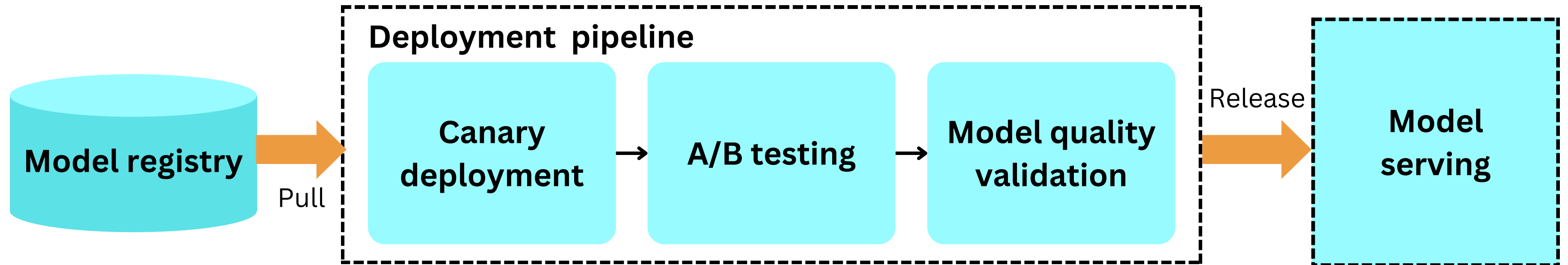


Model Development Life Cycle



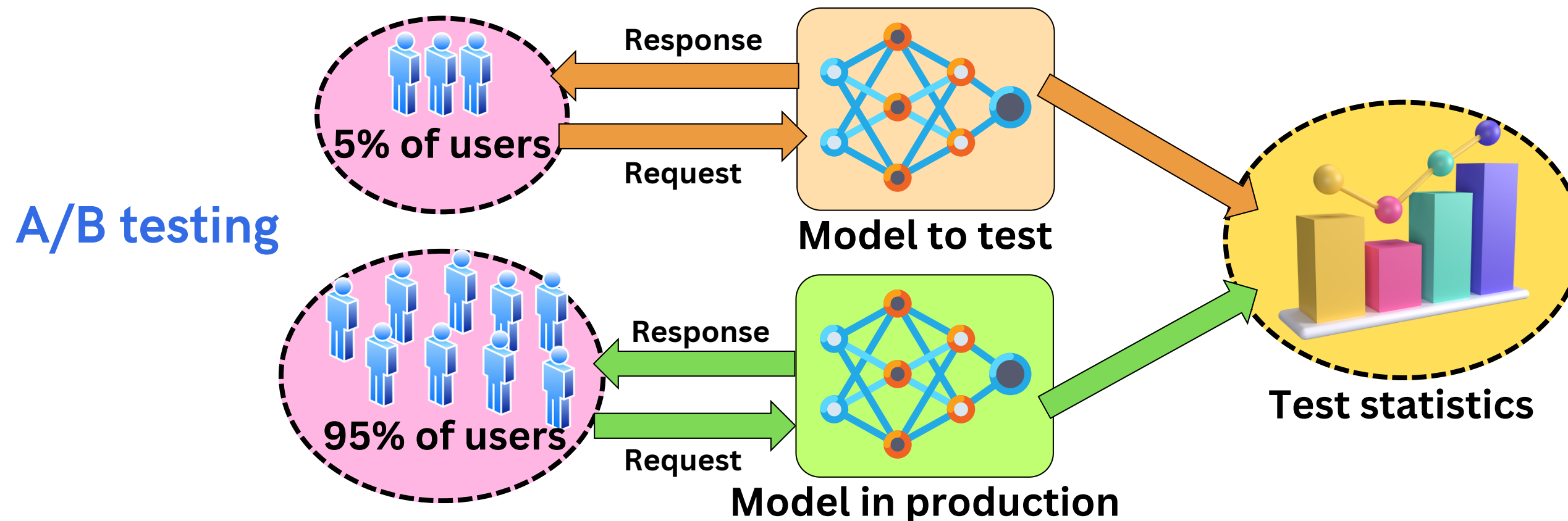
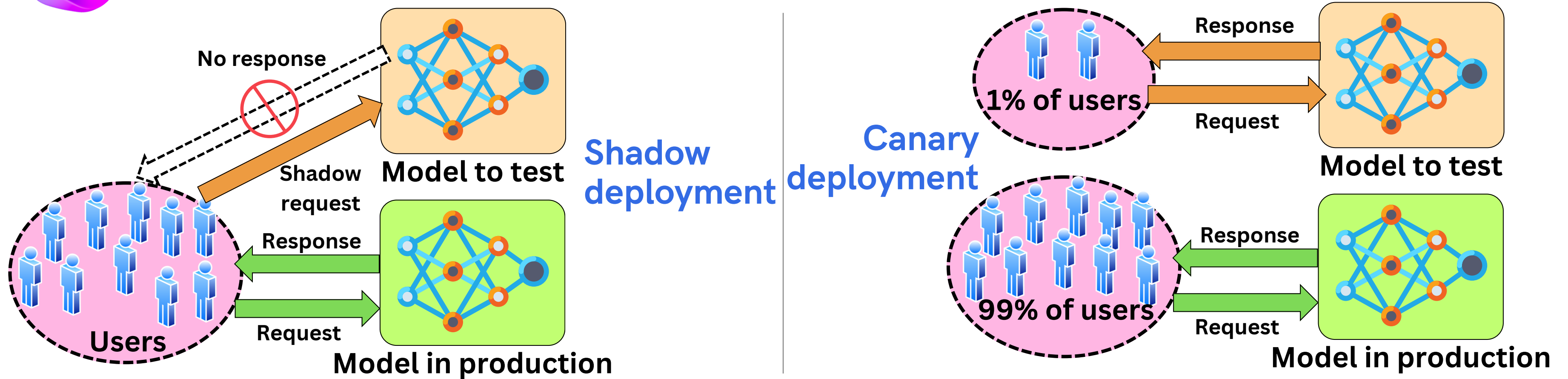


The Deployment Pipeline





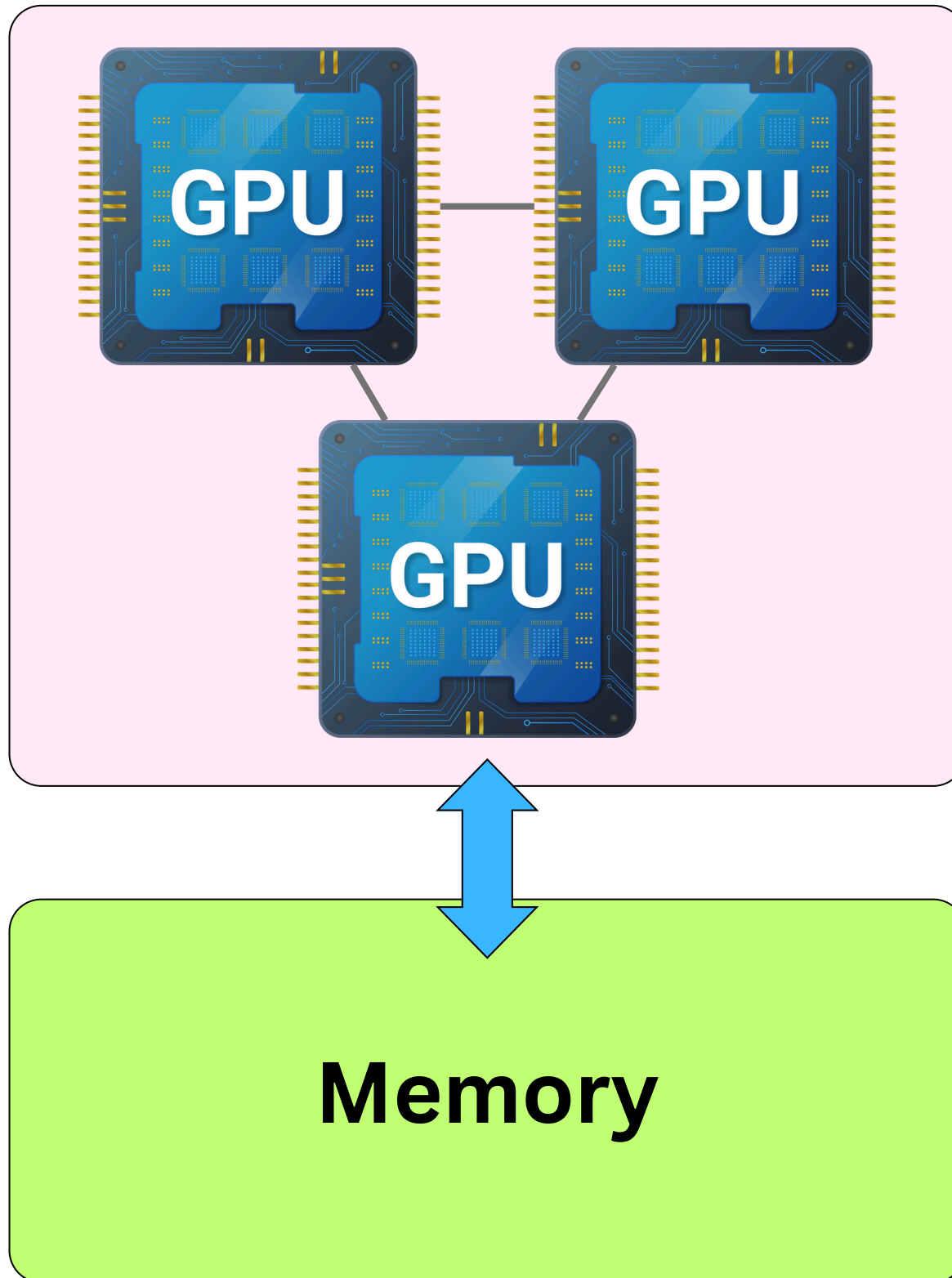
Pre-Production Deployment





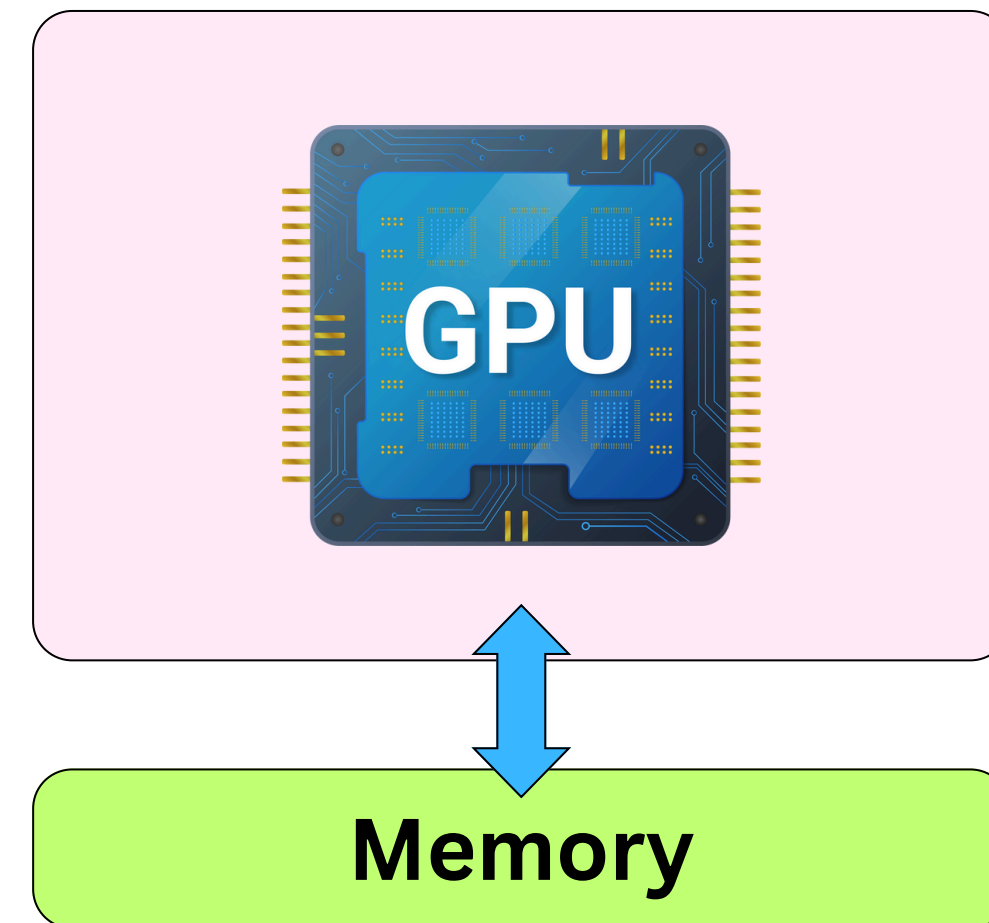
Training GPUs vs Serving GPUs

Training



- Nvidia A100
- Nvidia V100
- Nvidia H100

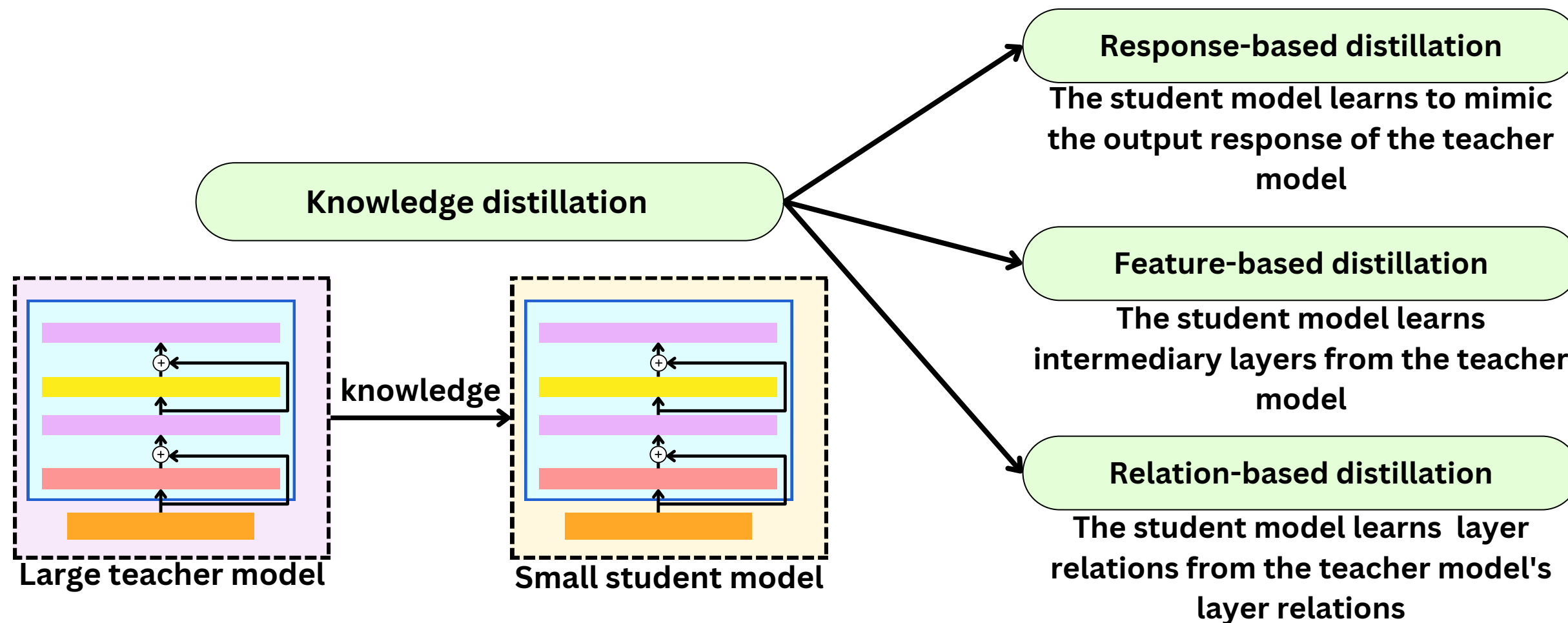
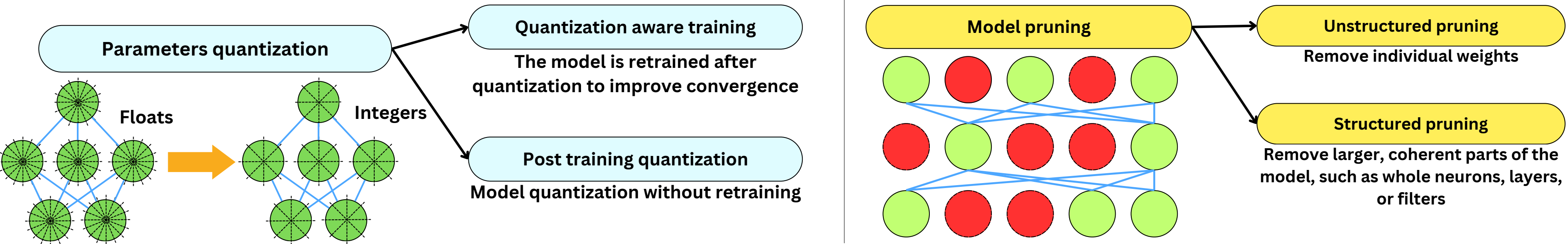
Serving



- Nvidia T4
- Nvidia A10
- Nvidia L4



Reducing the memory footprint





The Different Deployment Strategies

- With a dedicated GPU machine
- With a dedicated CPU machine
- Serverless deployment
- On the Edge



Dedicated GPU Machine

- **Sagemaker**
- **Huggingface GPU space**
- **Vertex AI**



Serverless Deployment

- **aws bedrock**
- **Hugging Face Inference API**
- **Replicate**

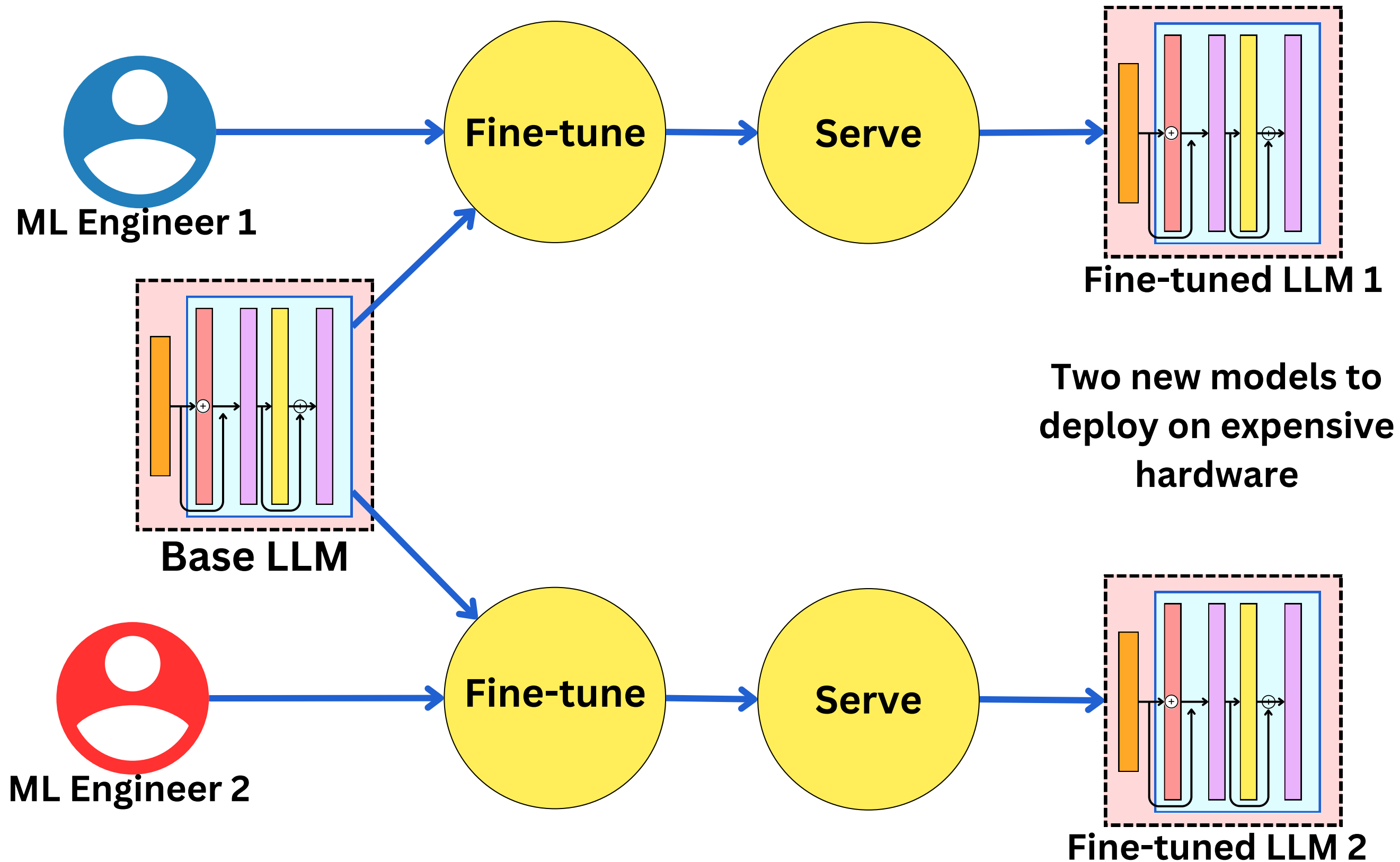


Using WebGPU

- [**https://github.com/mlc-ai/web-llm**](https://github.com/mlc-ai/web-llm)
- [**https://huggingface.co/Xenova**](https://huggingface.co/Xenova)

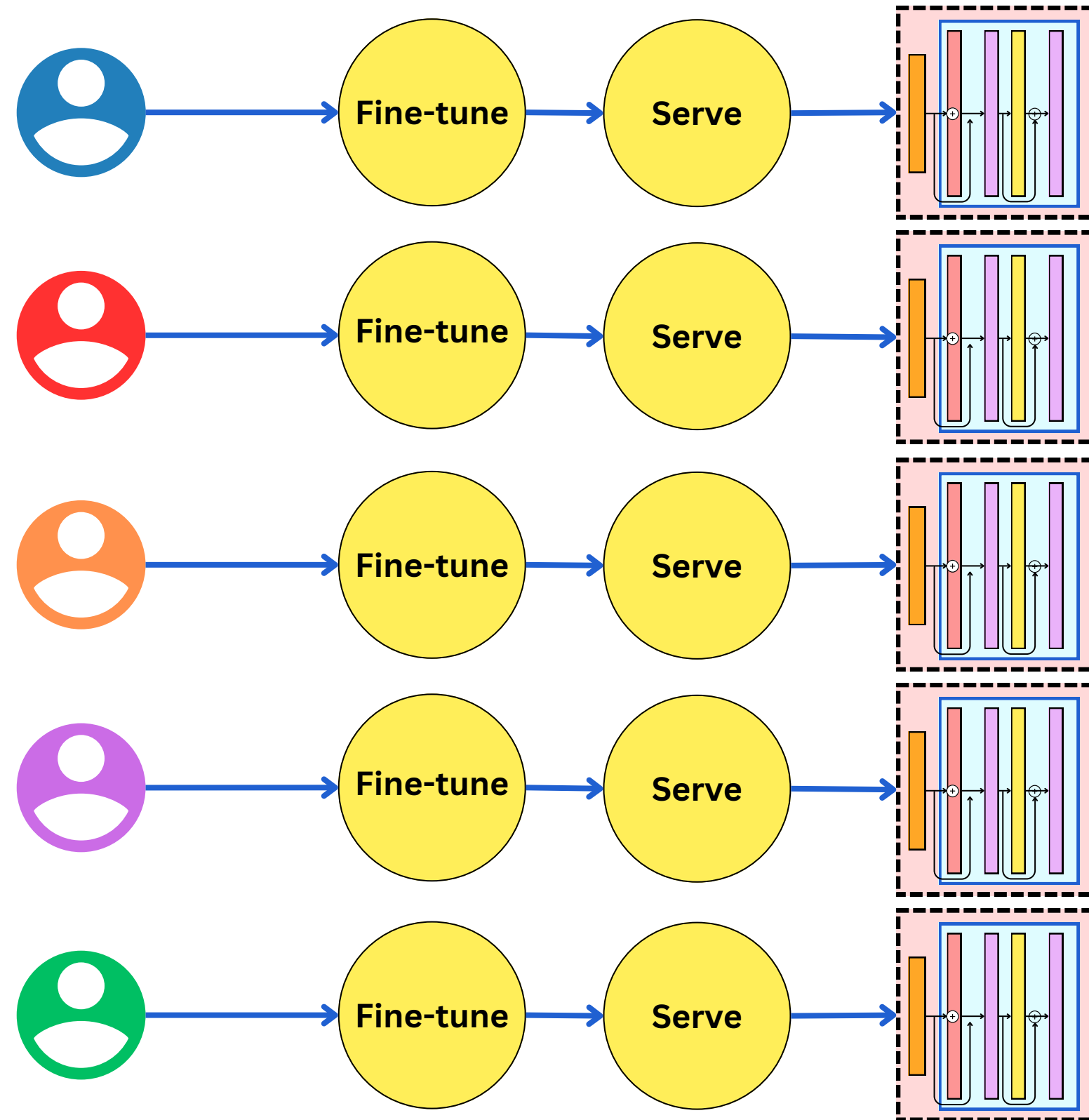


Multi-LoRA





Multi-LoRA

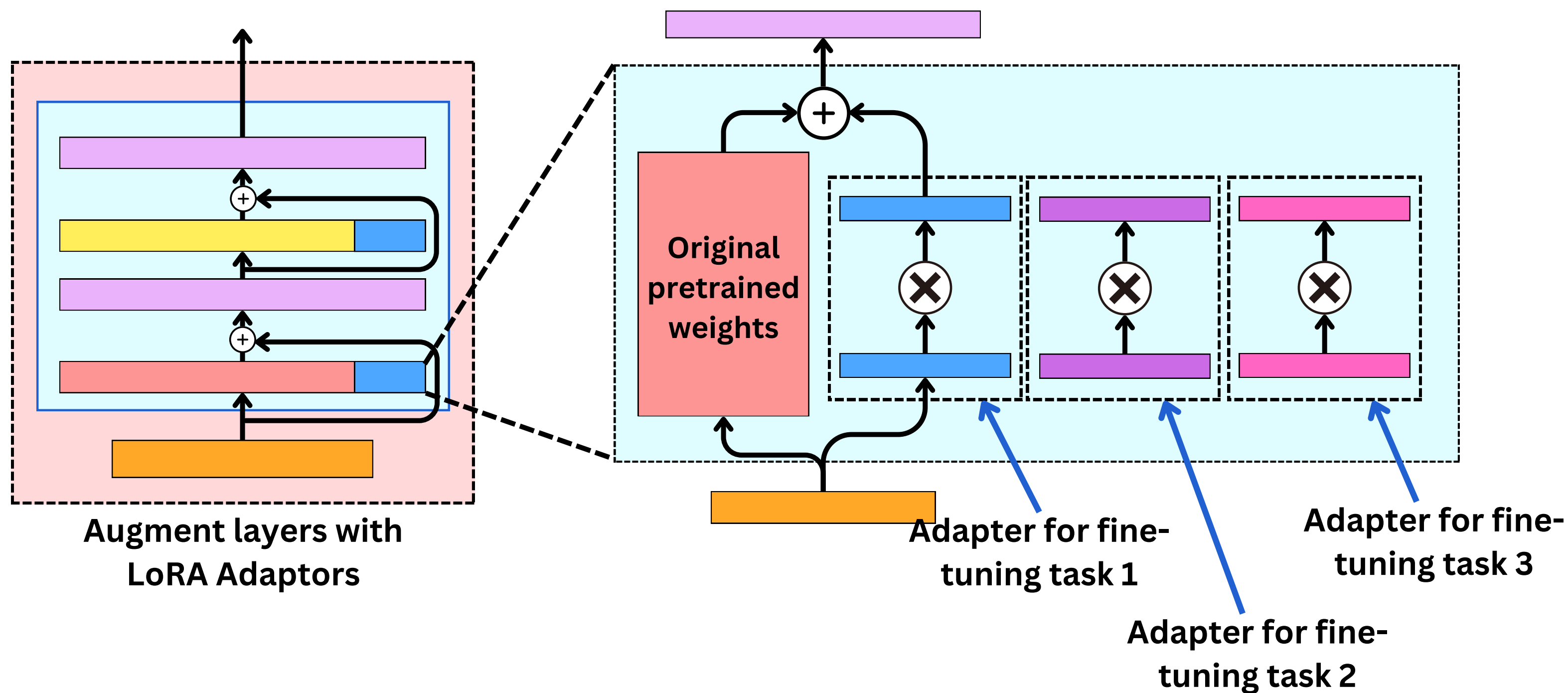


**Can become
very expensive**



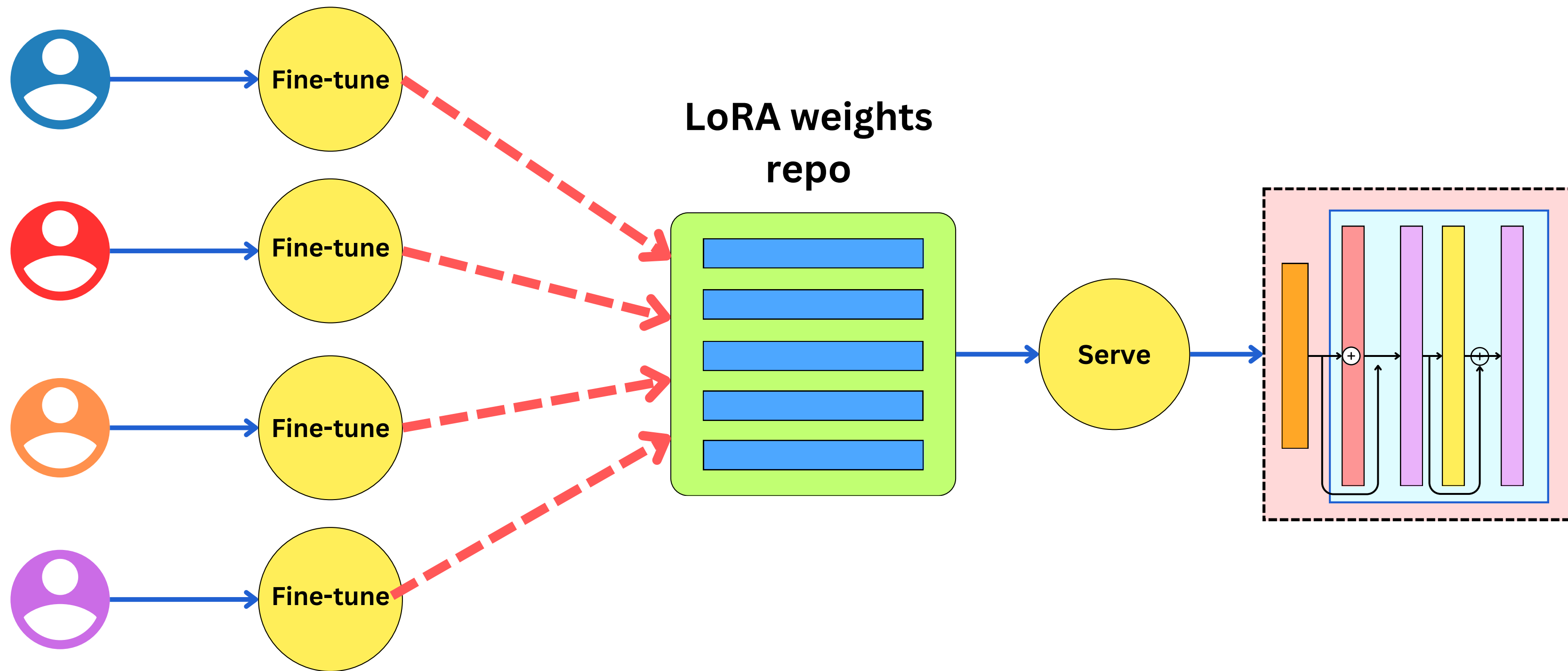
Multi-LoRA

Multiple adapters for the same base model





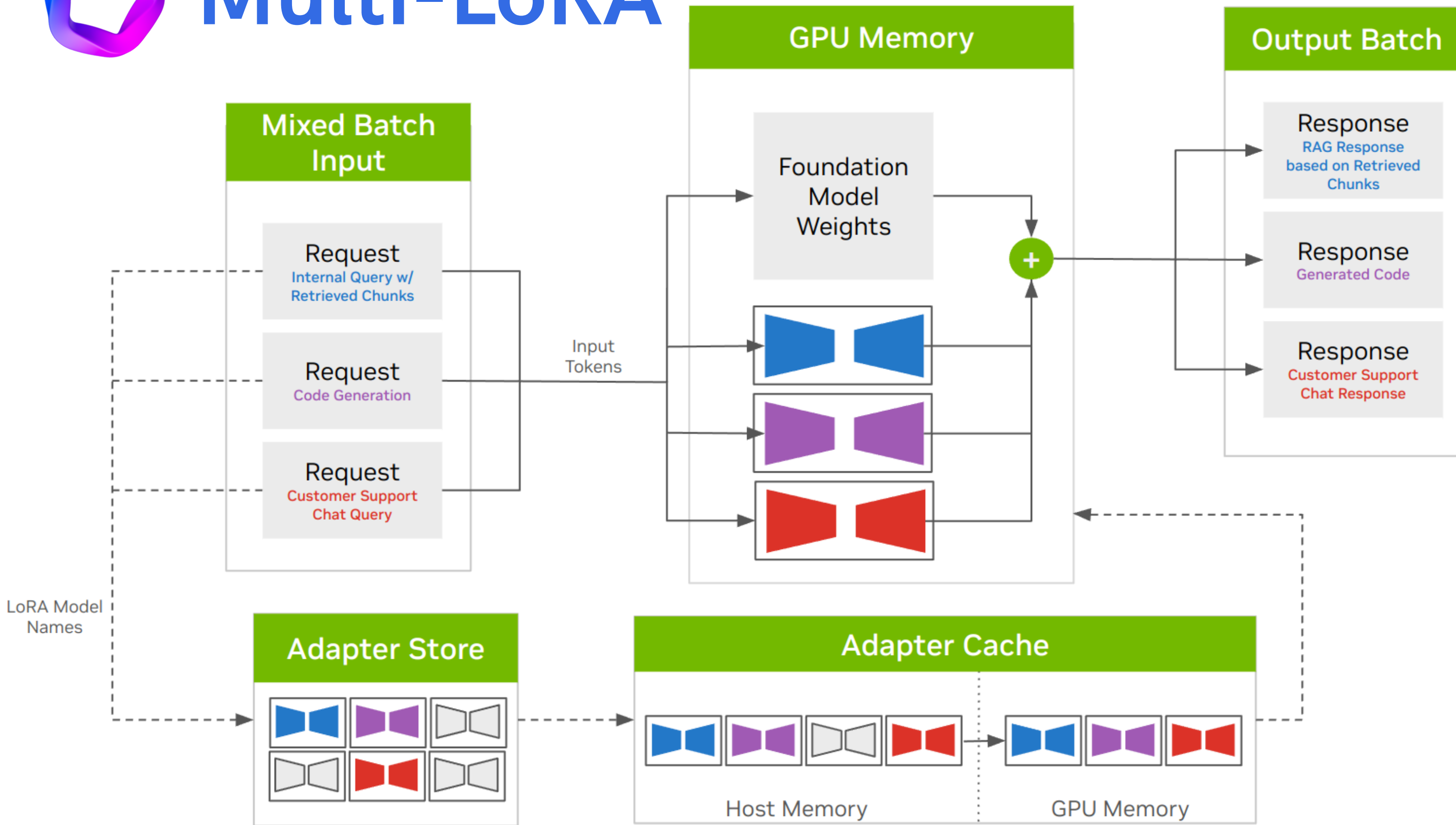
Multi-LoRA





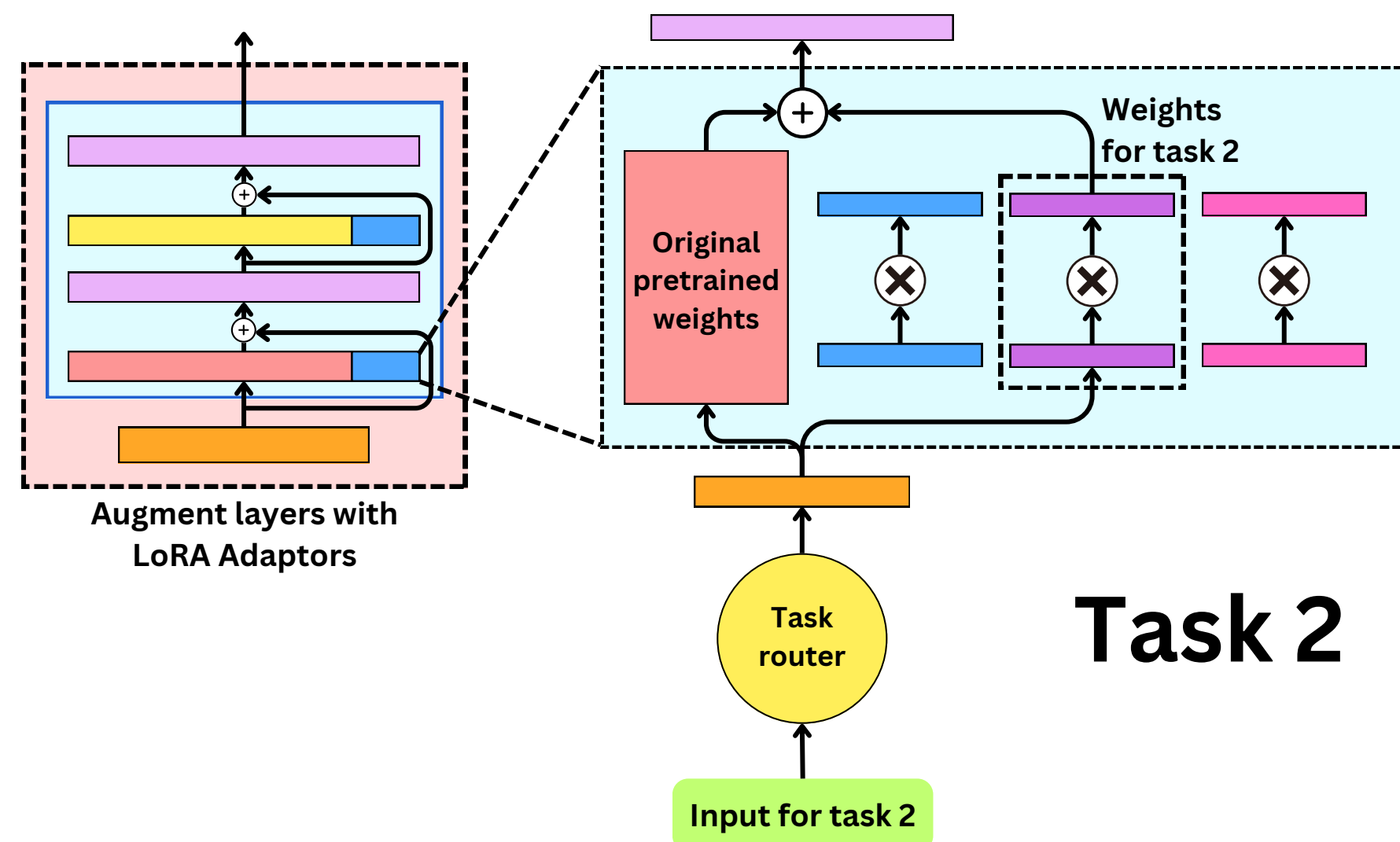
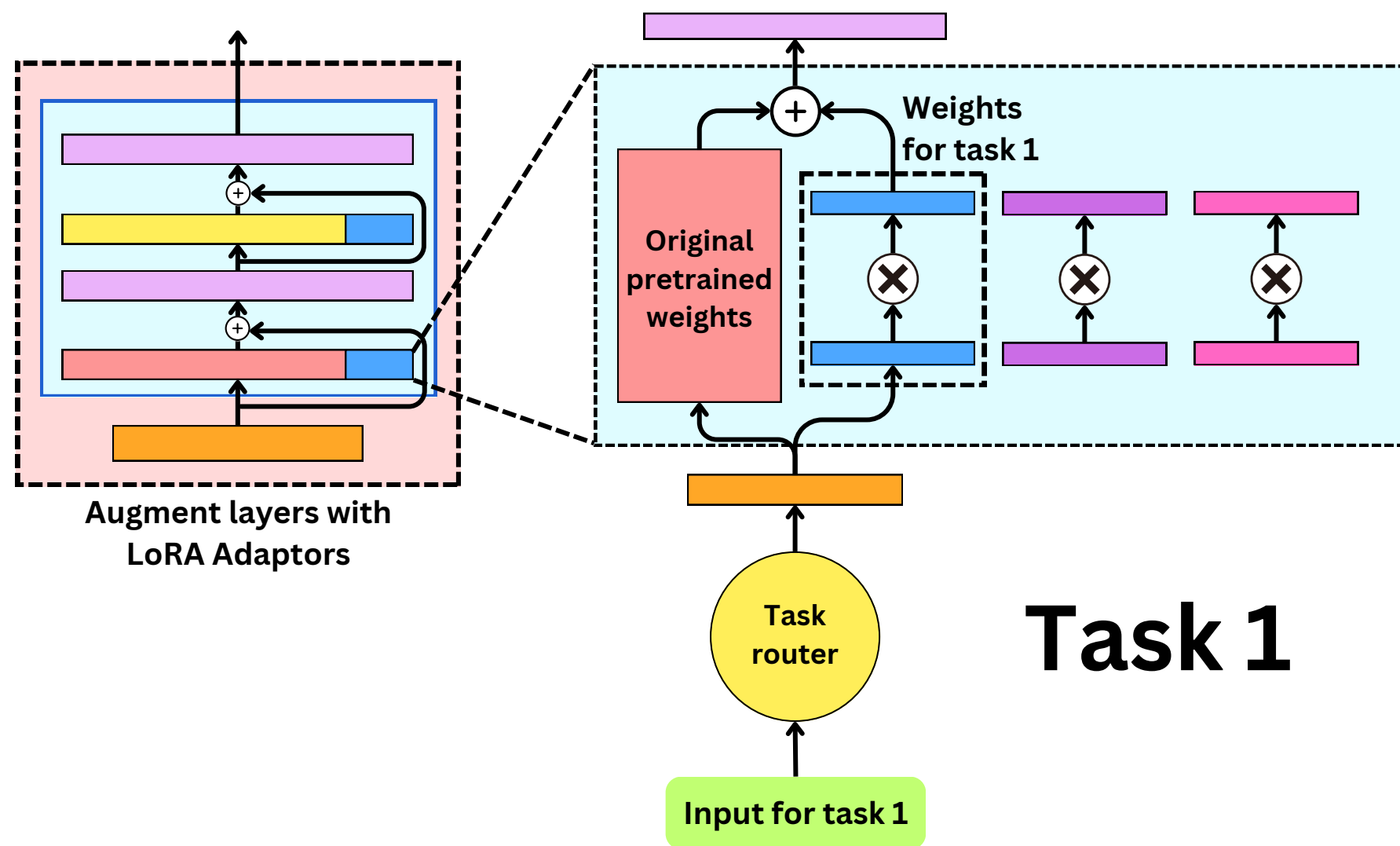
Multi-LoRA

Nvidia NIM



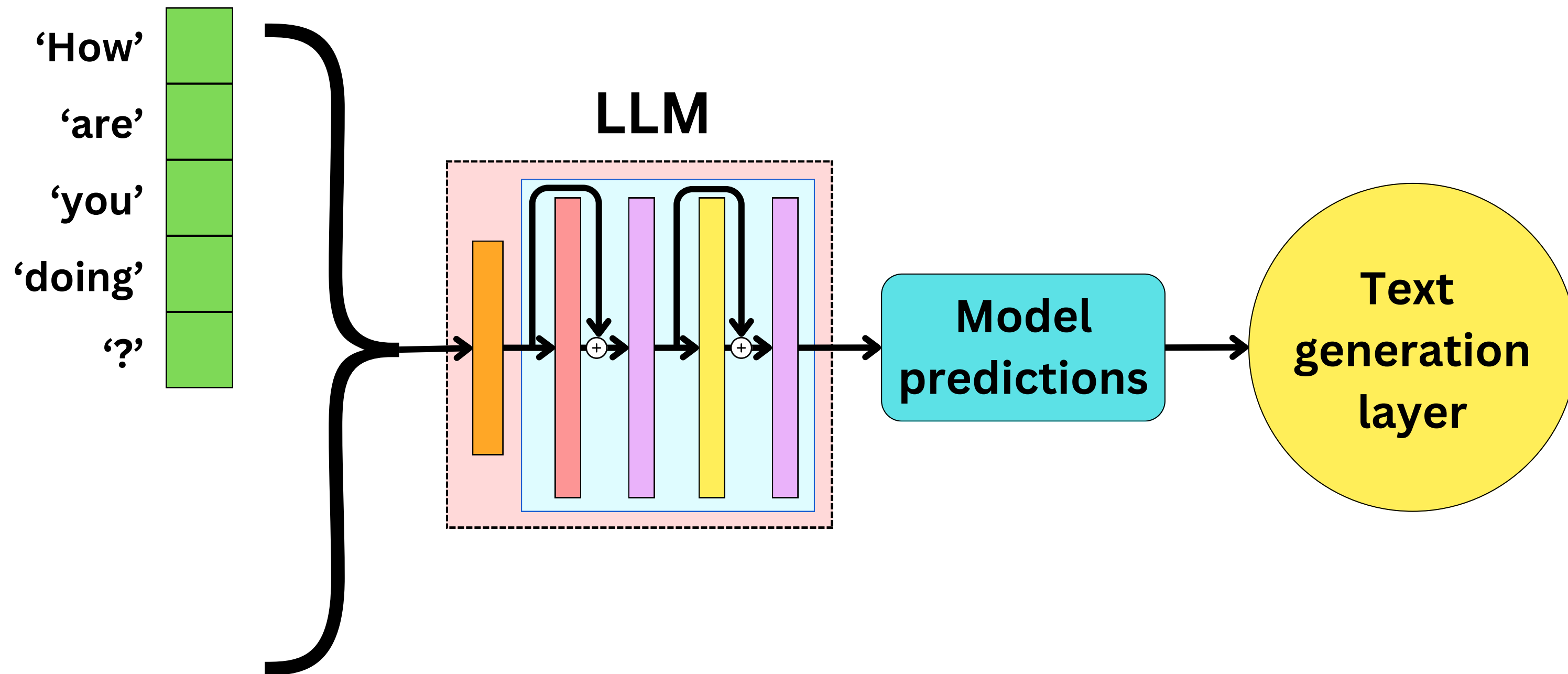


Multi-LoRA



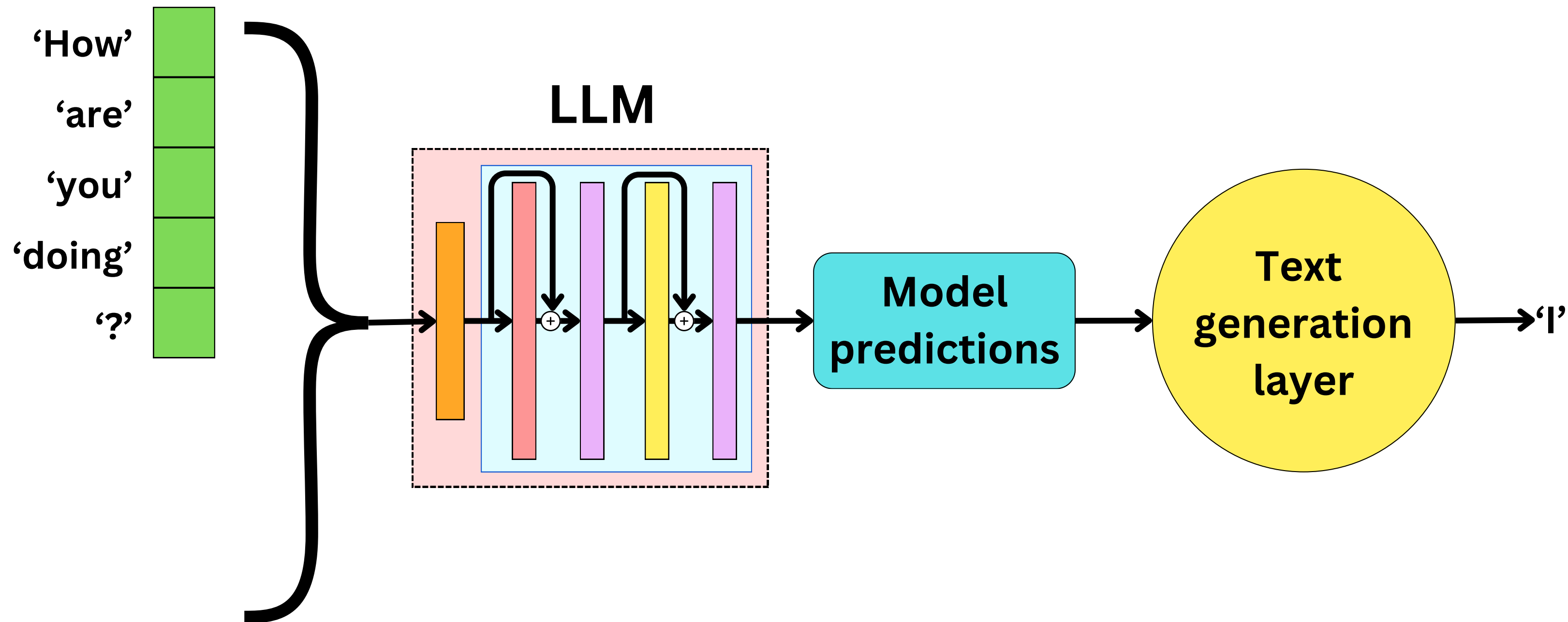


The Text Generation Layer



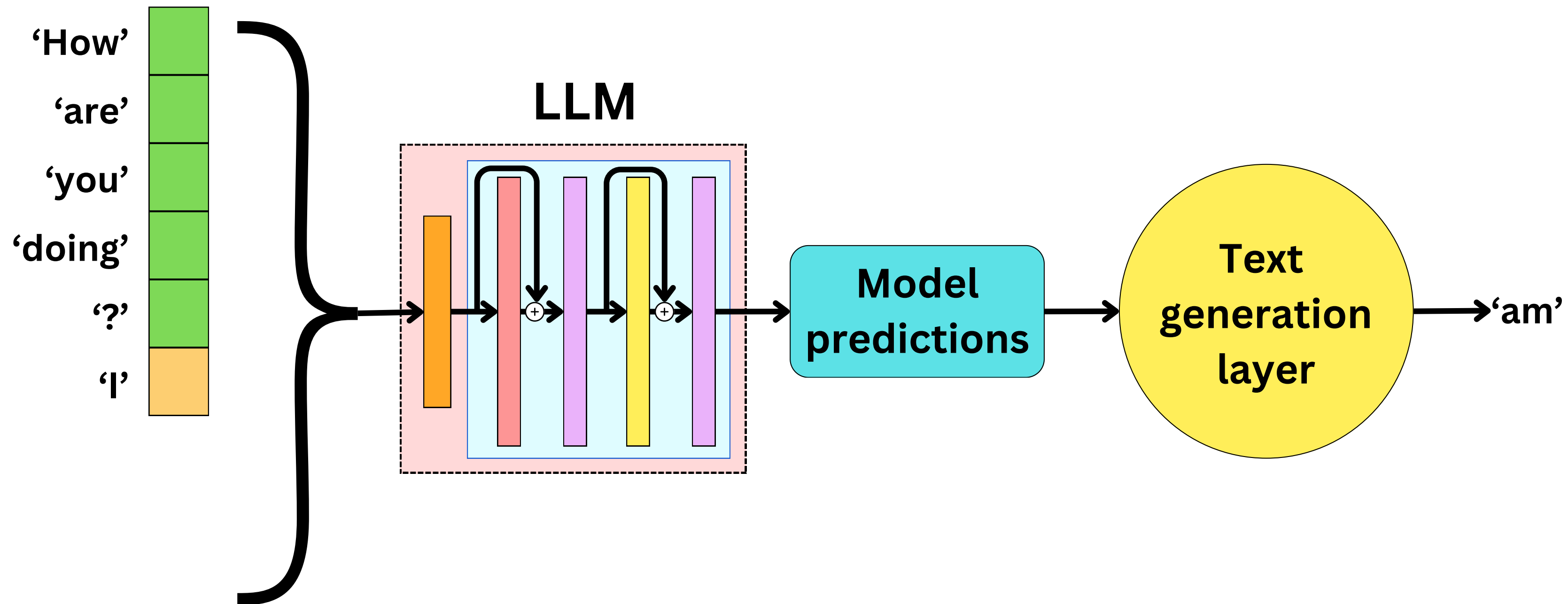


The Text Generation Layer



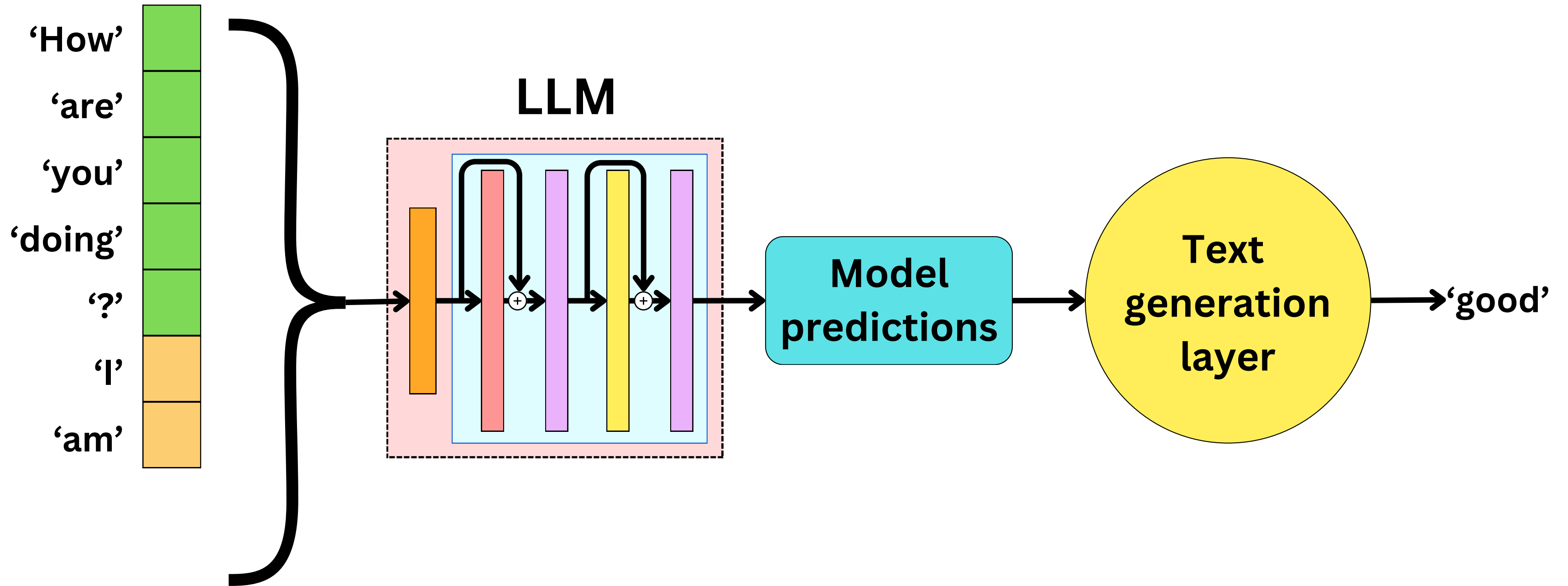


The Text Generation Layer



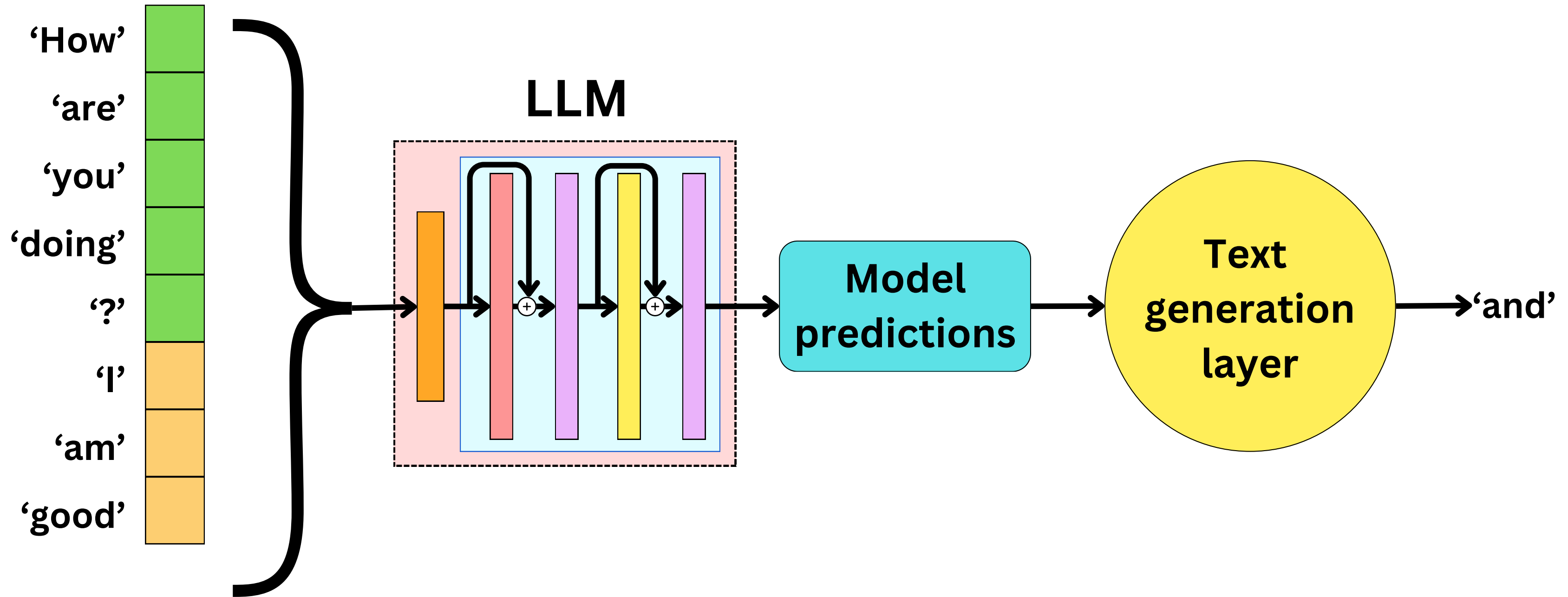


The Text Generation Layer



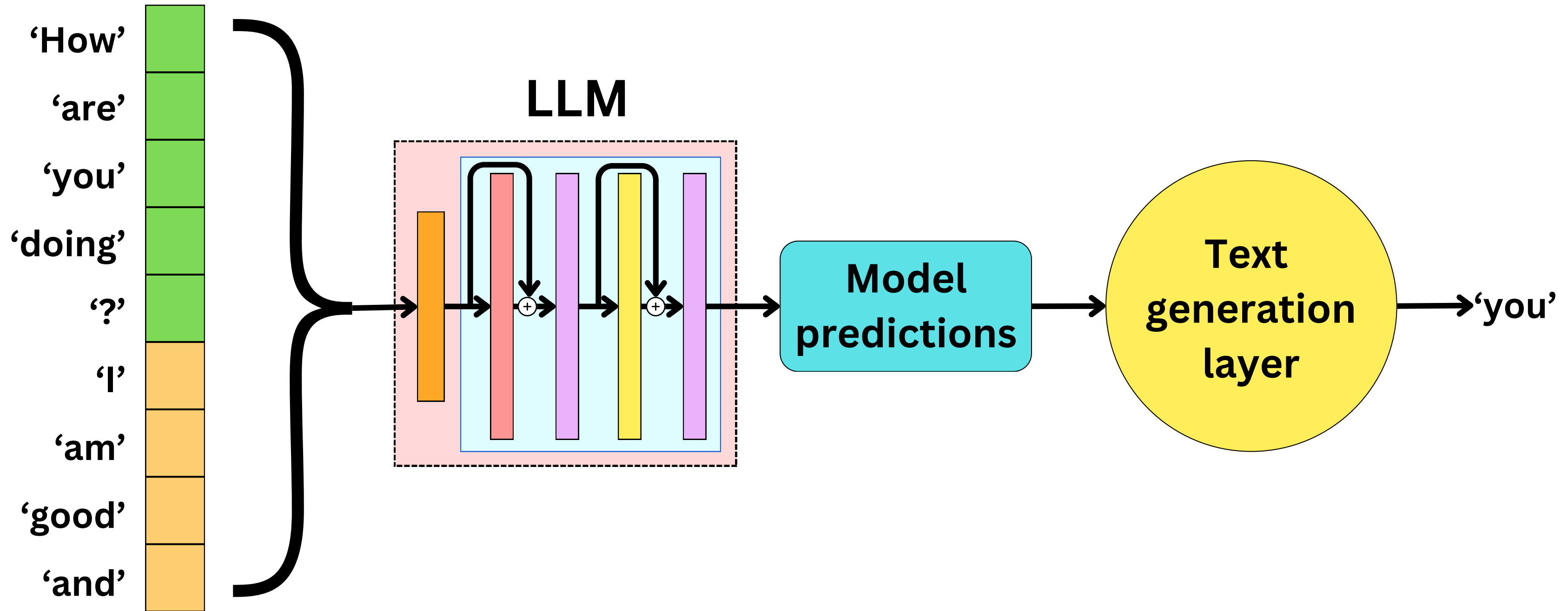


The Text Generation Layer



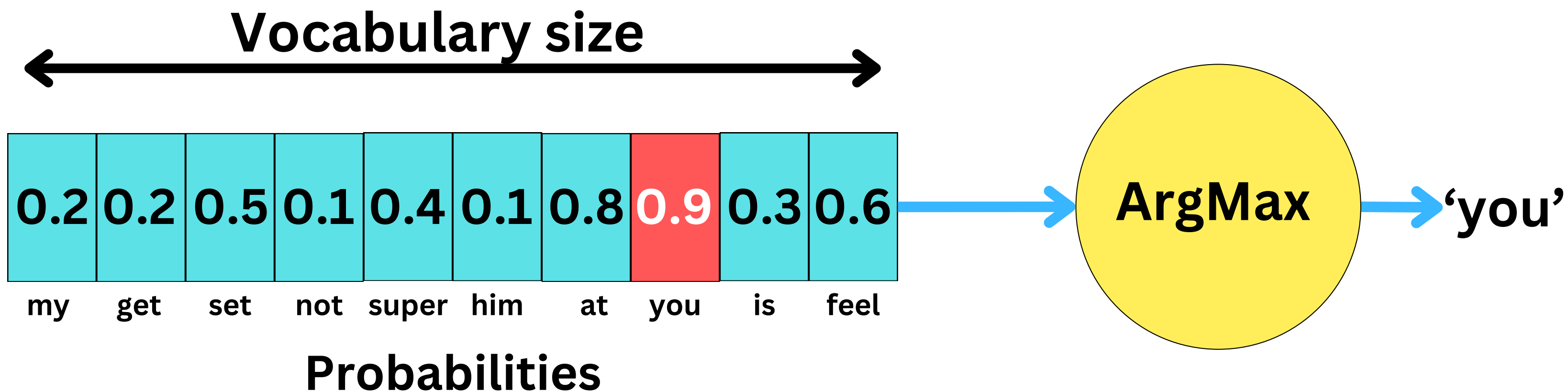


The Text Generation Layer



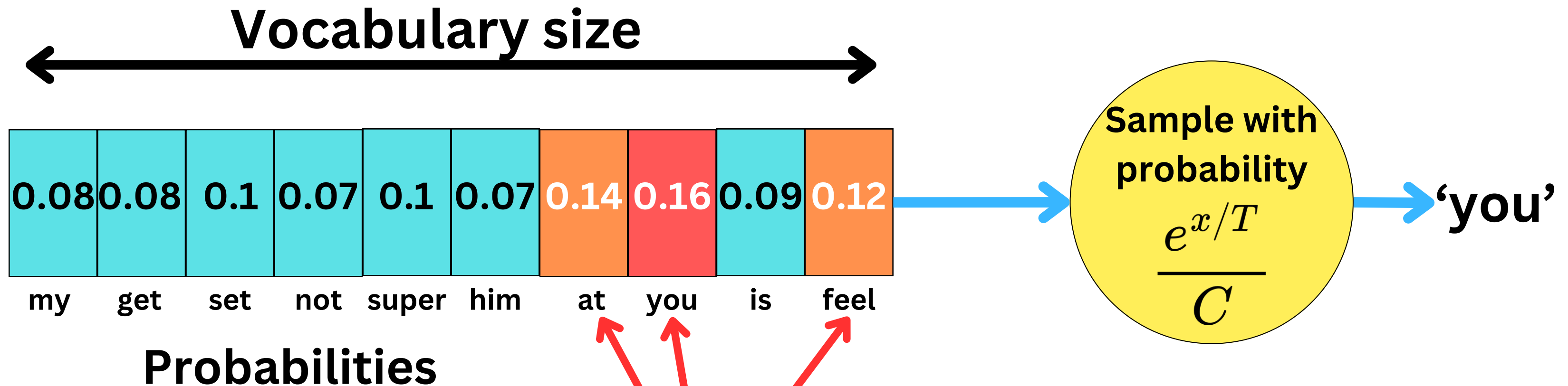


Greedy Decoding





Multinomial Sampling Generation

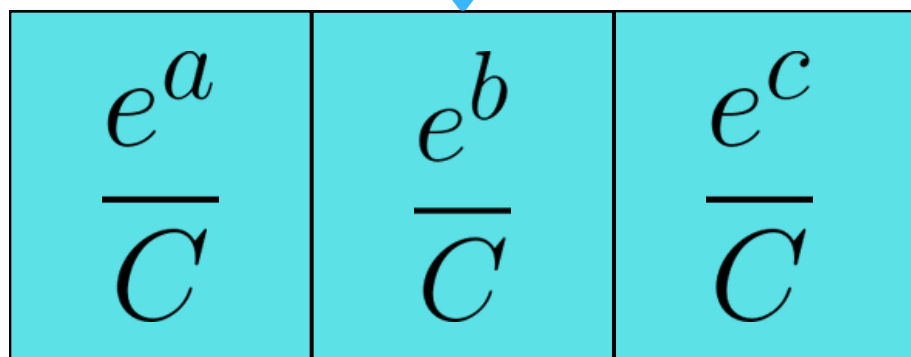
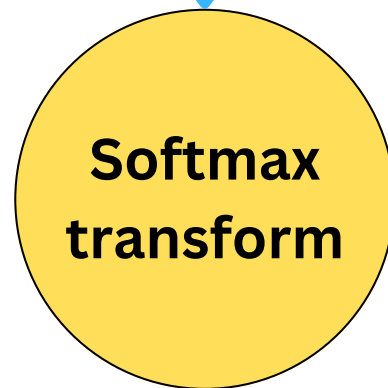
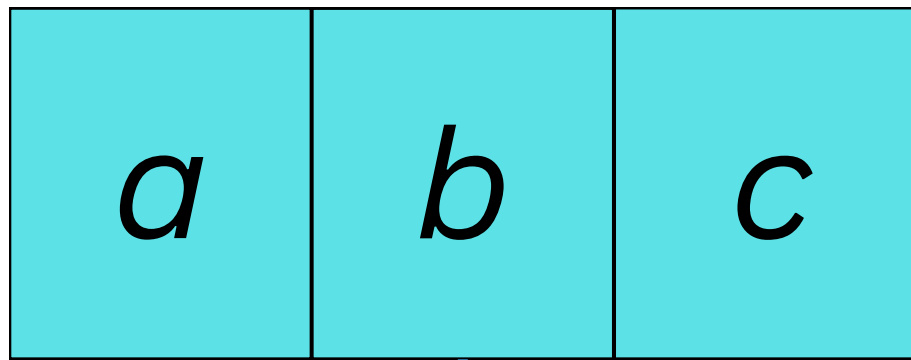


- 0.14 chance to be 'at'
- 0.16 chance to be 'you'
- 0.12 chance to be 'feel'



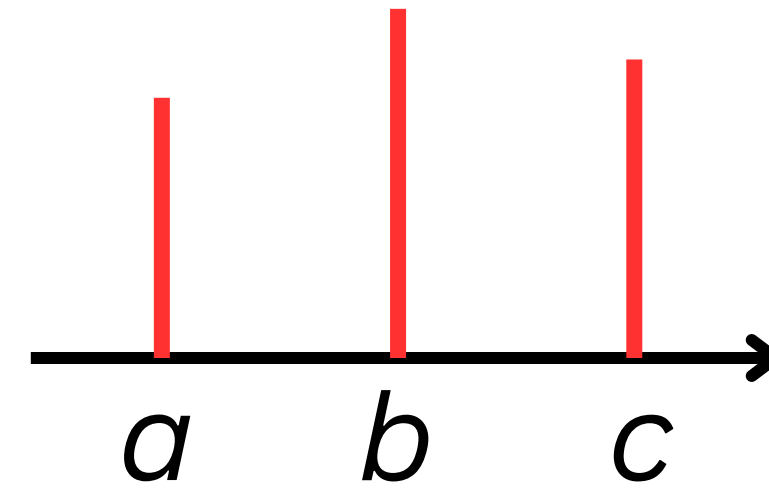
Multinomial Sampling Generation

Logits



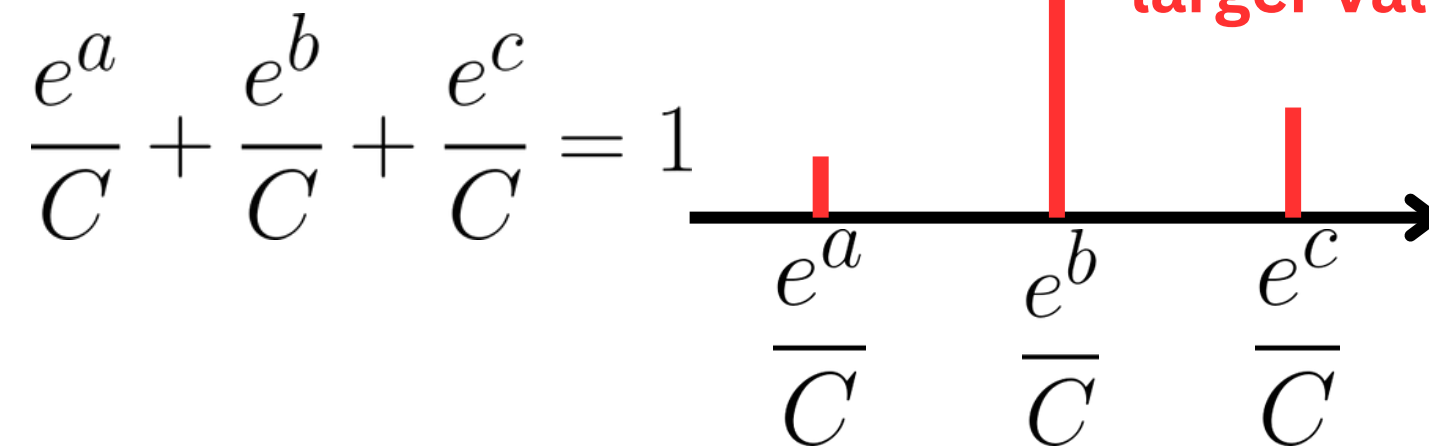
$$C = e^a + e^b + e^c$$

Probabilities



“Soft maximum”

Accentuate
larger values

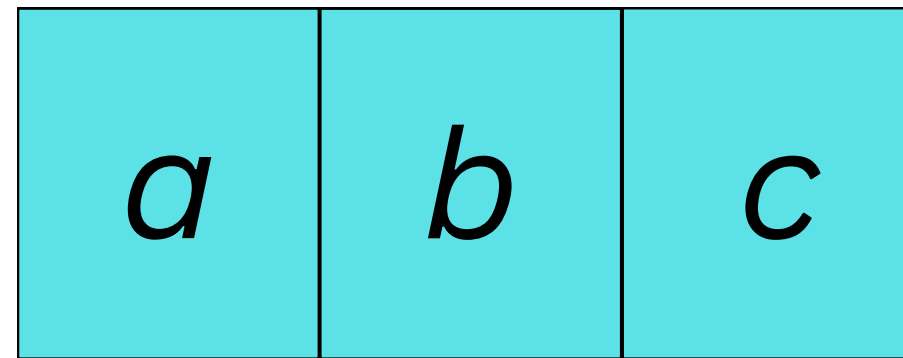


$$\frac{e^a}{C} + \frac{e^b}{C} + \frac{e^c}{C} = 1$$



Multinomial Sampling Generation

Logits

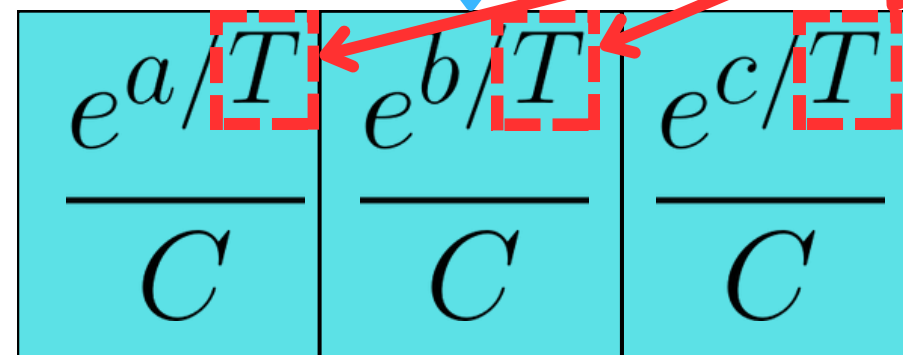


Softmax transform
with Temperature

Softmax
transform
with
Temperature

$$\frac{e^{a/T}}{C} + \frac{e^{b/T}}{C} + \frac{e^{c/T}}{C} = 1$$

Probabilities



Temperature

$$C = e^{a/T} + e^{b/T} + e^{c/T}$$



Multinomial Sampling Generation

$T = 0.1$

0.0	0.0	0.0	0.0	0.0	0.0	0.25	0.69	0.0	0.03
my	get	set	not	super	him	at	you	is	feel

Almost
greedy
search

$T = 1$

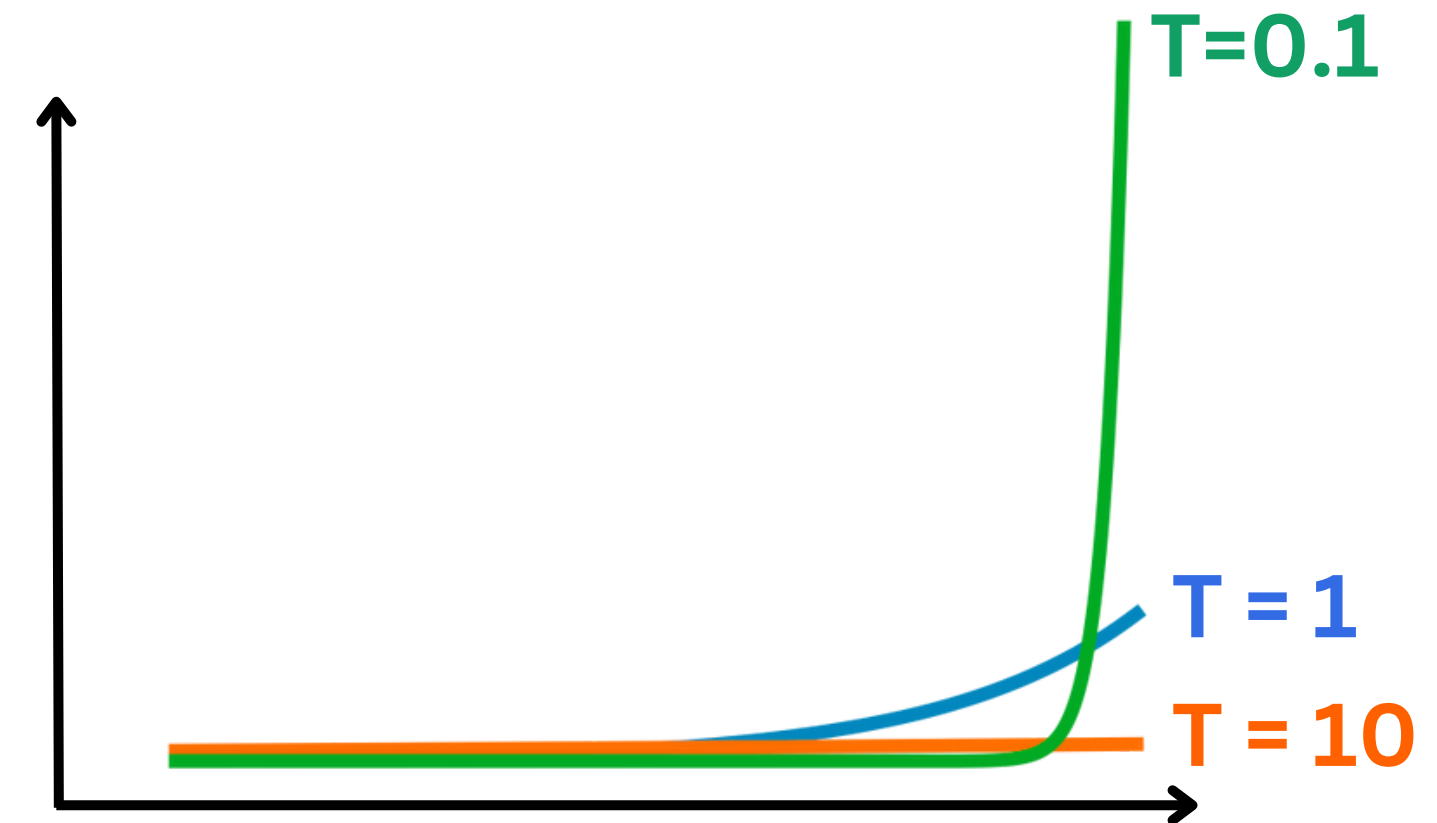
0.08	0.08	0.1	0.07	0.1	0.07	0.14	0.16	0.09	0.12
my	get	set	not	super	him	at	you	is	feel

In
between

$T = 10$

0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1
my	get	set	not	super	him	at	you	is	feel

Almost
uniformly
random





Multinomial Sampling Generation

Greedy search or multinomial sampling search

Iteration 1

$P(\text{token 1} \mid \text{prompt})$

Iteration 2

$P(\text{token 2} \mid \text{prompt}, \text{token 1})$

Iteration 3

$P(\text{token 3} \mid \text{prompt}, \text{token 1}, \text{token 2})$

...

...

Iteration N

$P(\text{token N} \mid \text{prompt}, \text{token 1}, \dots, \text{token N - 1})$



Beam Search Generation

But this is not the probability we care about!

$P(\text{Output sequence} \mid \text{prompt})$

$P(\text{token 1, ..., token N} \mid \text{prompt})$



This is the probability we want to maximize



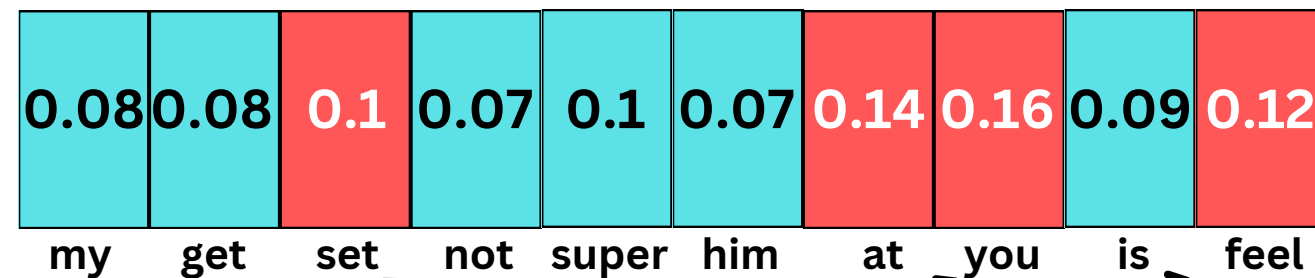
Beam Search Generation

$$\begin{aligned} & P(\text{token 1, ..., token N} \mid \text{prompt}) \\ &= P(\text{token N} \mid \text{prompt, token 1, ..., token N - 1}) \\ &\quad \times P(\text{token 1, ..., token N - 1} \mid \text{prompt}) \\ &= P(\text{token N} \mid \text{prompt, token 1, ..., token N - 1}) \\ &\times P(\text{token N - 1} \mid \text{prompt, token 1, ..., token N - 2}) \\ &\quad \times P(\text{token 1, ..., token N - 2} \mid \text{prompt}) \\ &= P(\text{token N} \mid \text{prompt, token 1, ..., token N - 1}) \\ &\times P(\text{token N - 1} \mid \text{prompt, token 1, ..., token N - 2}) \\ &\quad \dots \\ &\quad \times P(\text{token 2} \mid \text{prompt, token 1}) \\ &\quad \times P(\text{token 1} \mid \text{prompt}) \end{aligned}$$



Beam Search Generation

First Iteration

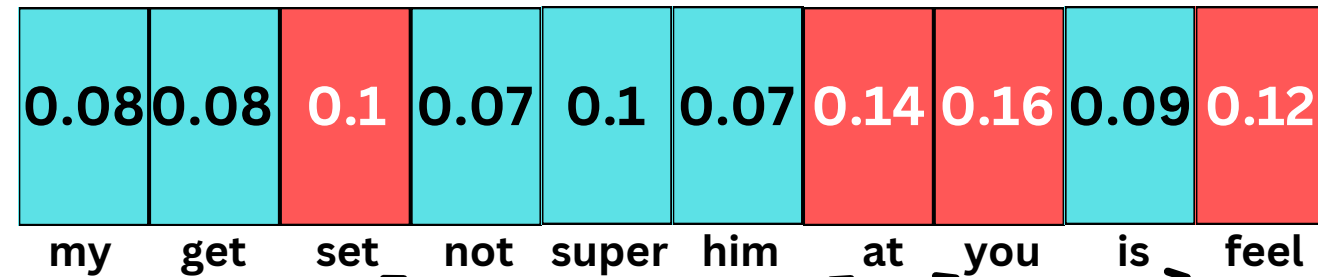


Beam size =4



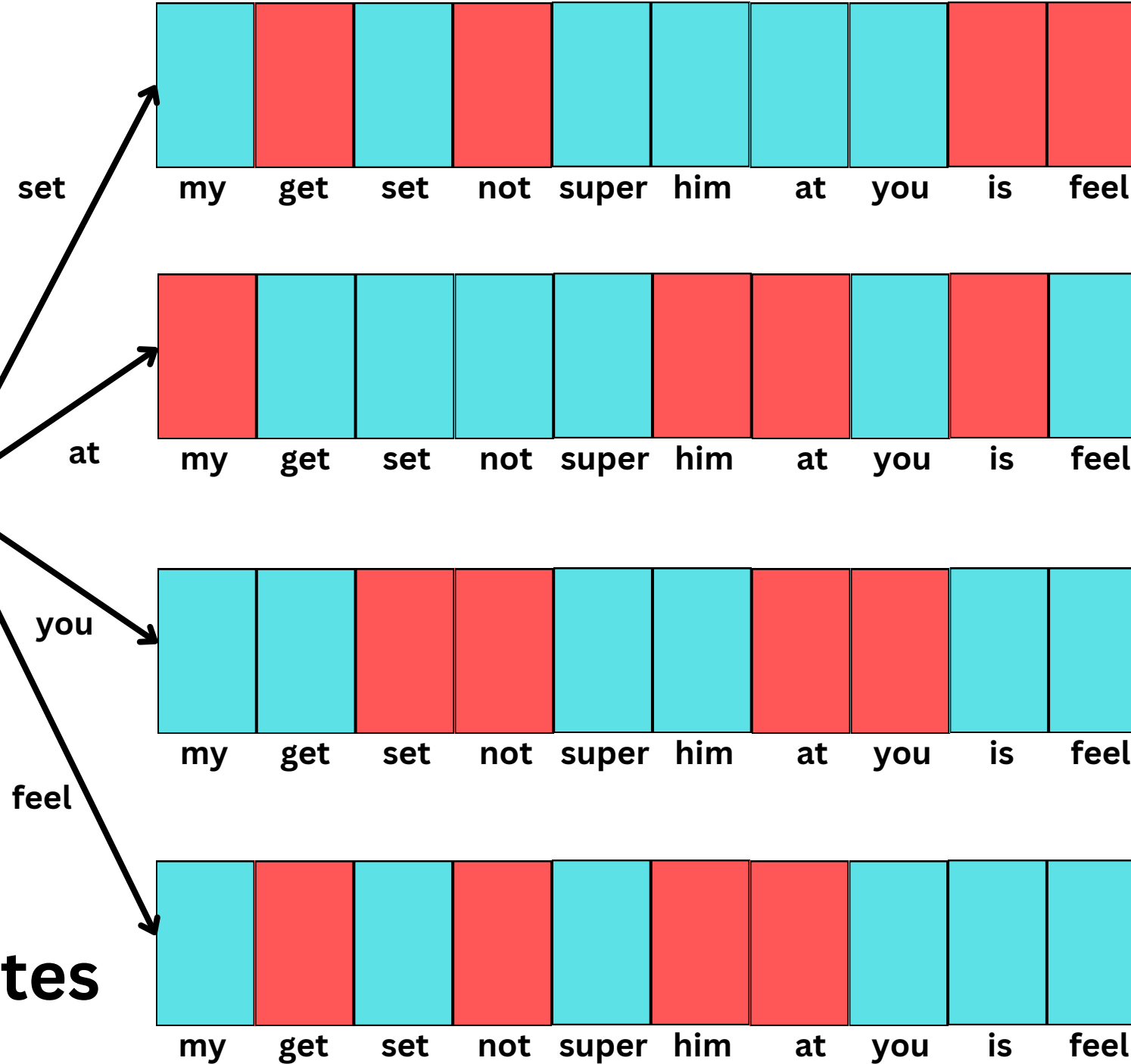
Beam Search Generation

First Iteration



Beam size =4

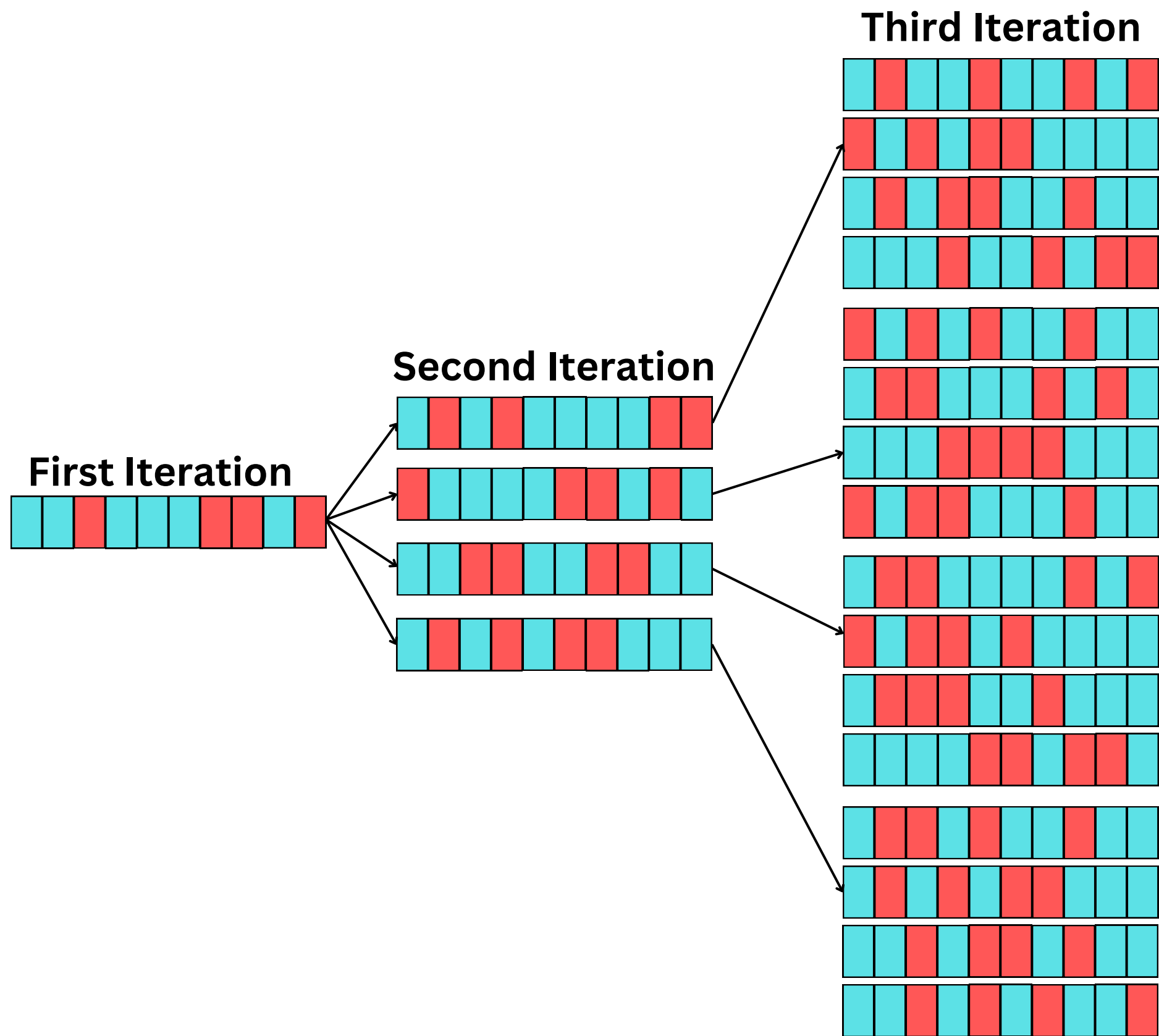
Second Iteration



4 candidates



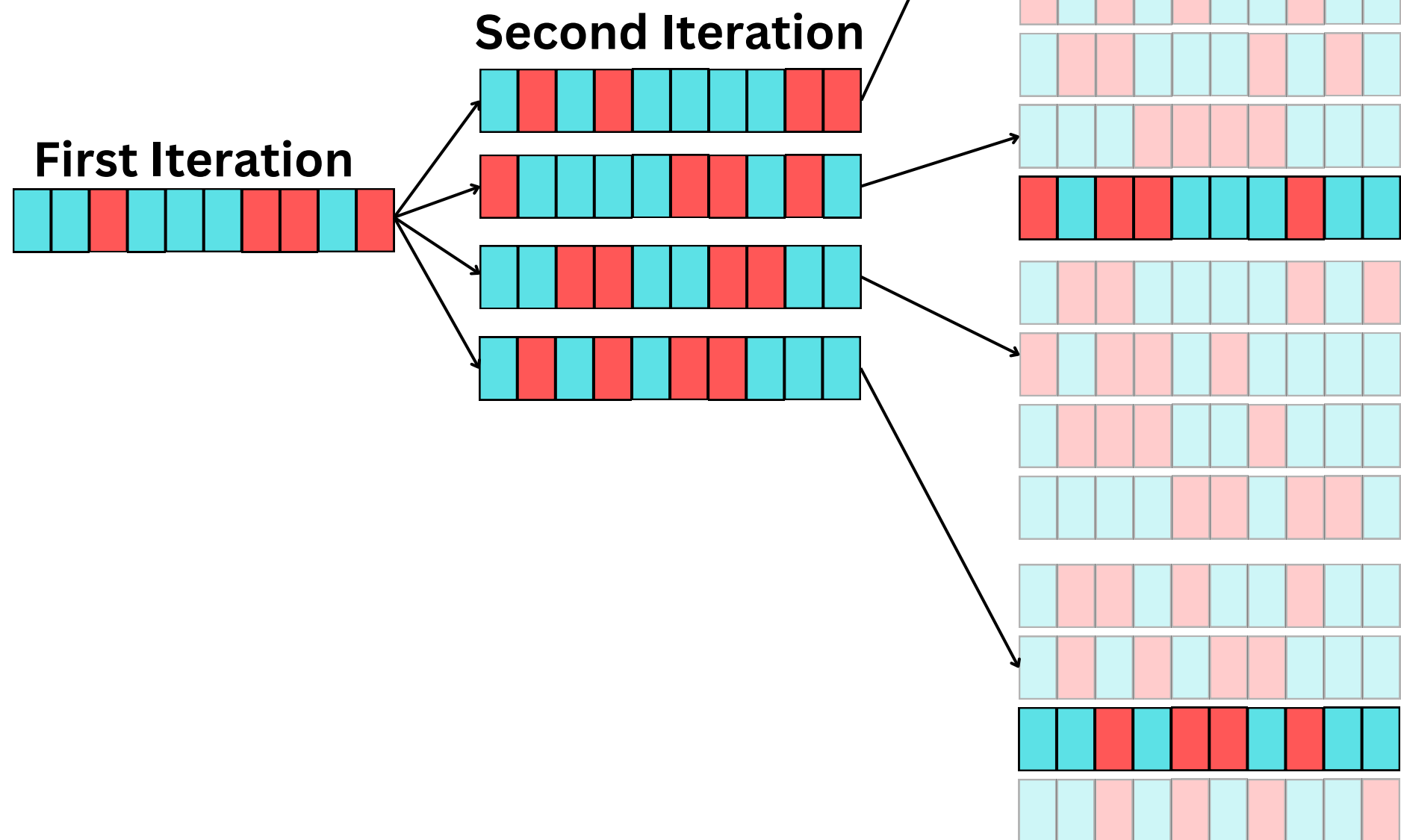
Beam Search Generation





Beam Search Generation

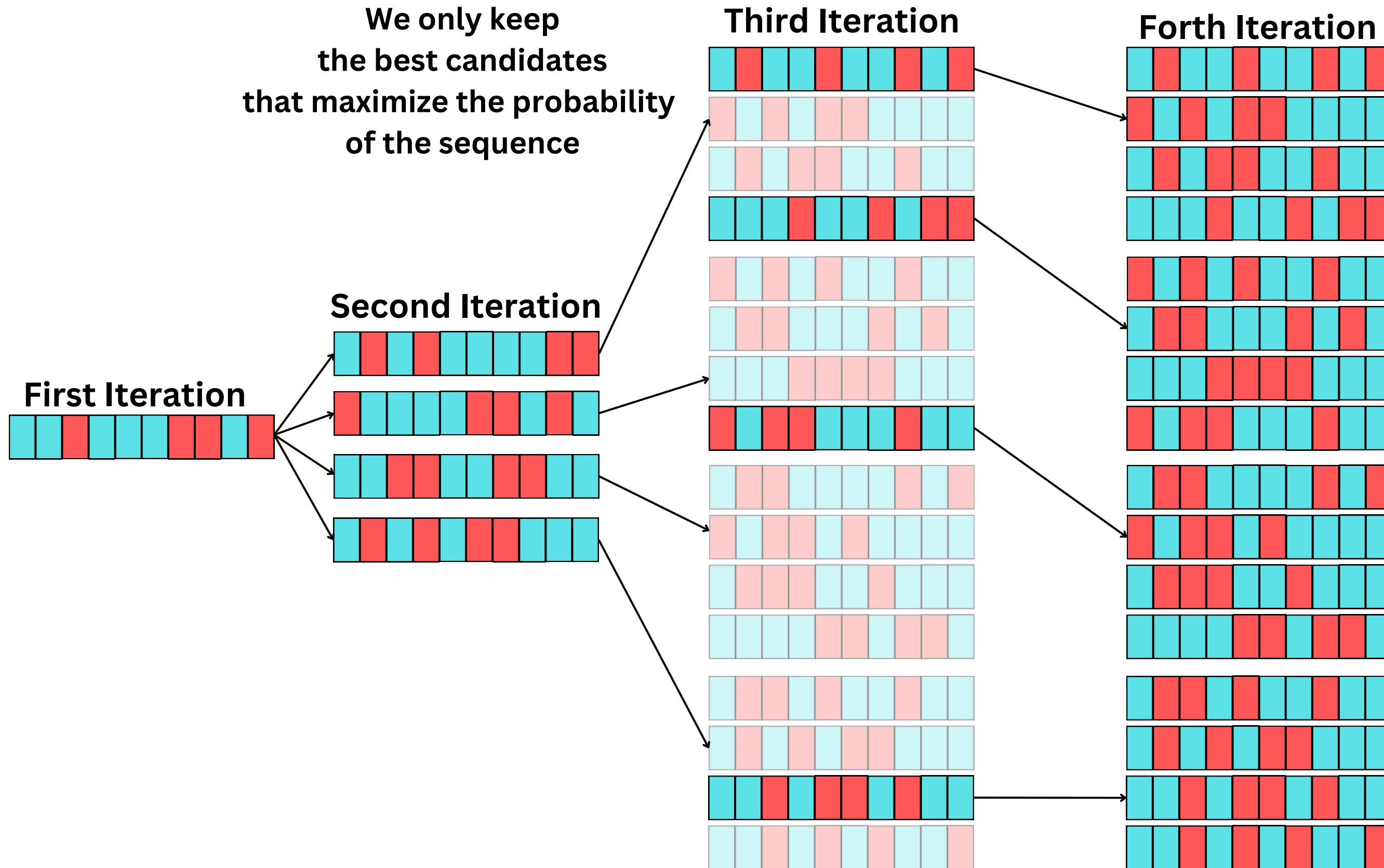
We only keep
the best candidates
that maximize the probability
of the sequence





Beam Search Generation

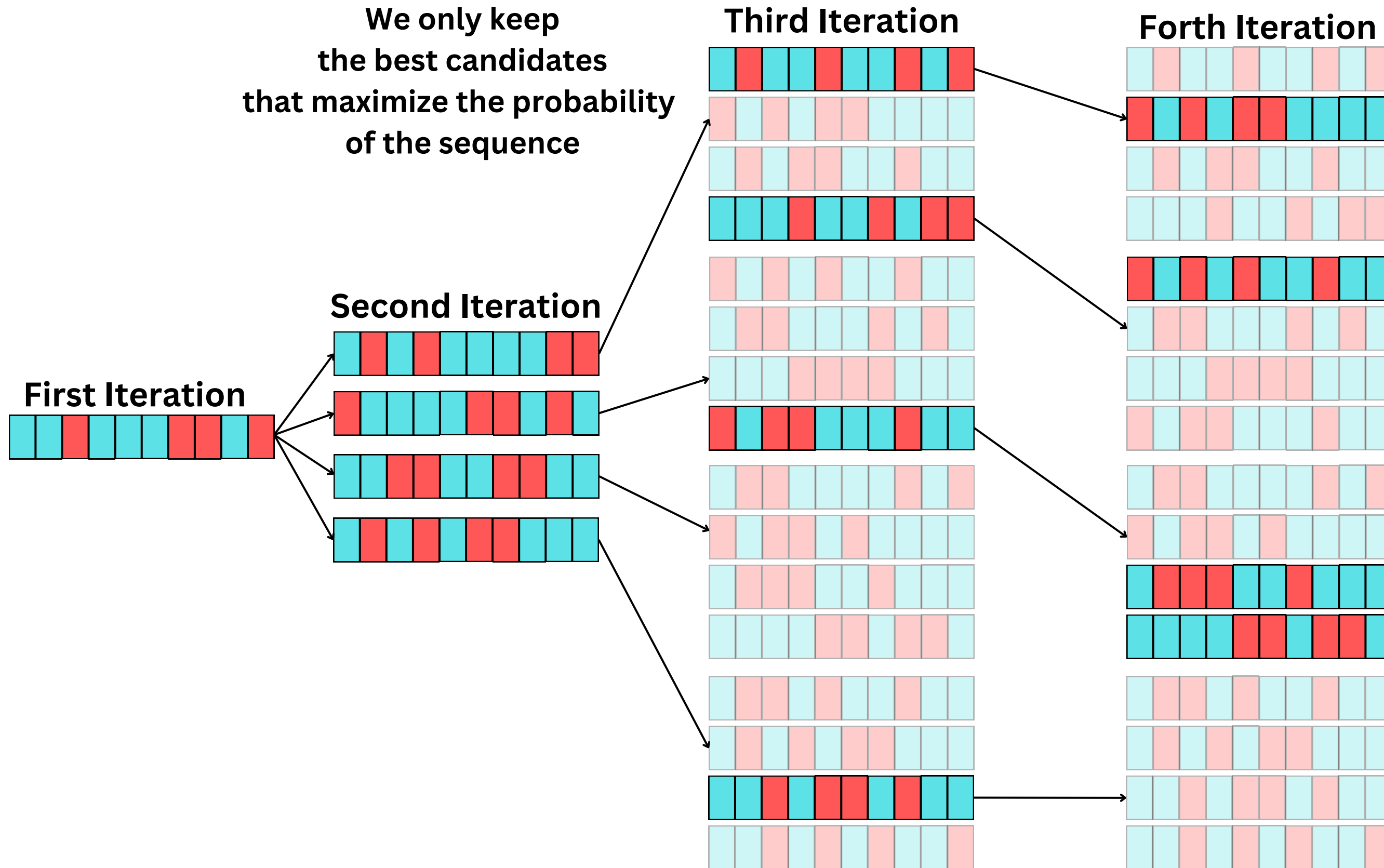
We only keep
the best candidates
that maximize the probability
of the sequence





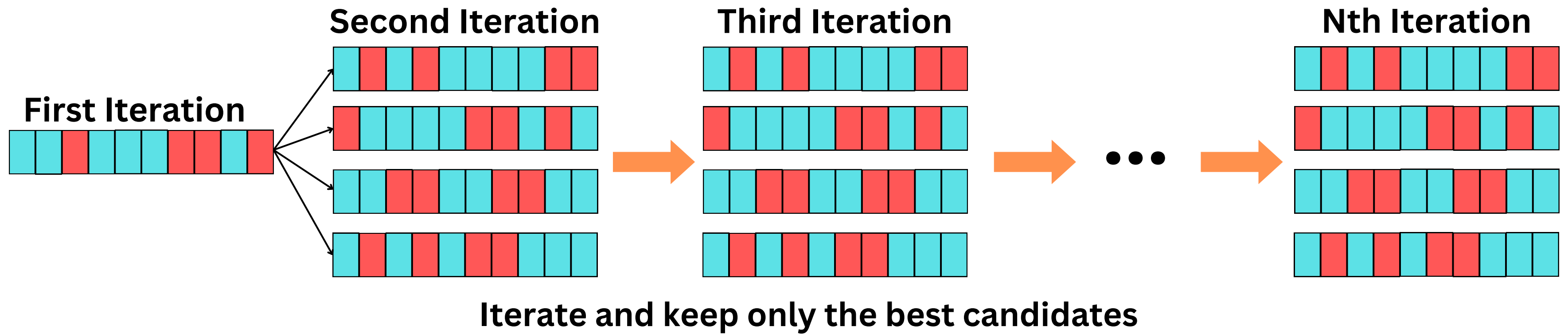
Beam Search Generation

We only keep
the best candidates
that maximize the probability
of the sequence



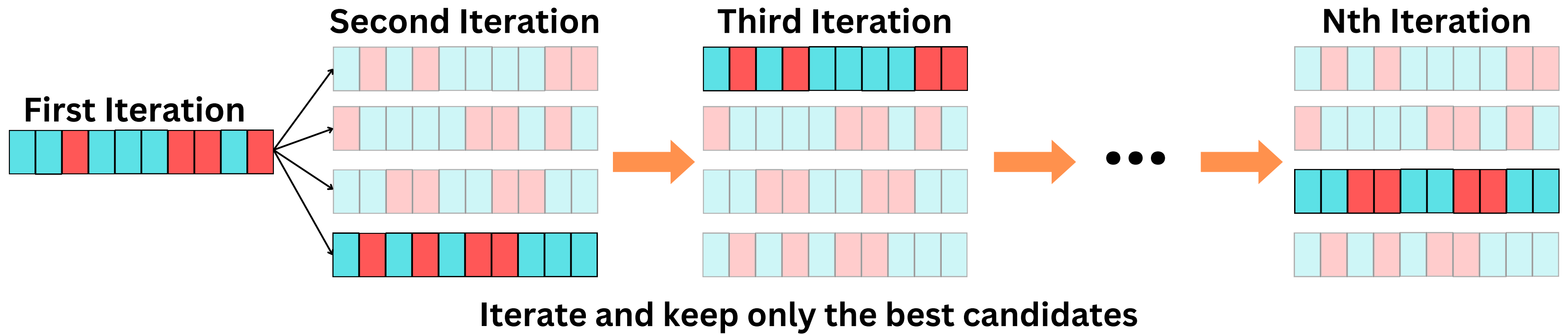


Beam Search Generation





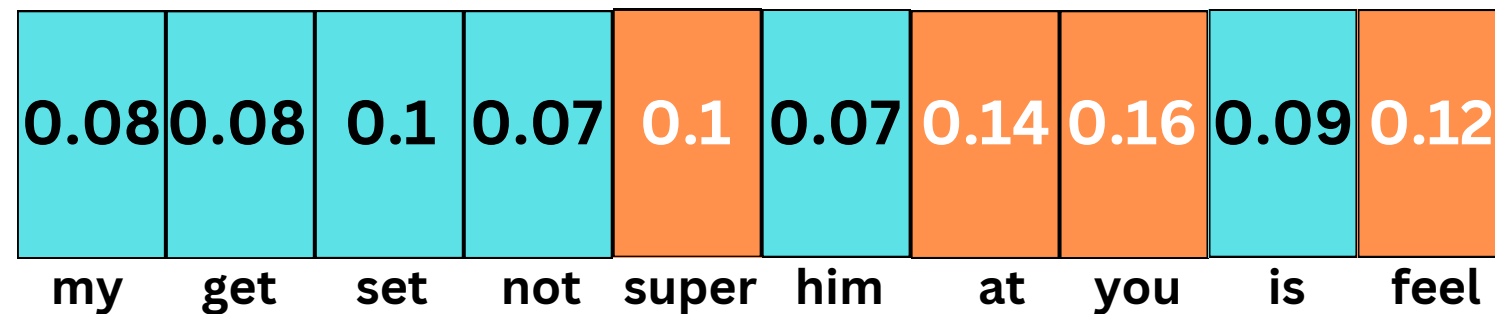
Beam Search Generation





Contrastive Search Generation

First Iteration



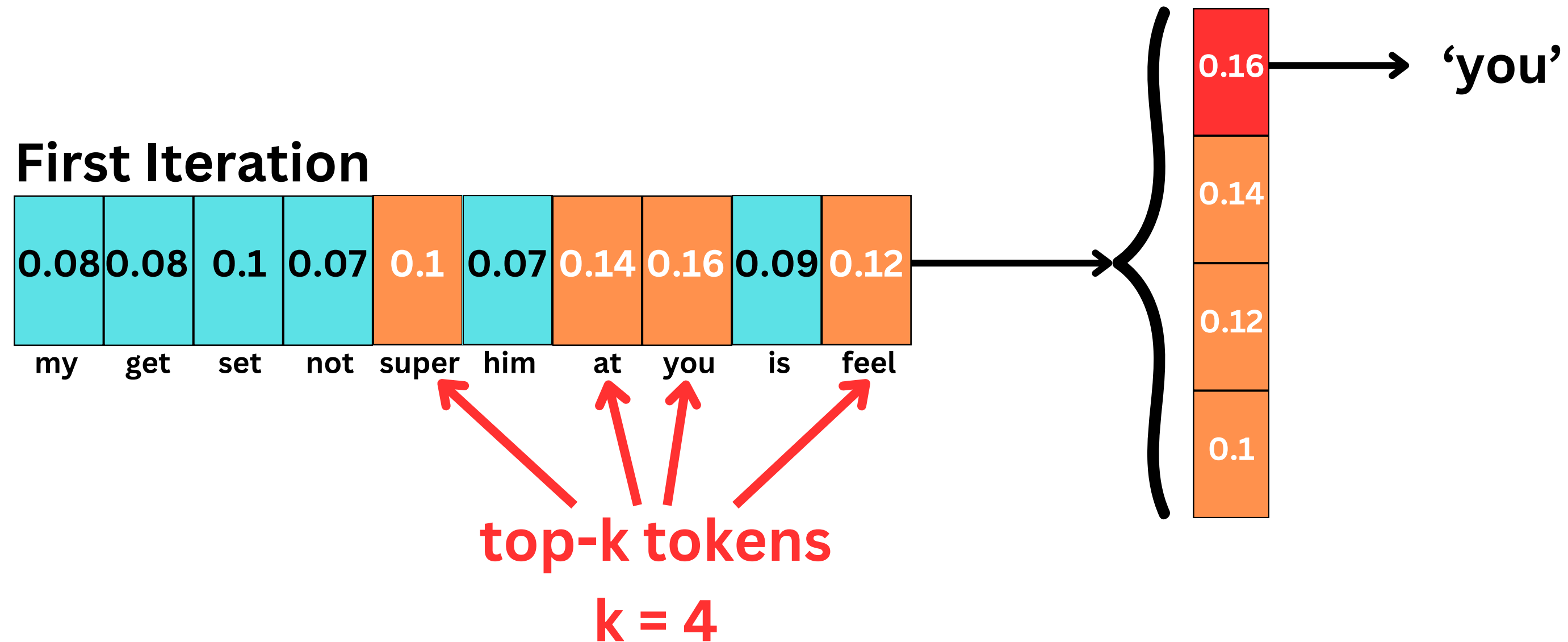
top-k tokens

k = 4



Contrastive Search Generation

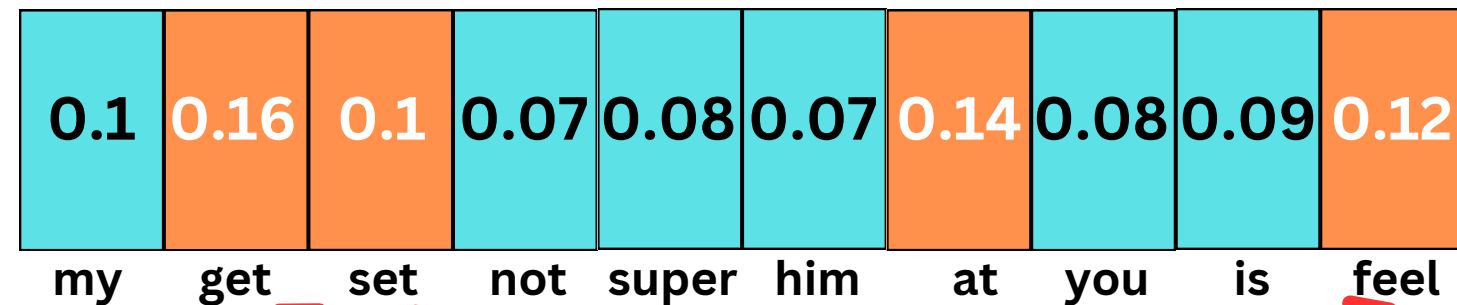
First Iteration





Contrastive Search Generation

Second Iteration



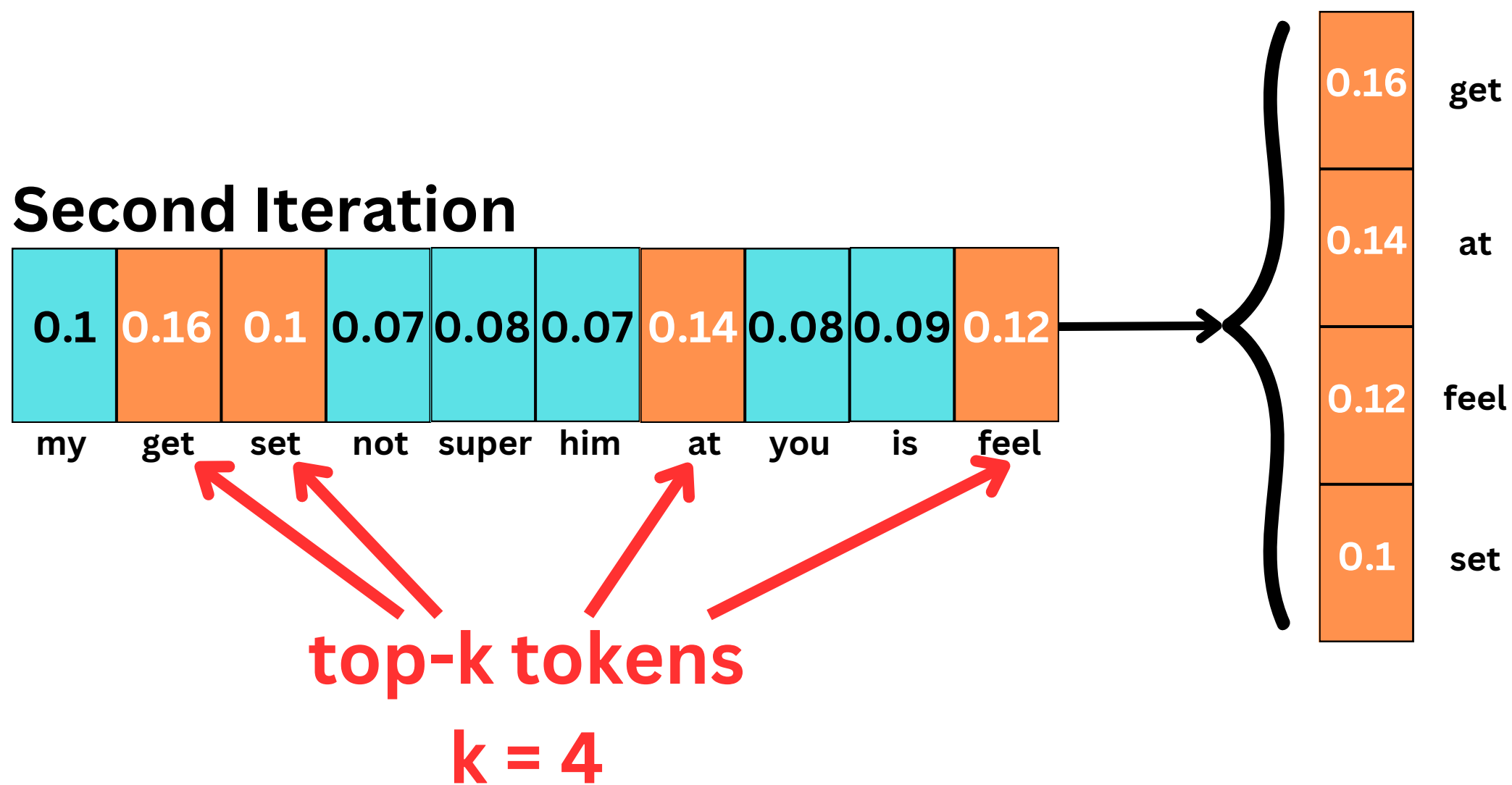
top-k tokens

k = 4



Contrastive Search Generation

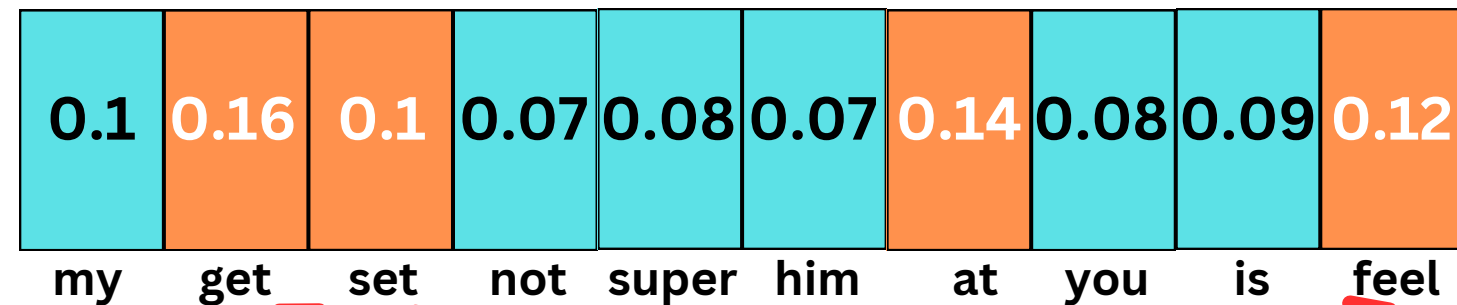
Second Iteration



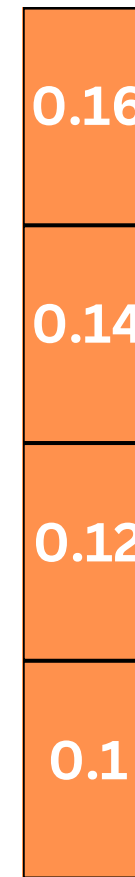


Contrastive Search Generation

Second Iteration



top-k tokens
k = 4



get

- $\cos(\text{get}, \text{you})$

at

- $\cos(\text{at}, \text{you})$

feel

- $\cos(\text{feel}, \text{you})$

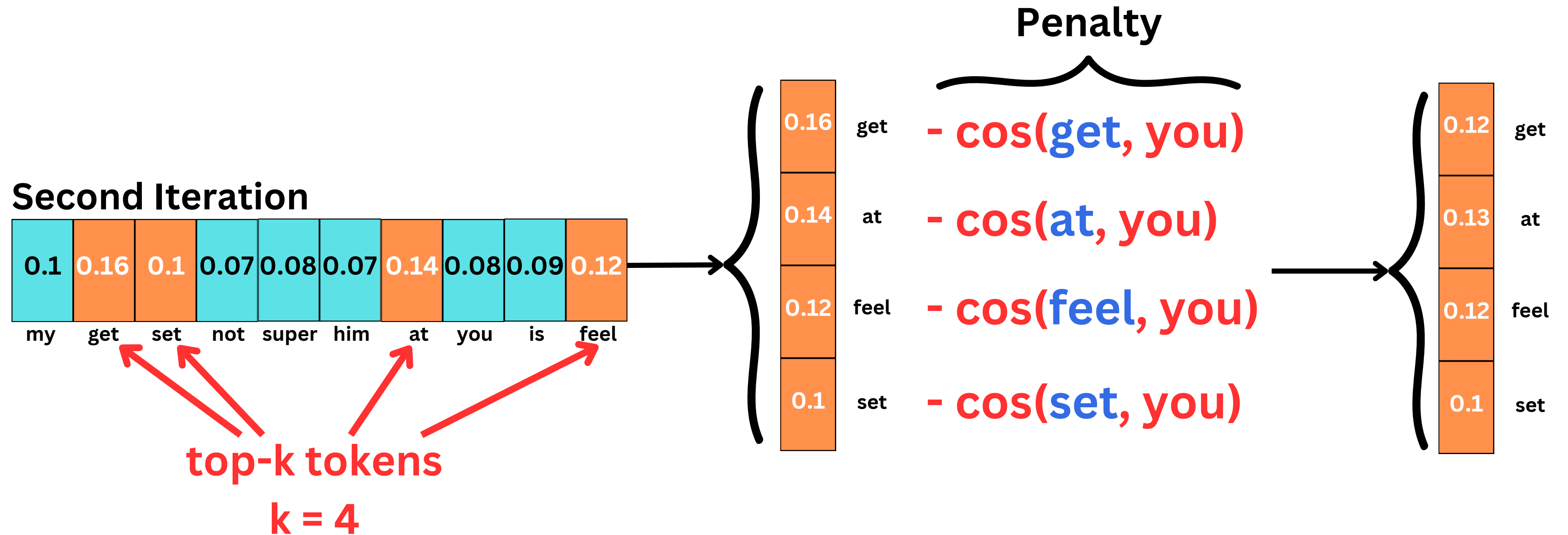
set

- $\cos(\text{set}, \text{you})$

Penalty



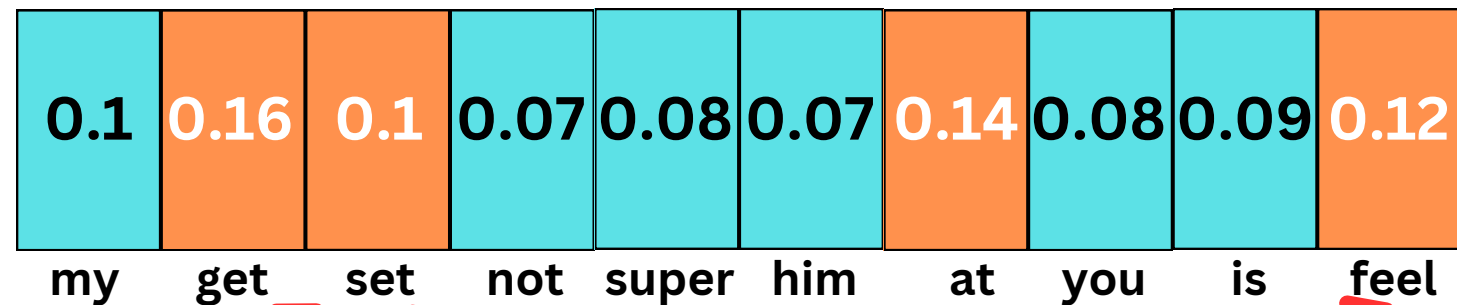
Contrastive Search Generation



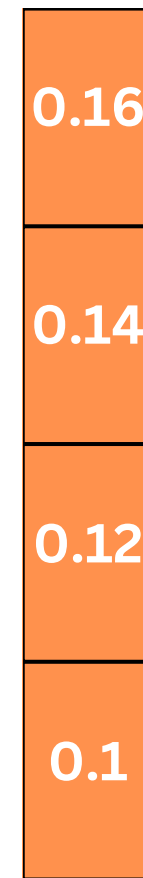


Contrastive Search Generation

Second Iteration



top-k tokens
k = 4



get

at

feel

set

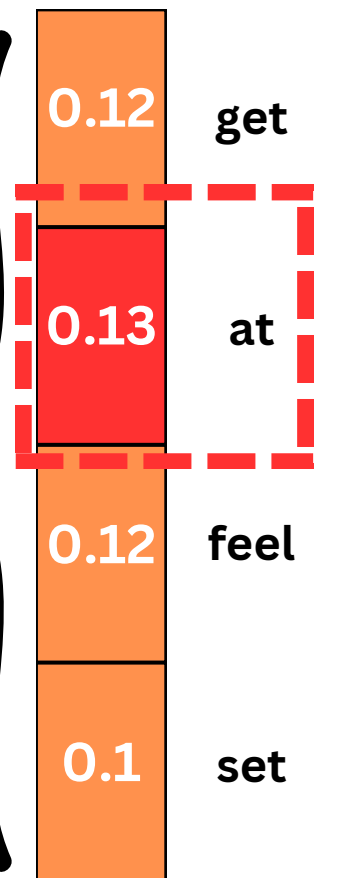
- $\cos(\text{get}, \text{you})$

- $\cos(\text{at}, \text{you})$

- $\cos(\text{feel}, \text{you})$

- $\cos(\text{set}, \text{you})$

Penalty



get

at

feel

set



Contrastive Search Generation

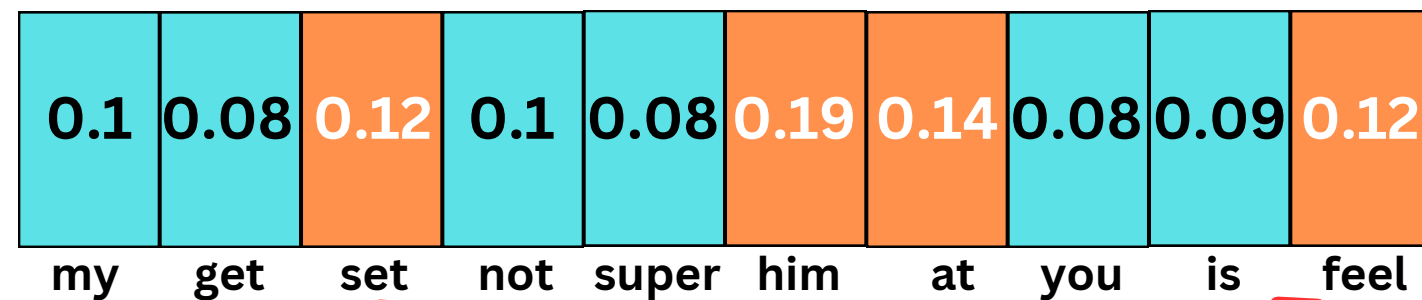
Third Iteration

0.1	0.08	0.12	0.1	0.08	0.19	0.14	0.08	0.09	0.12
my	get	set	not	super	him	at	you	is	feel



Contrastive Search Generation

Third Iteration



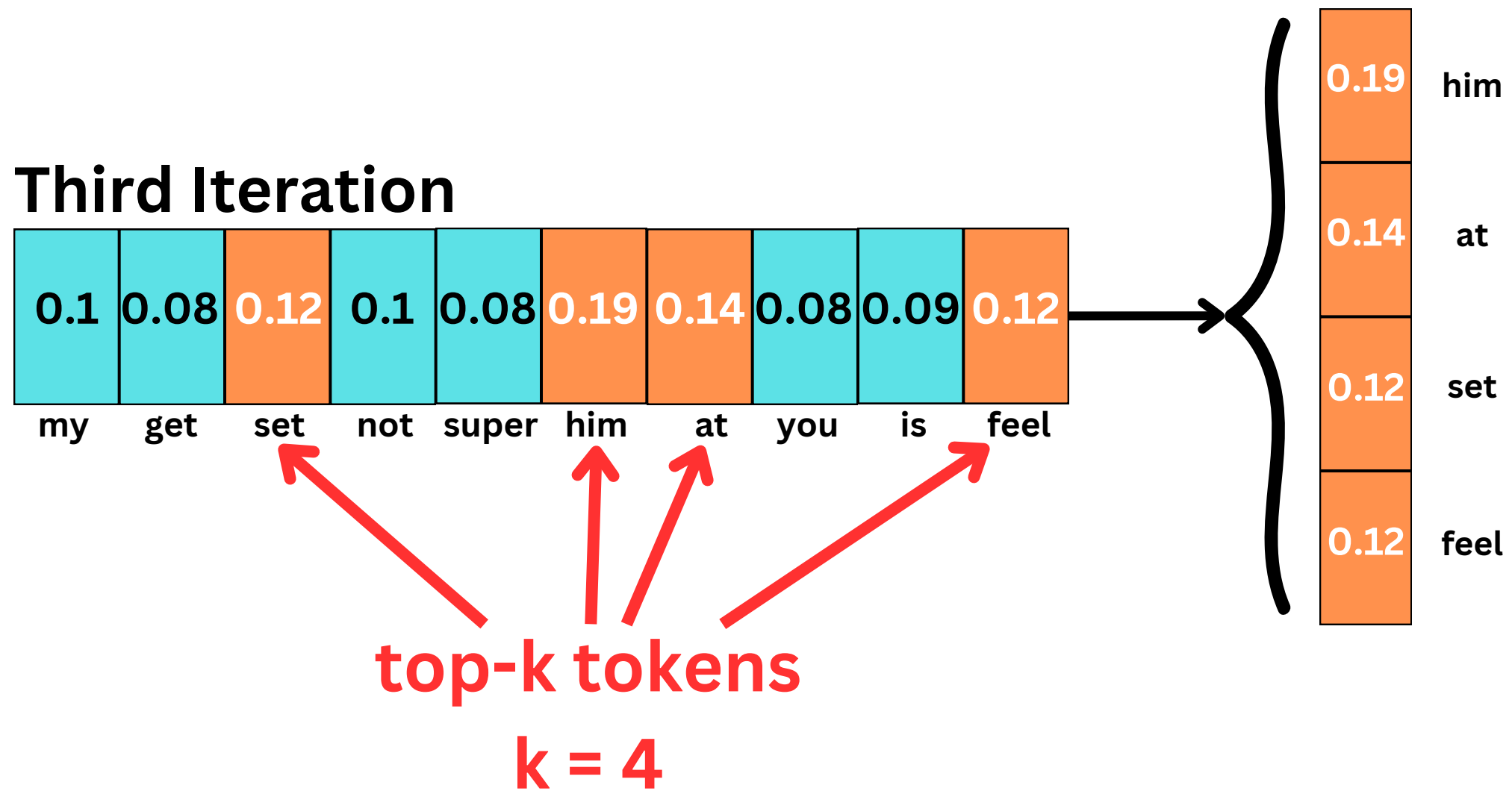
top-k tokens

k = 4



Contrastive Search Generation

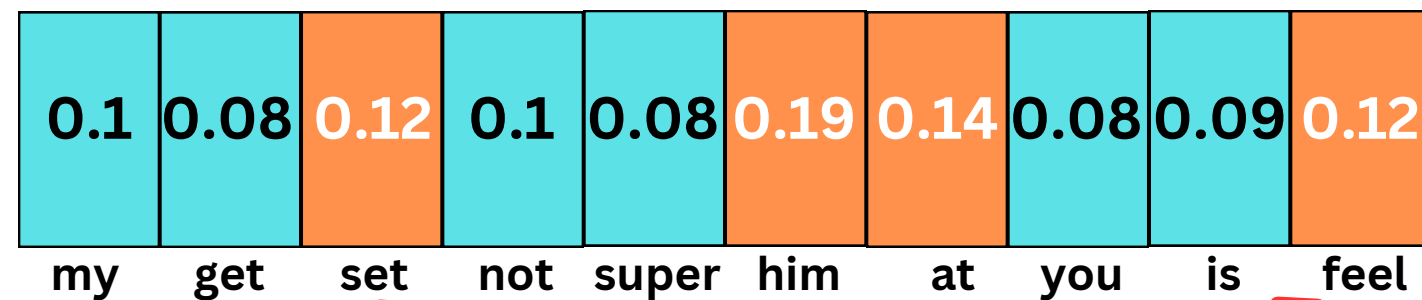
Third Iteration





Contrastive Search Generation

Third Iteration



top-k tokens
k = 4



Penalty

him - $\max(\cos(\text{him}, \text{at}), \cos(\text{him}, \text{you}))$

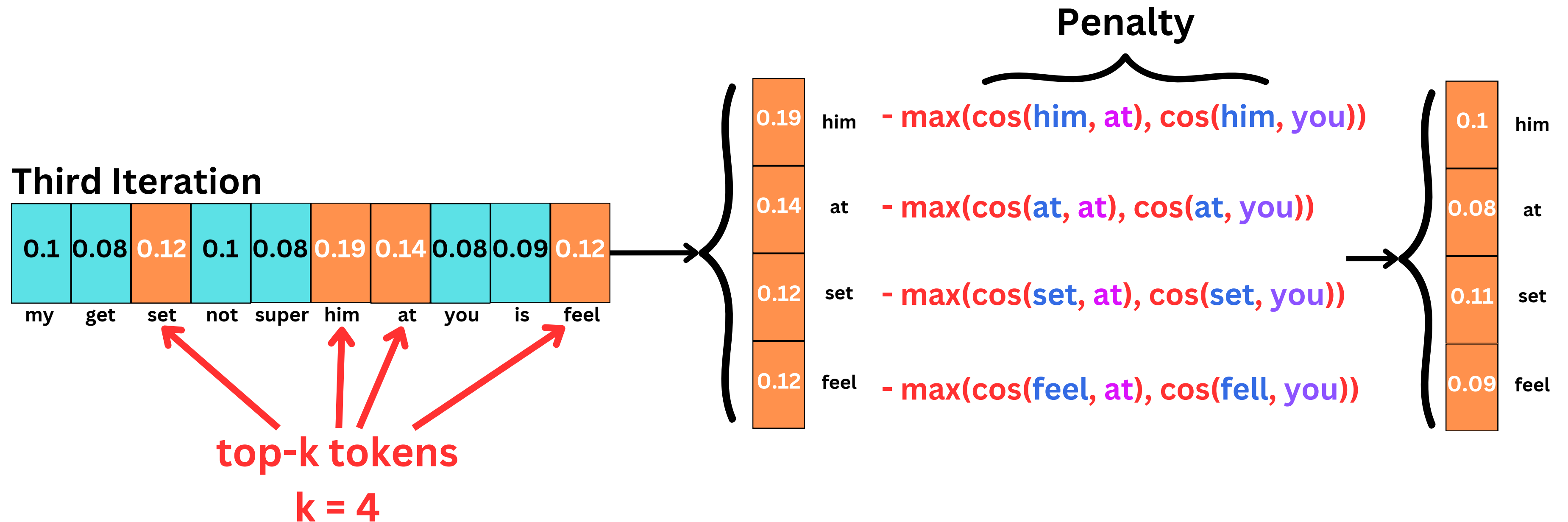
at - $\max(\cos(\text{at}, \text{at}), \cos(\text{at}, \text{you}))$

set - $\max(\cos(\text{set}, \text{at}), \cos(\text{set}, \text{you}))$

feel - $\max(\cos(\text{feel}, \text{at}), \cos(\text{feel}, \text{you}))$

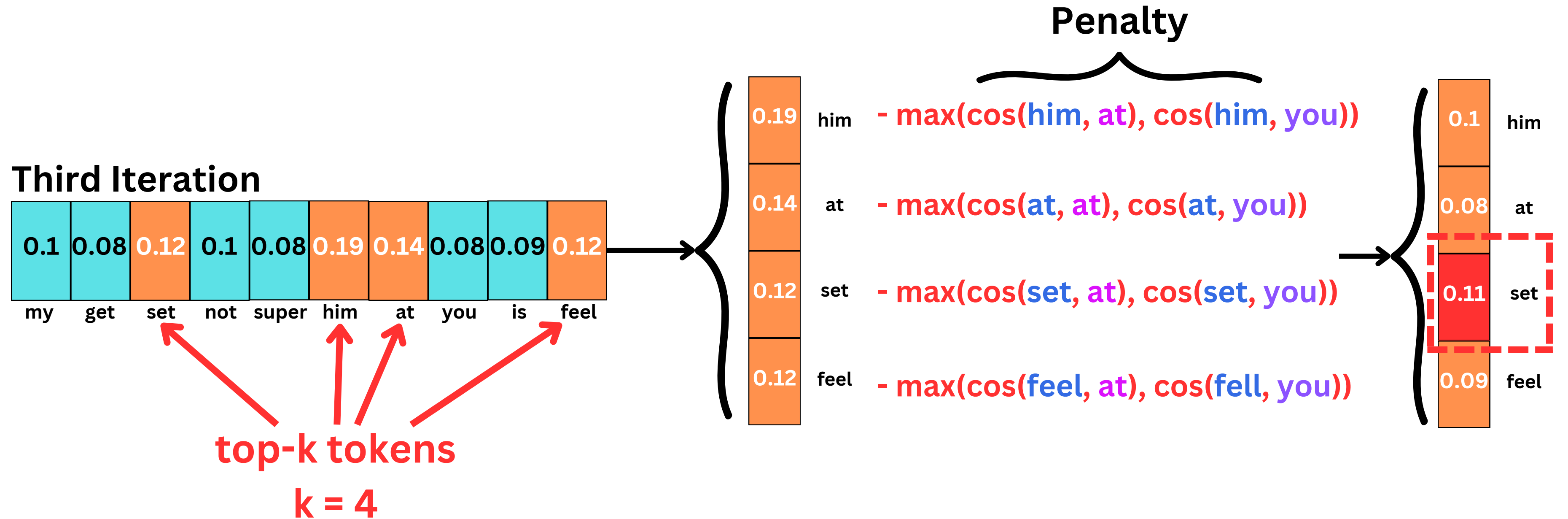


Contrastive Search Generation



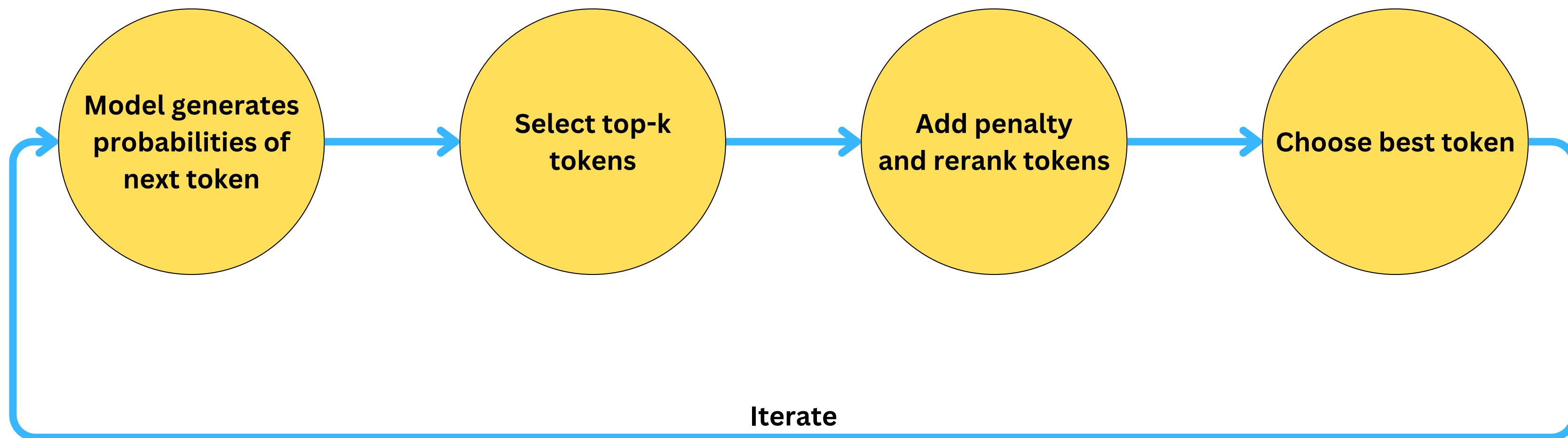


Contrastive Search Generation



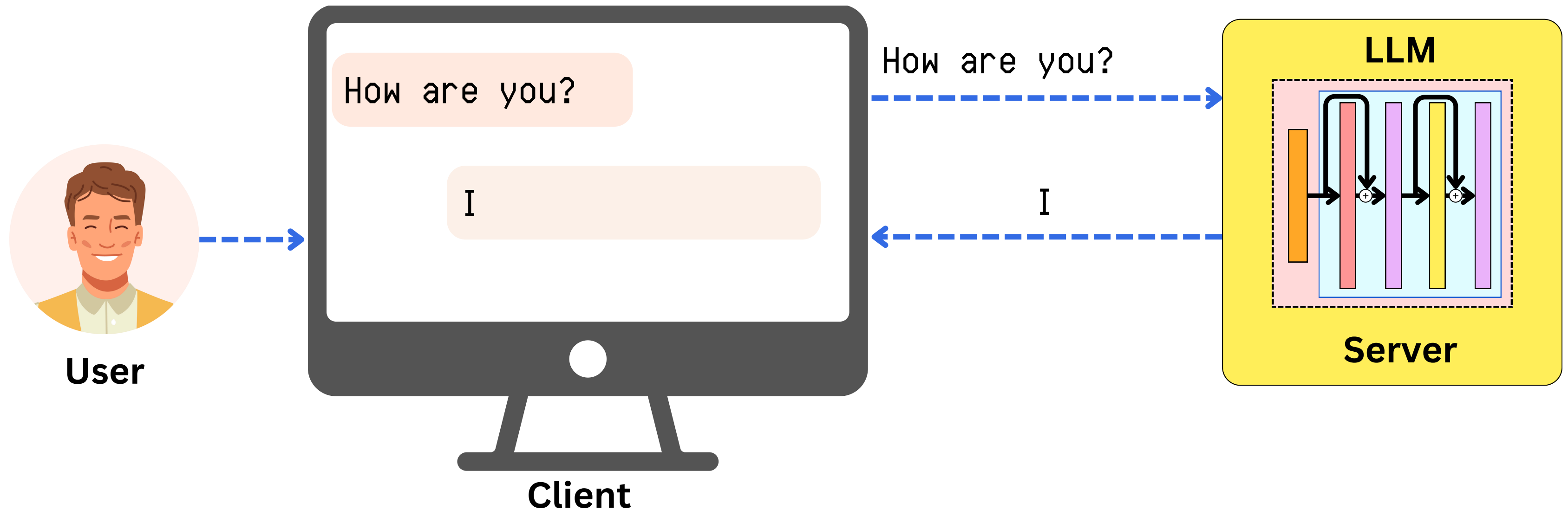


Contrastive Search Generation



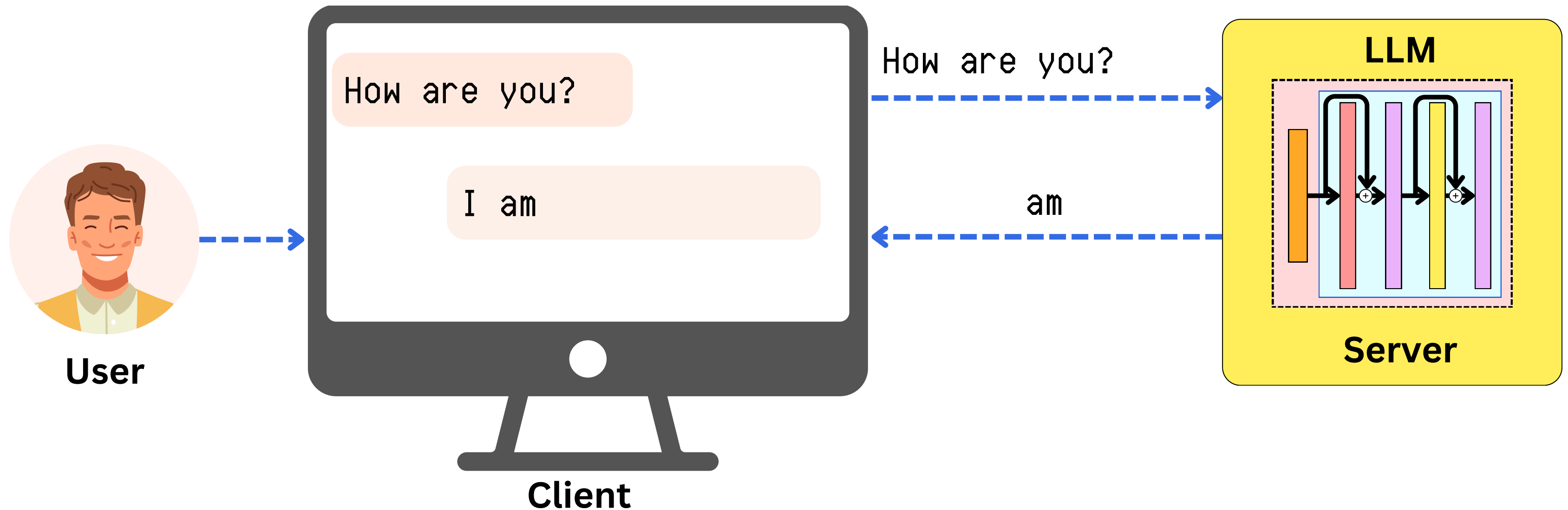


Streaming Text Generation



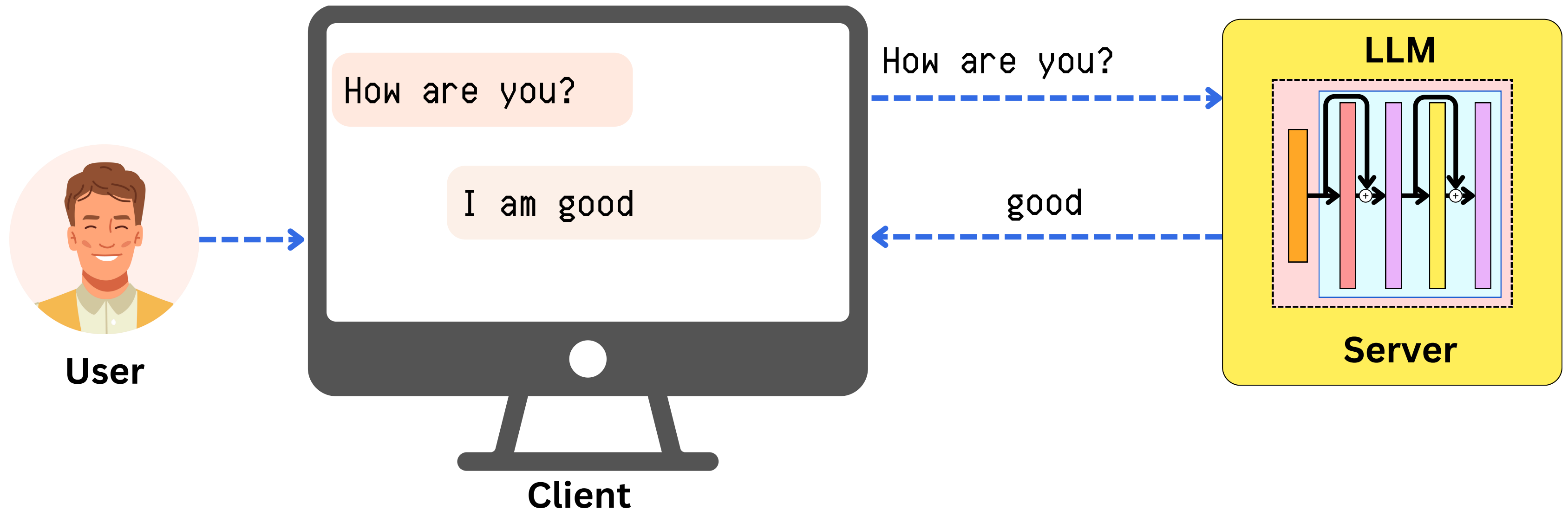


Streaming Text Generation



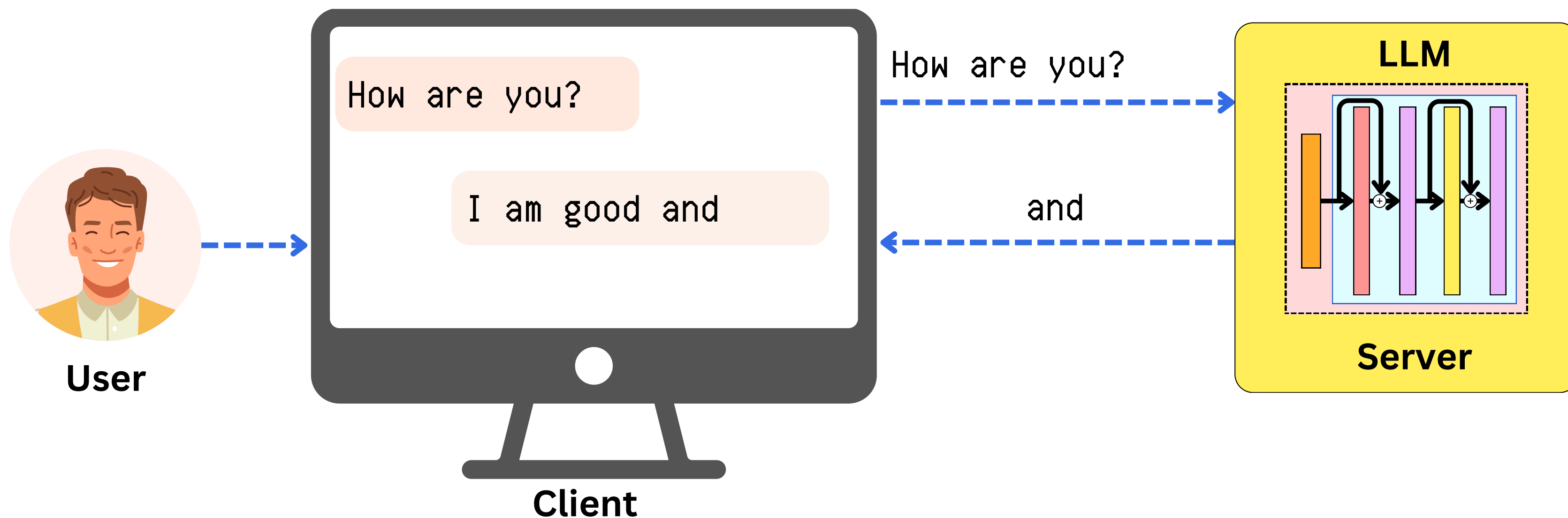


Streaming Text Generation



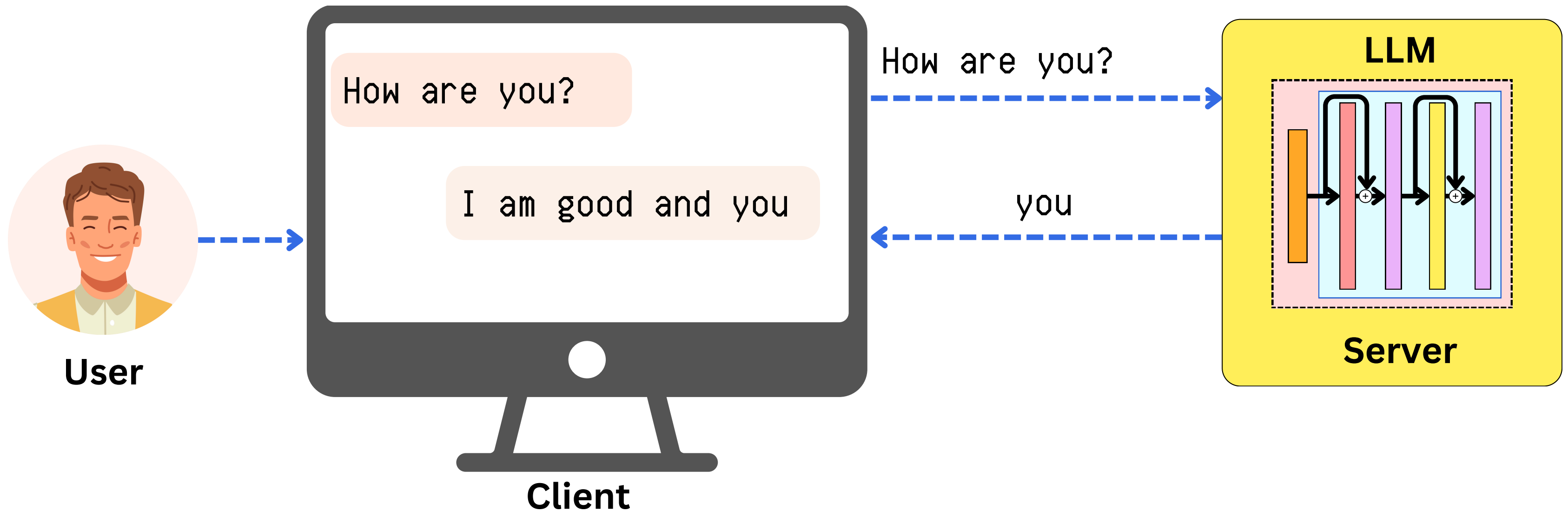


Streaming Text Generation



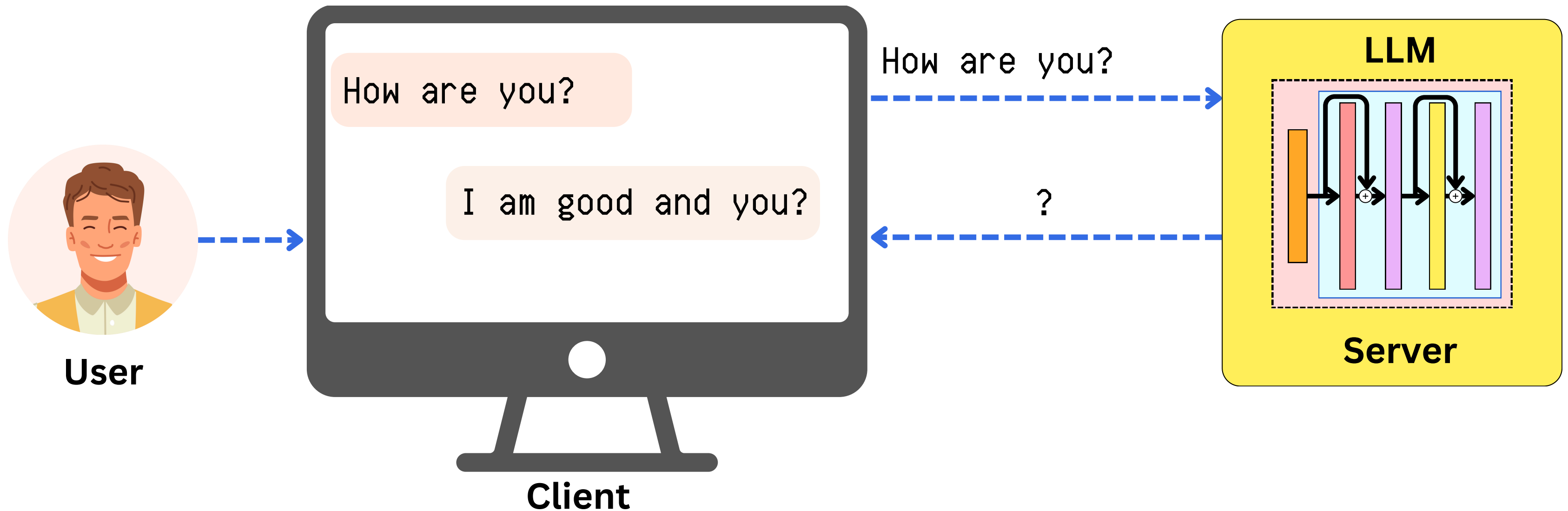


Streaming Text Generation



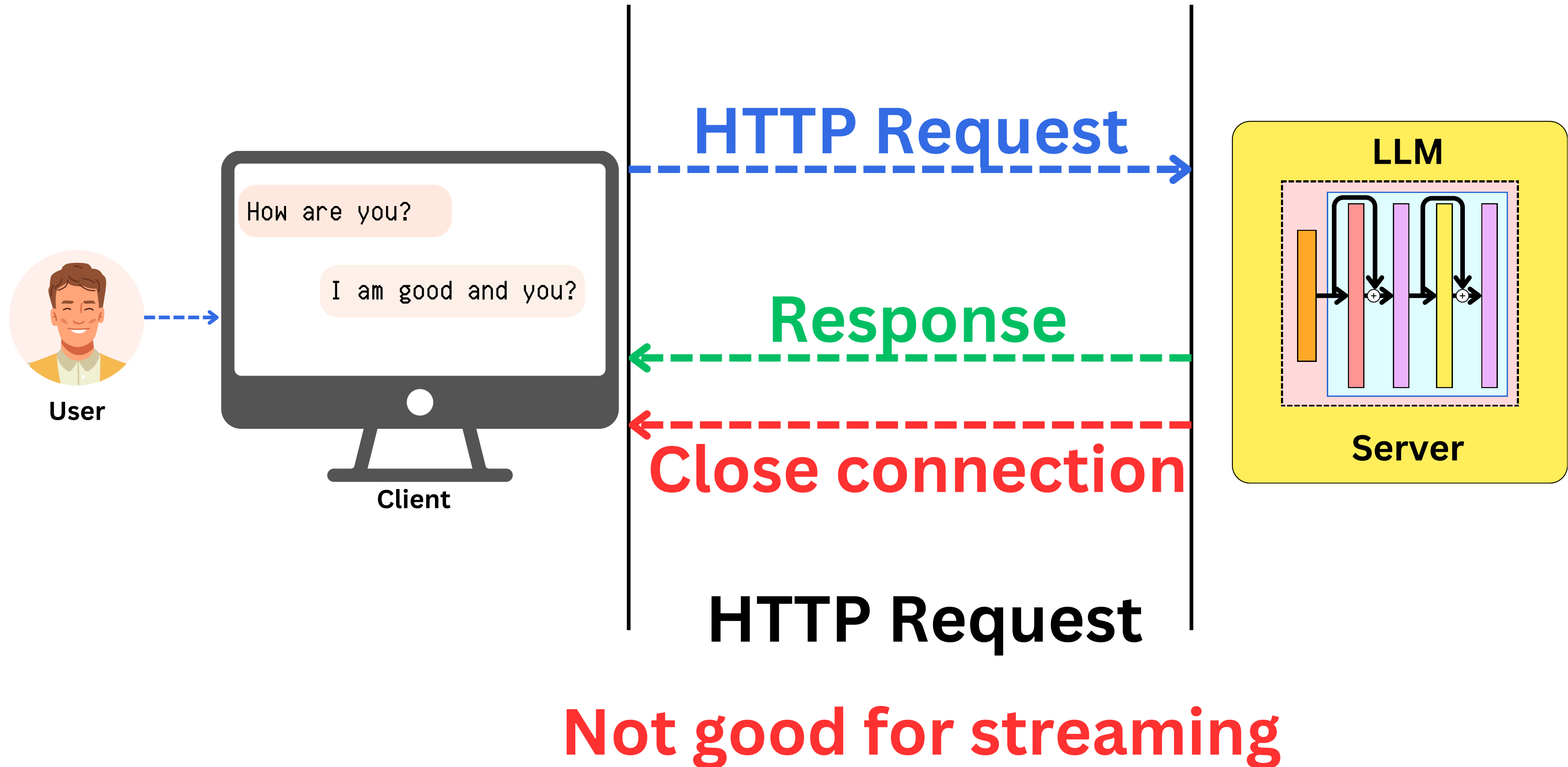


Streaming Text Generation



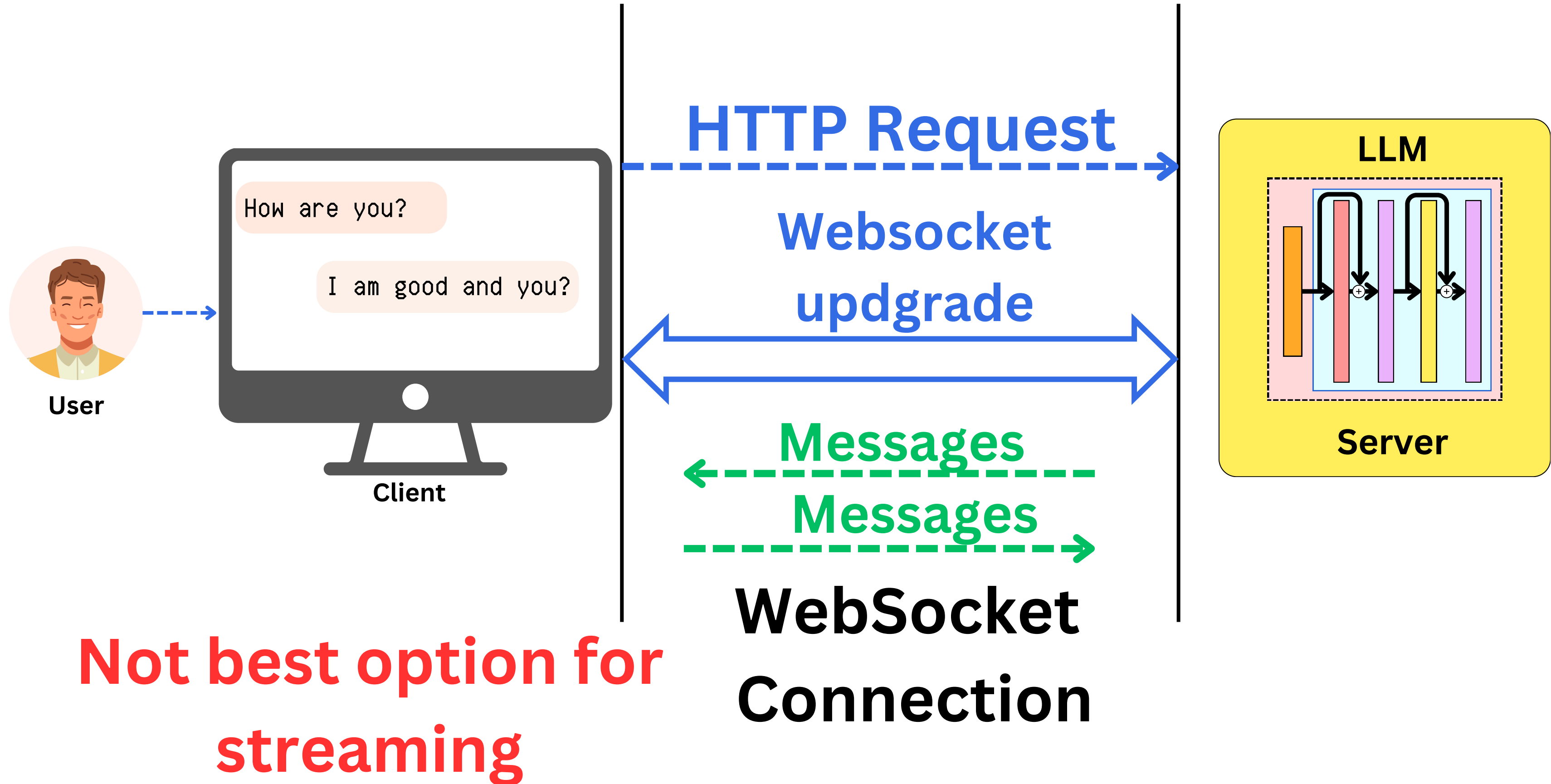


Streaming Text Generation



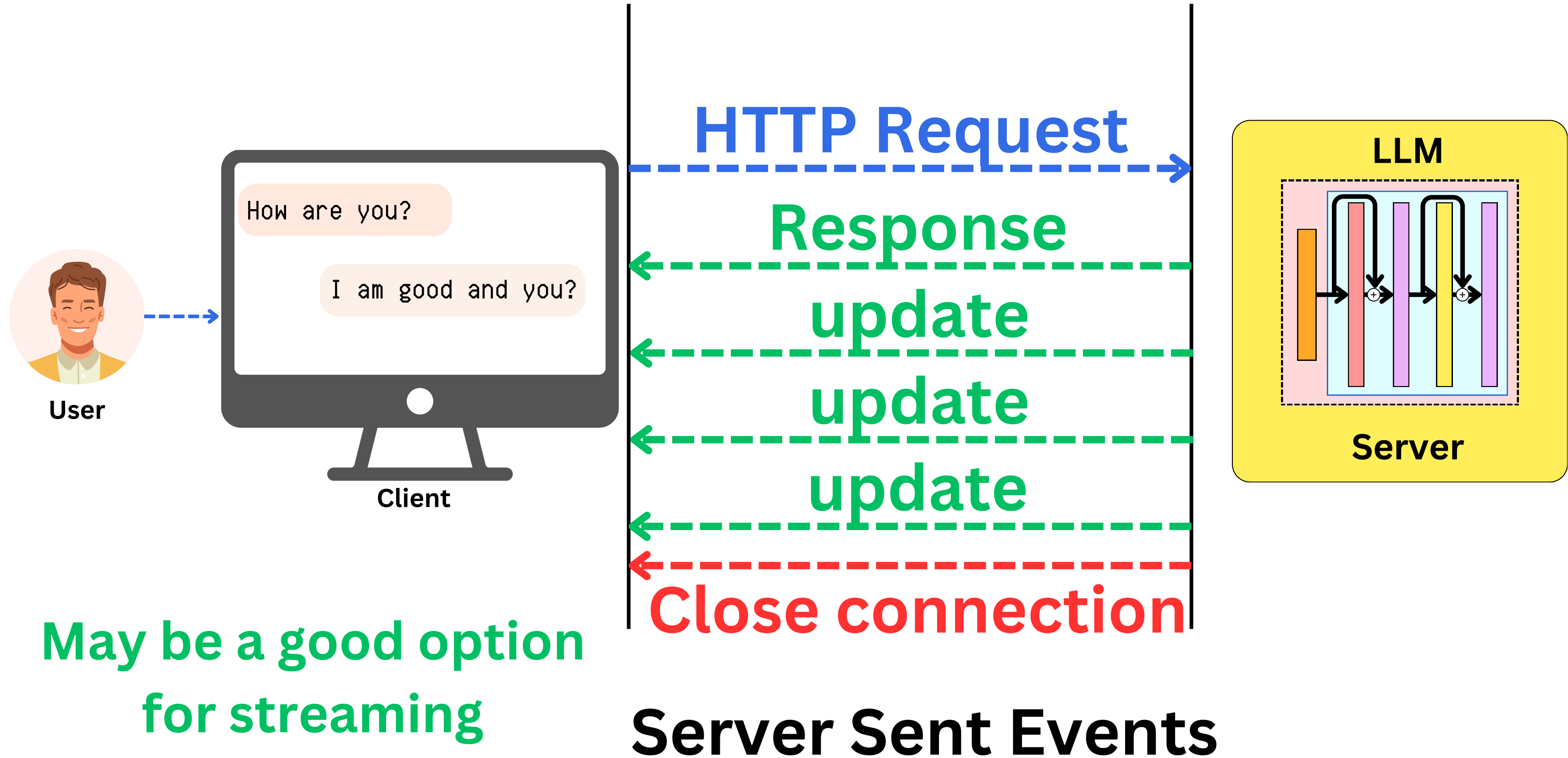


Streaming Text Generation





Streaming Text Generation





Streaming Text Generation

```
const eventSource = new EventSource("/stream");

eventSource.onmessage = function(event) {
  console.log("Message from server ", event.data);
};
```

Client-side
Javascript

```
from fastapi import FastAPI
from fastapi.responses import StreamingResponse

app = FastAPI()

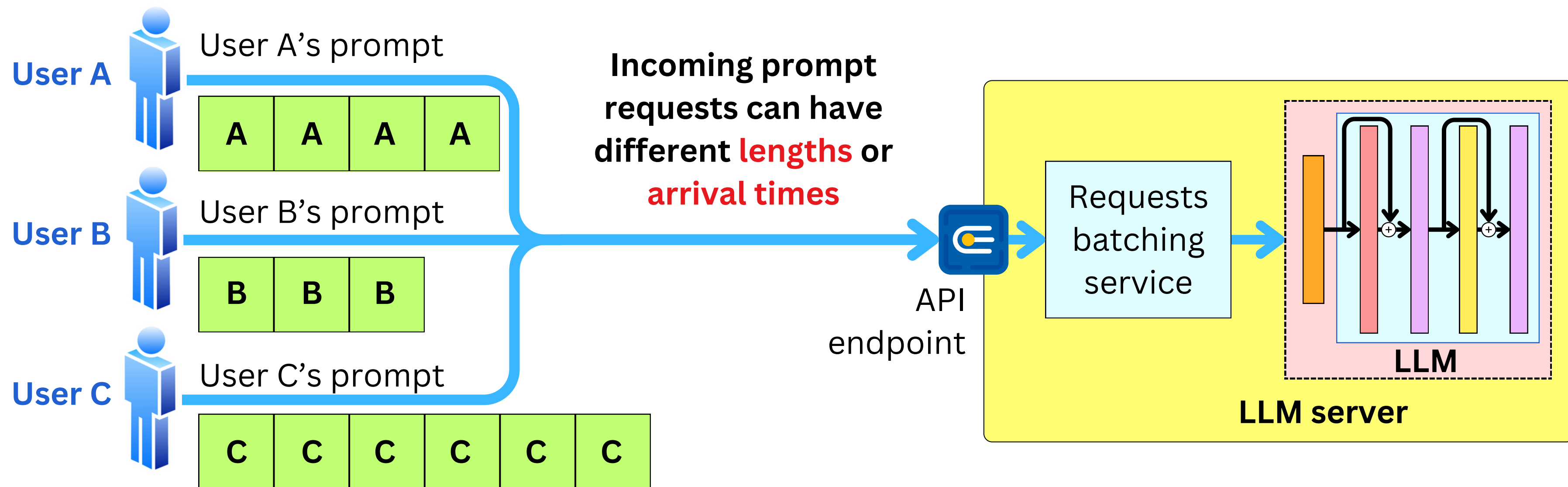
async def event_stream():
    for chunk in model.generate(prompt):
        yield chunk

@app.get("/stream")
async def stream():
    return StreamingResponse(
        event_stream(),
        media_type="text/event-stream"
    )
```

Server-side
FastAPI / Python



Batching Strategy

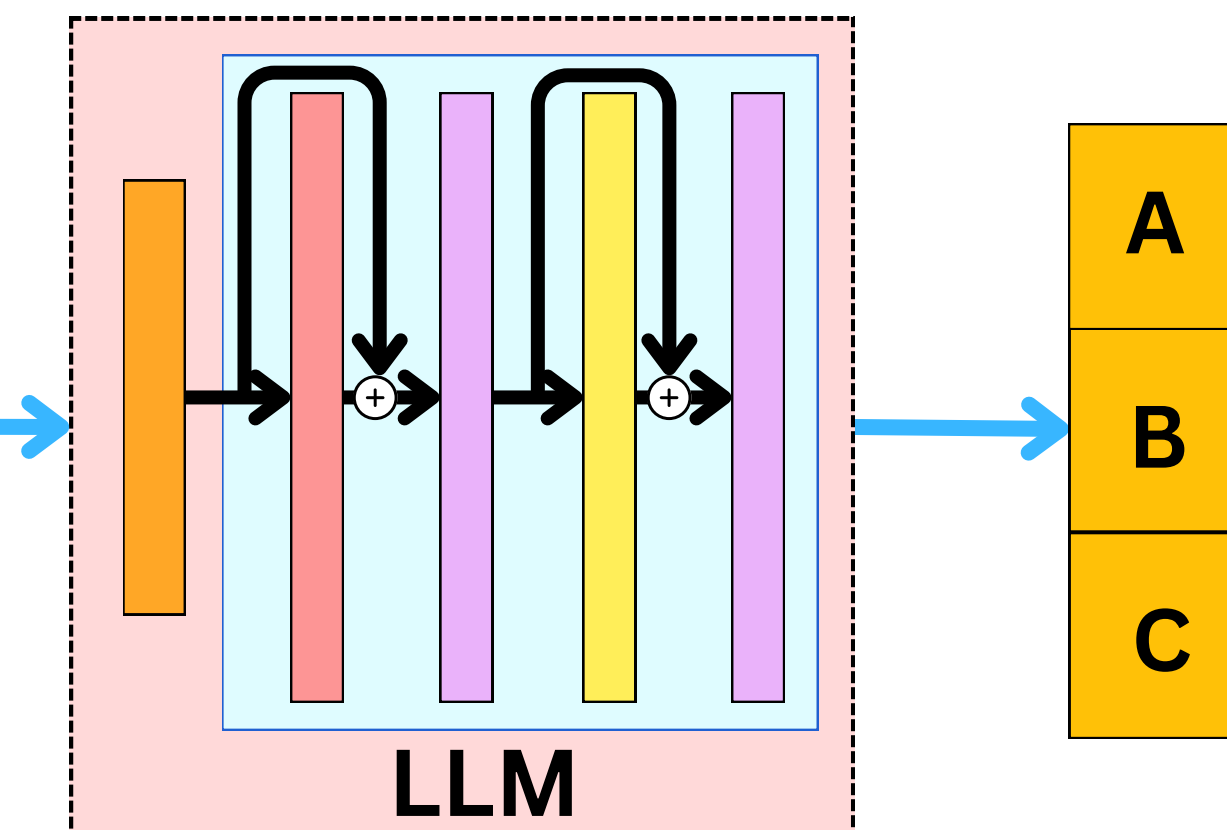




Naive Batching

First iteration

[PAD]	[PAD]	A	A	A	A
[PAD]	[PAD]	[PAD]	B	B	B
C	C	C	C	C	C

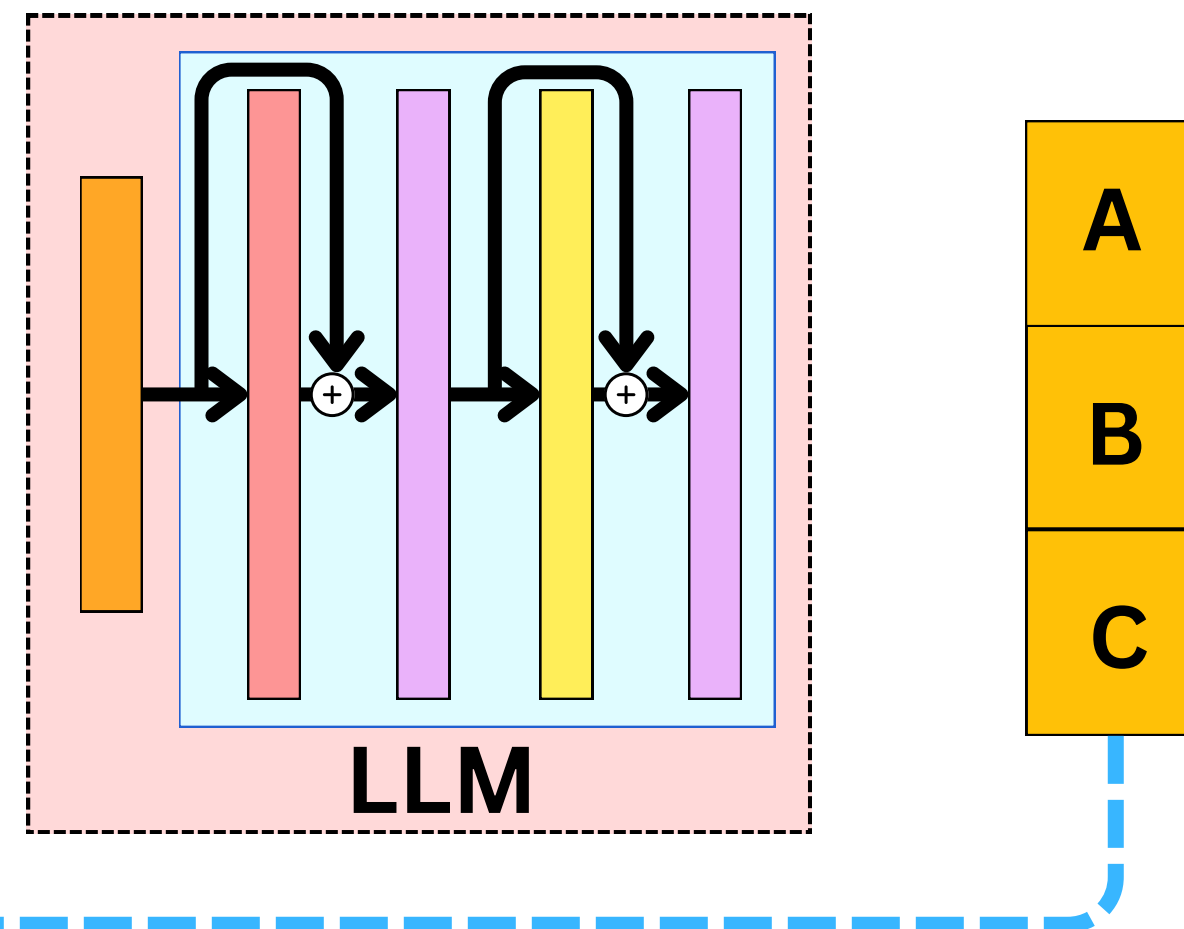




Naive Batching

First iteration

[PAD]	[PAD]	A	A	A	A	A
[PAD]	[PAD]	[PAD]	B	B	B	B
C	C	C	C	C	C	C

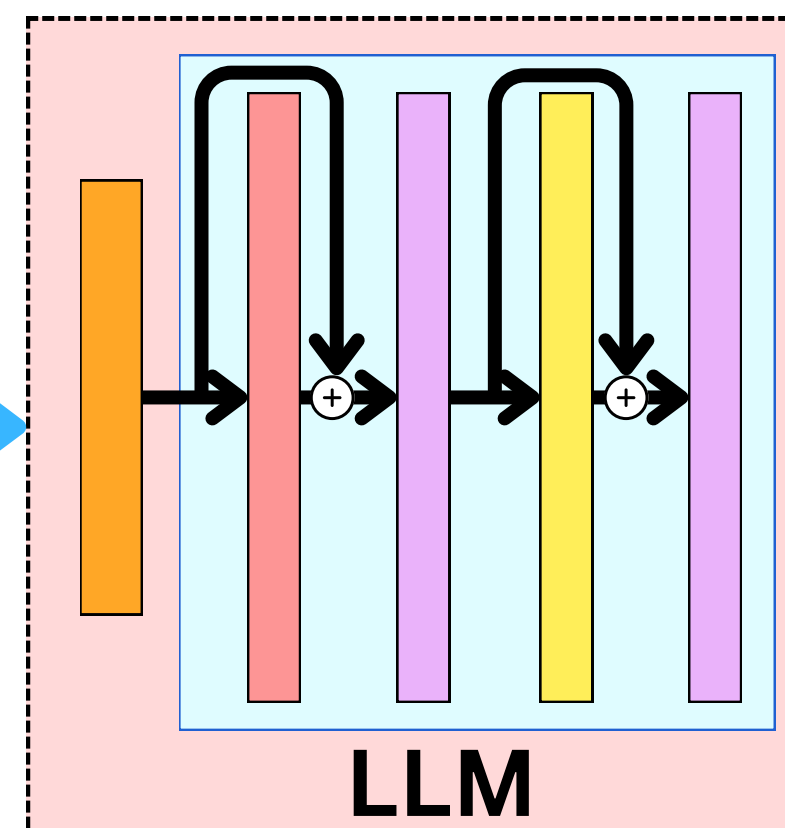




Naive Batching

Second iteration

[PAD]	[PAD]	A	A	A	A	A
[PAD]	[PAD]	[PAD]	B	B	B	B
C	C	C	C	C	C	C



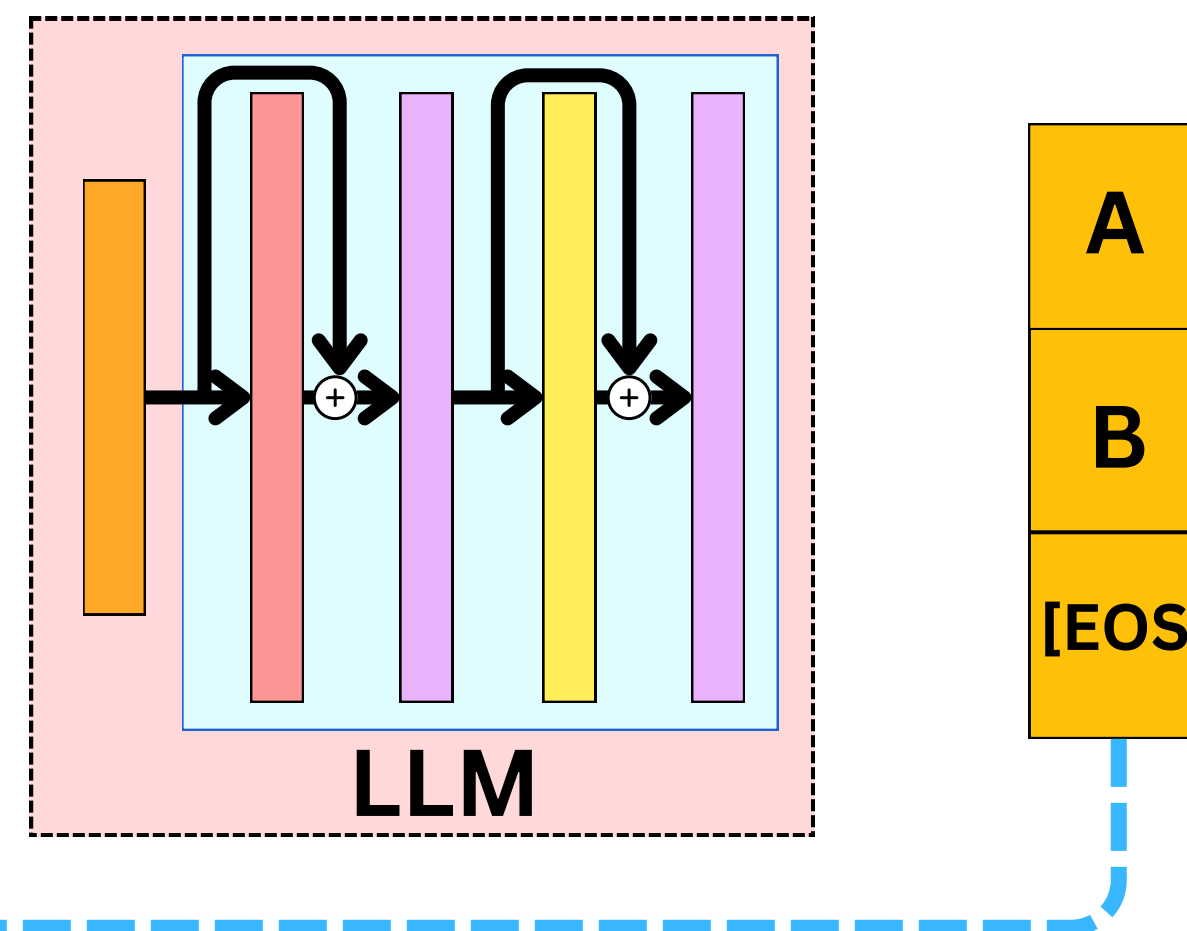
A
B
[EOS]



Naive Batching

Second iteration

[PAD]	[PAD]	A	A	A	A	A	A
[PAD]	[PAD]	[PAD]	B	B	B	B	B
C	C	C	C	C	C	C	[EOS]

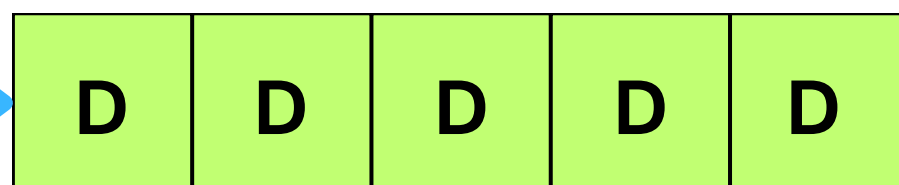
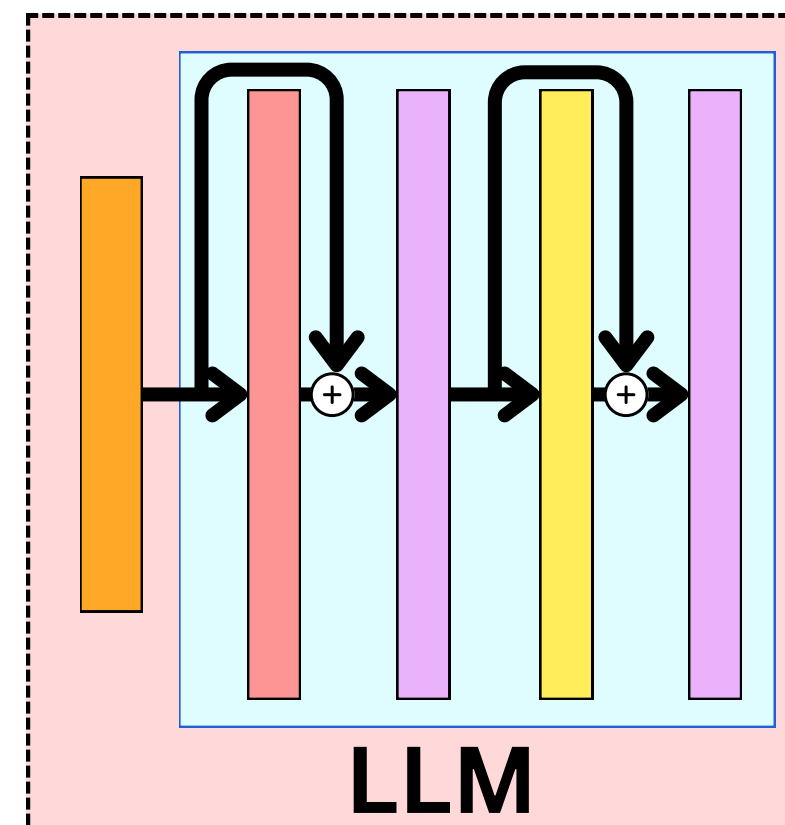




Naive Batching

Third iteration

[PAD]	[PAD]	A	A	A	A	A	A
[PAD]	[PAD]	[PAD]	B	B	B	B	B
C	C	C	C	C	C	C	[EOS]

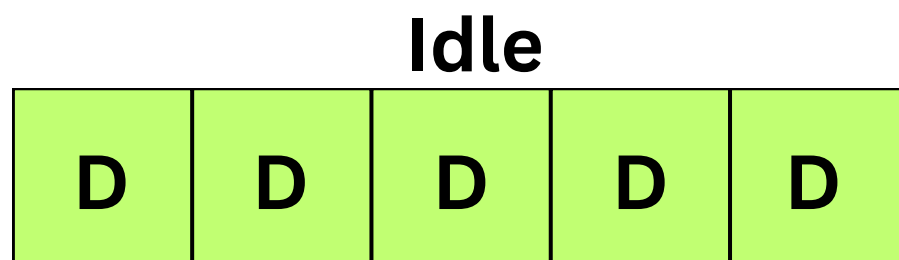
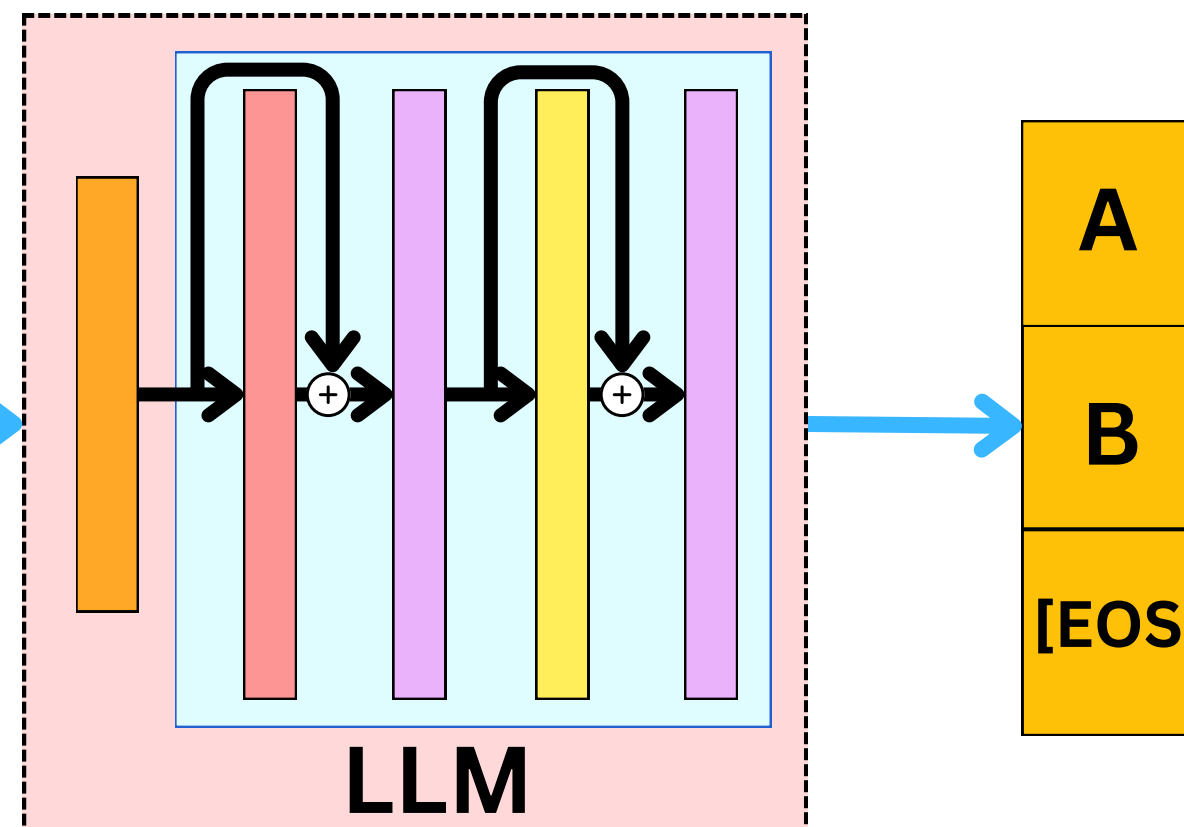
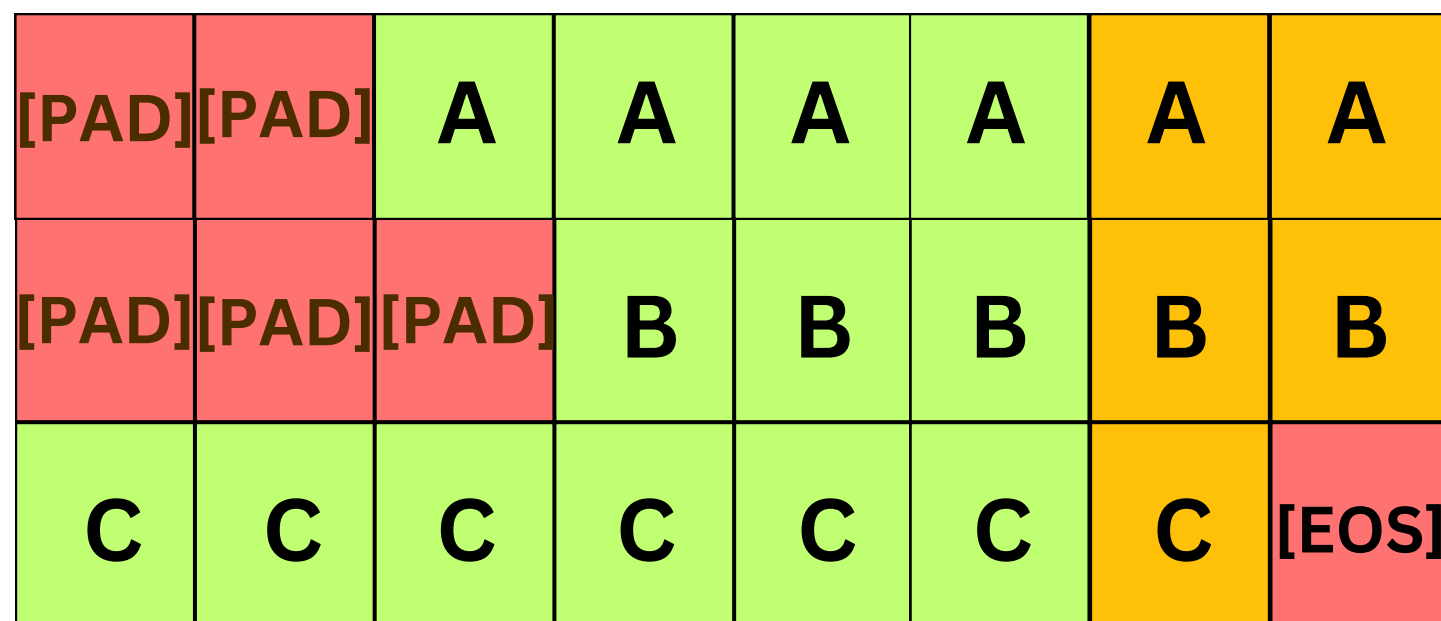


New incoming request



Naive Batching

Third iteration



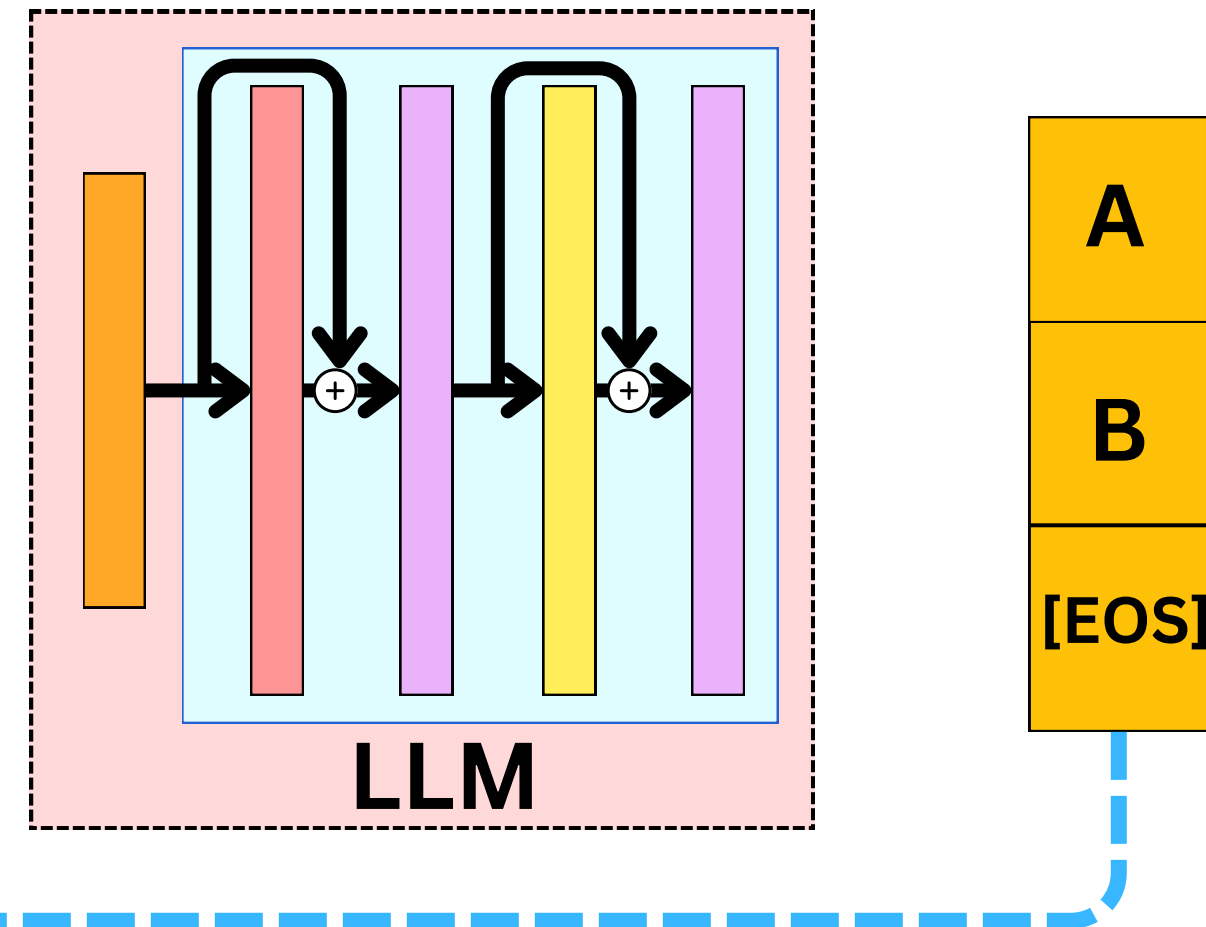
New incoming request



Naive Batching

Third iteration

[PAD]	[PAD]	A	A	A	A	A	A	A
[PAD]	[PAD]	[PAD]	B	B	B	B	B	B
C	C	C	C	C	C	C	[EOS]	[EOS]



Idle

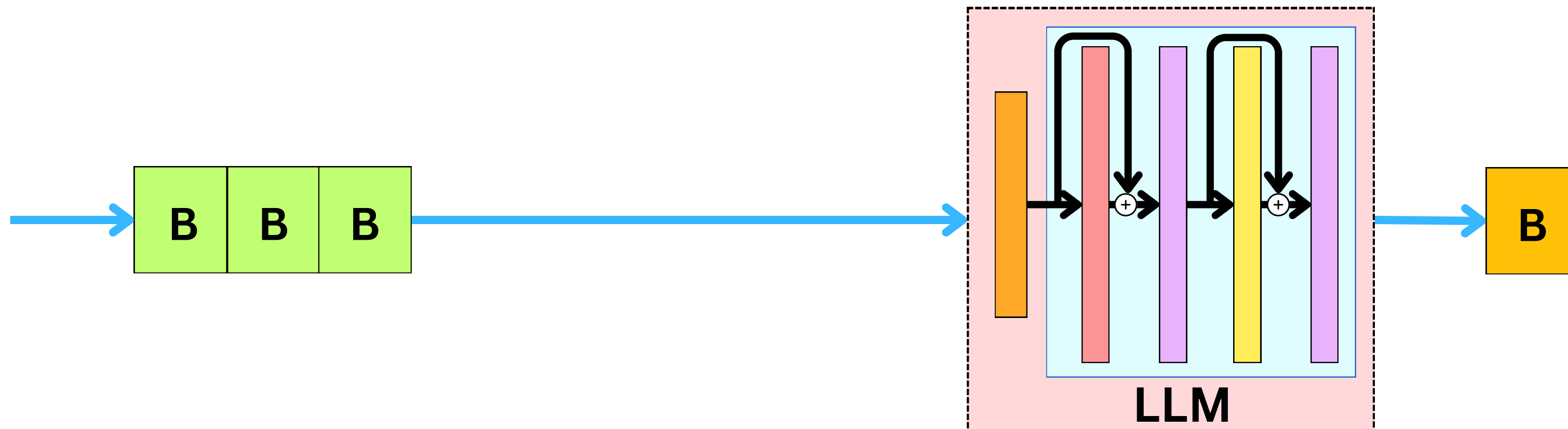
D	D	D	D	D
---	---	---	---	---

New incoming request



Continuous Batching

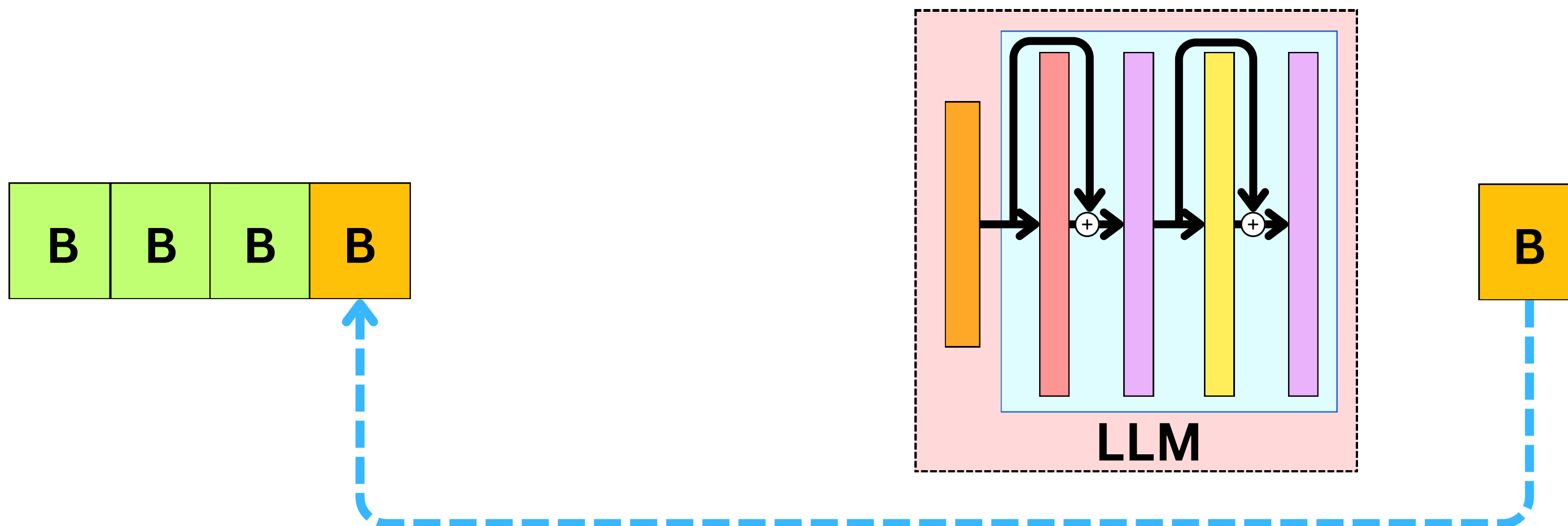
First iteration





Continuous Batching

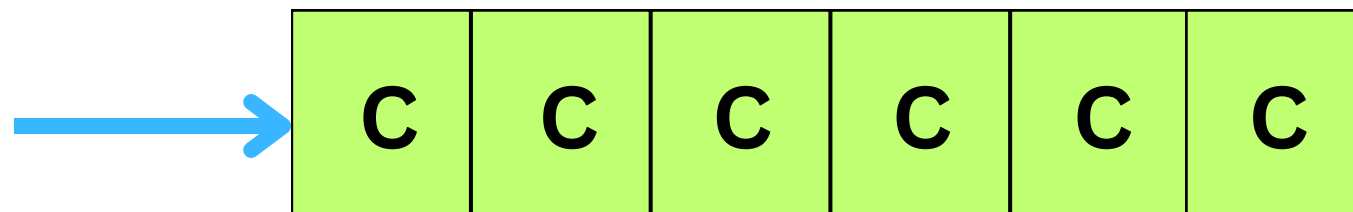
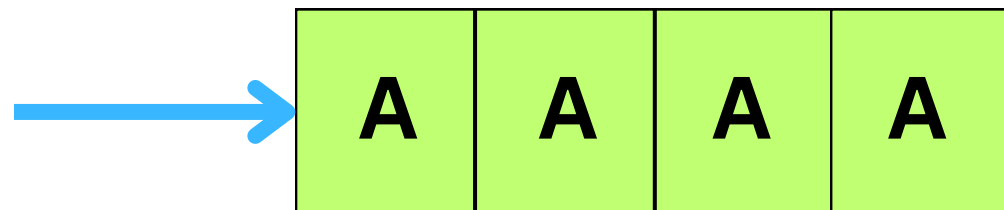
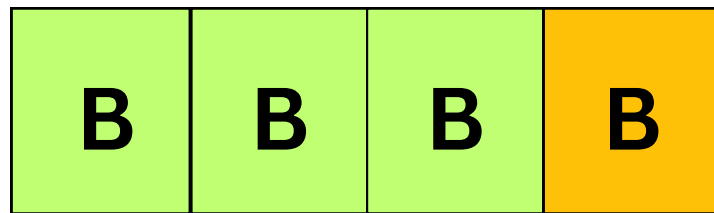
First iteration



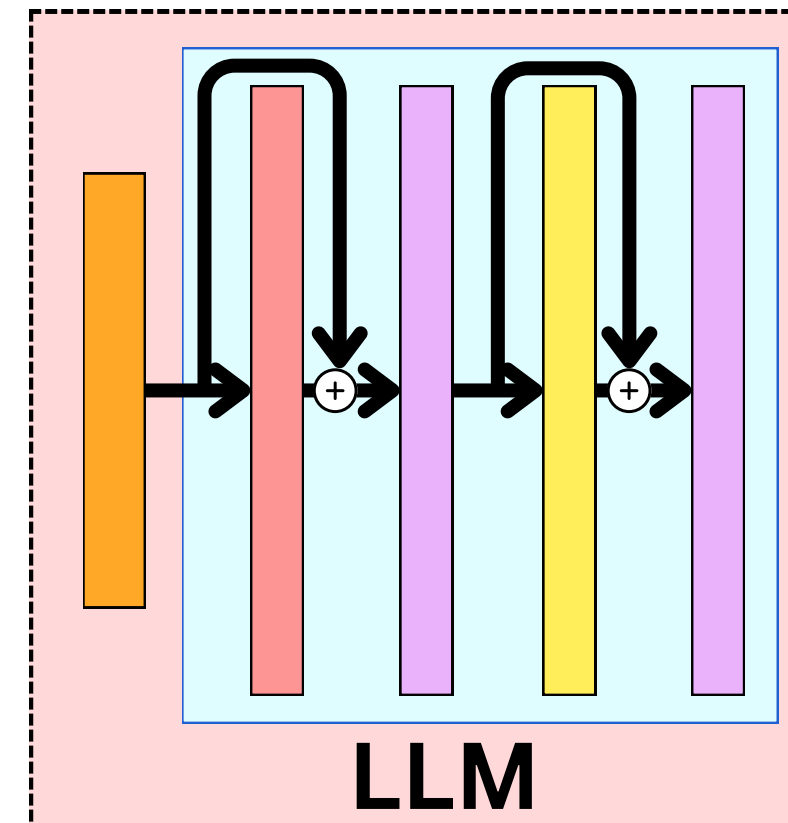


Continuous Batching

Second iteration



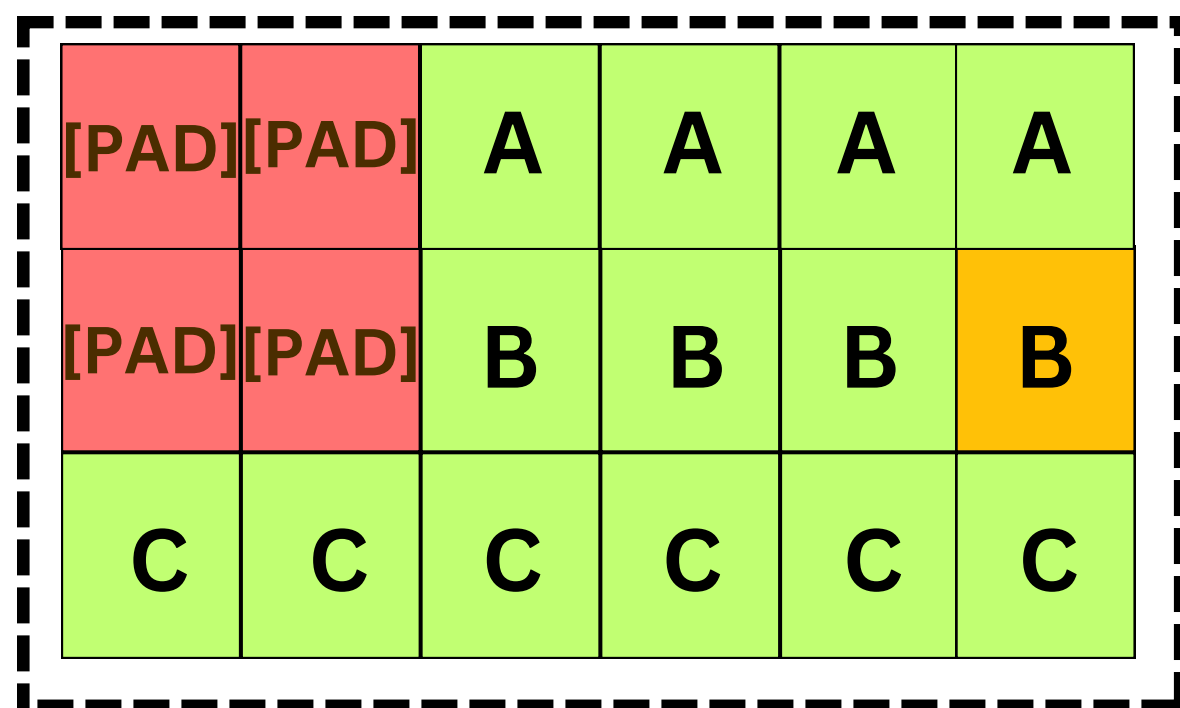
New incoming requests



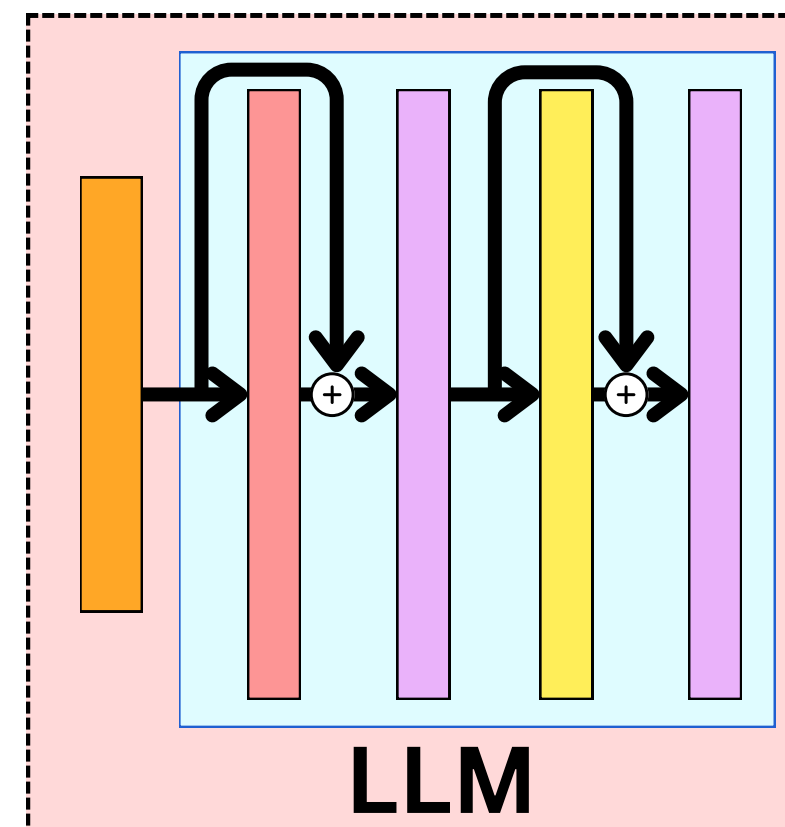
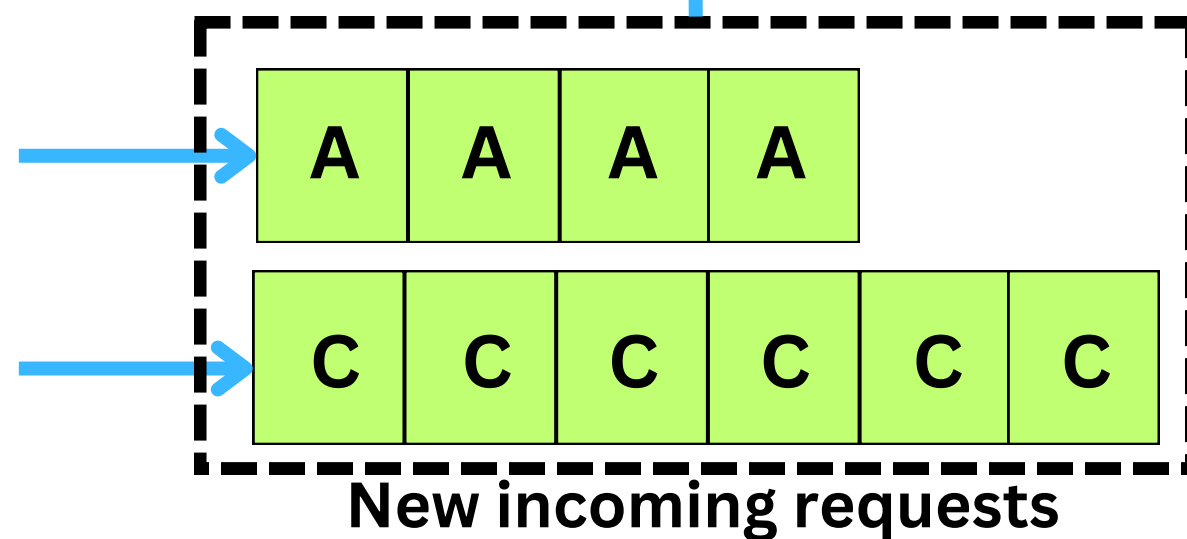


Continuous Batching

Second iteration



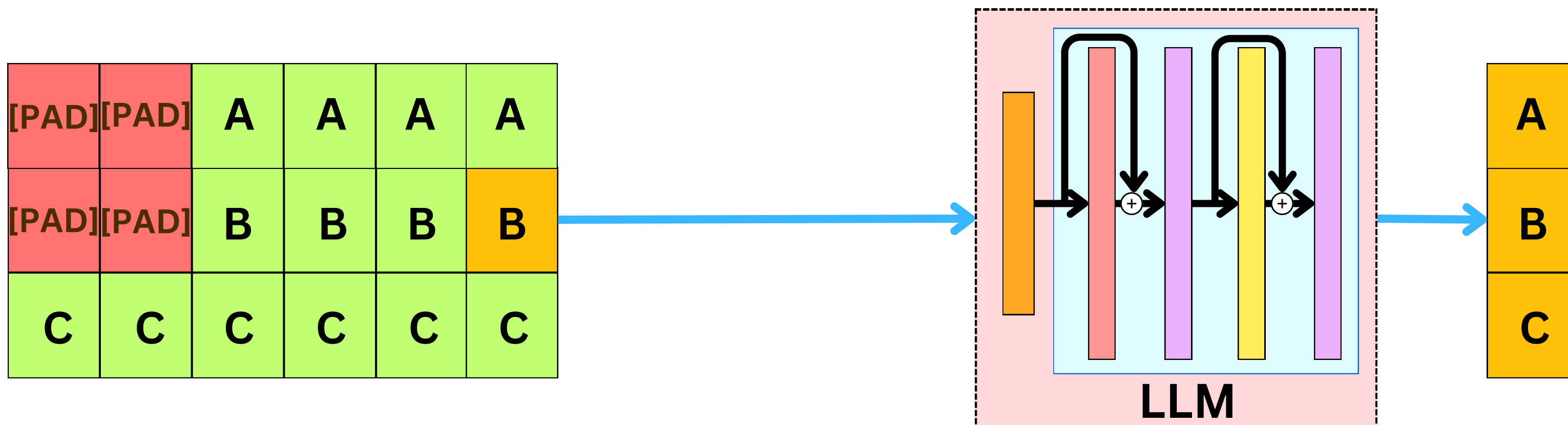
Rebatch





Continuous Batching

Second iteration

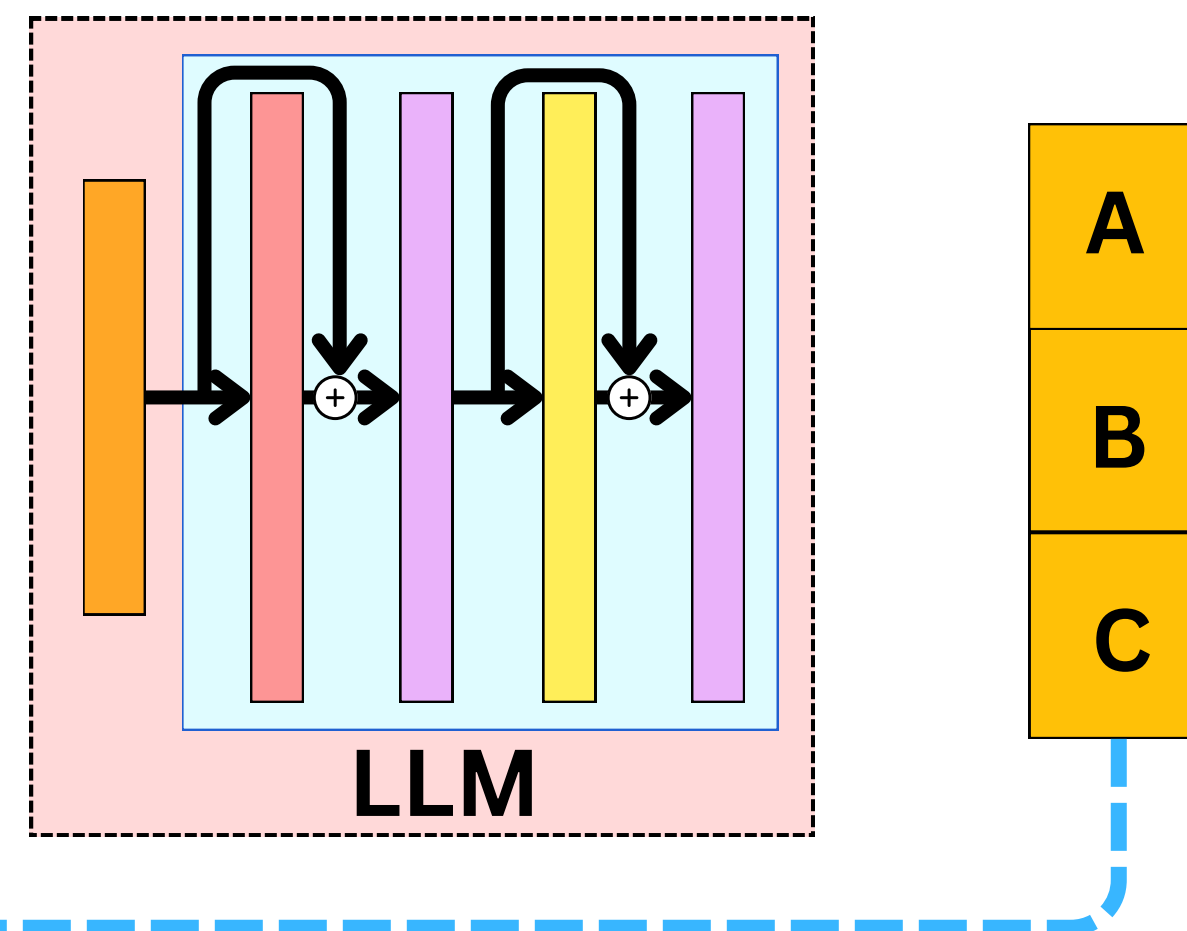




Continuous Batching

Second iteration

[PAD]	[PAD]	A	A	A	A	A
[PAD]	[PAD]	B	B	B	B	B
C	C	C	C	C	C	C

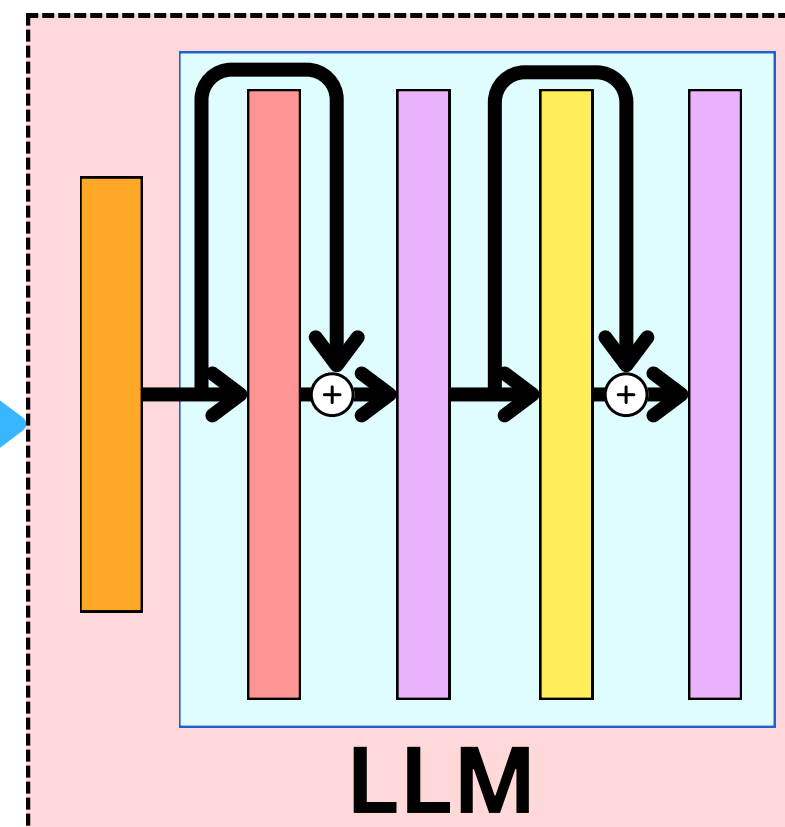




Continuous Batching

Third iteration

[PAD]	[PAD]	A	A	A	A	A
[PAD]	[PAD]	B	B	B	B	B
C	C	C	C	C	C	C

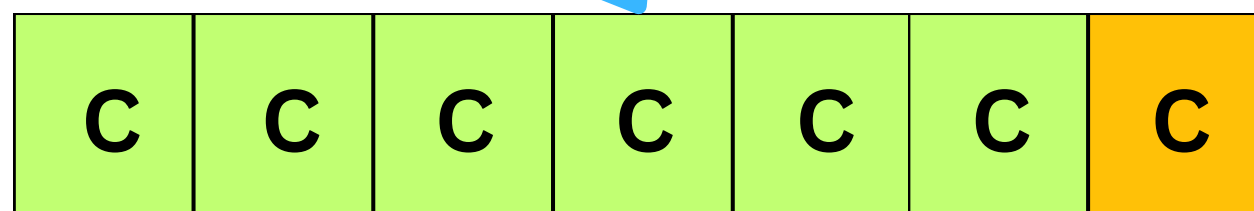
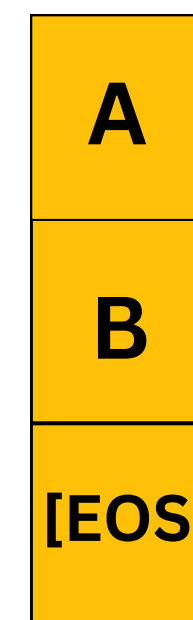
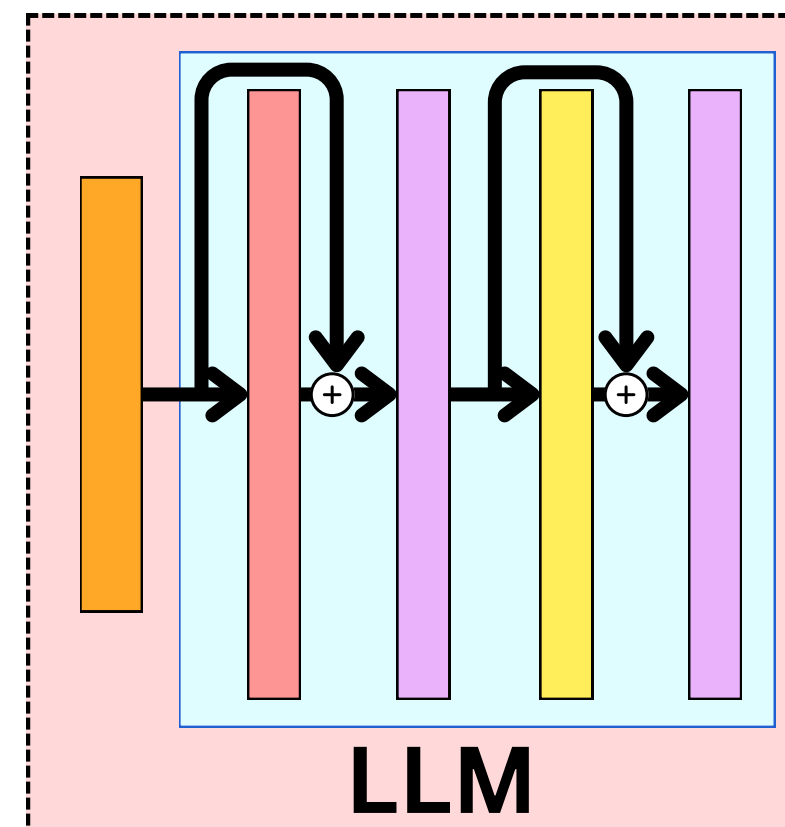
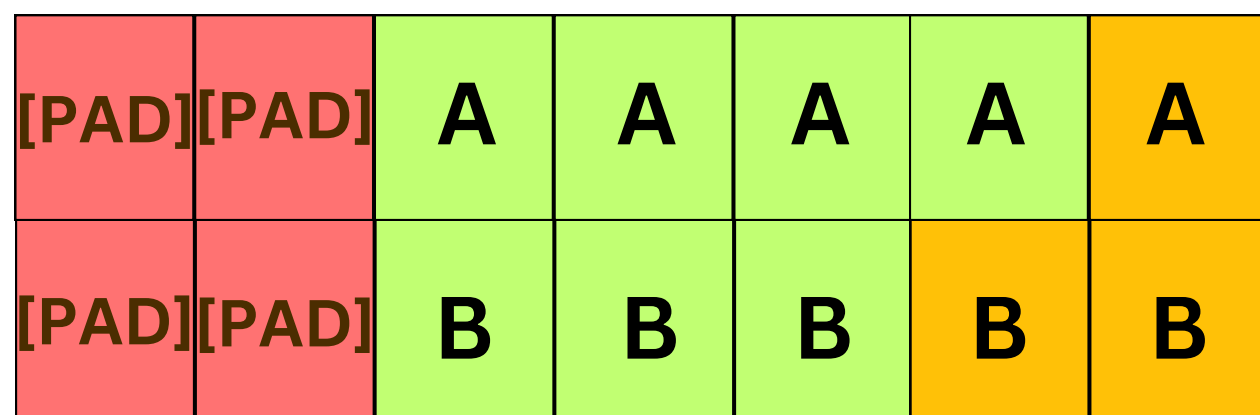


A
B
[EOS]



Continuous Batching

Third iteration



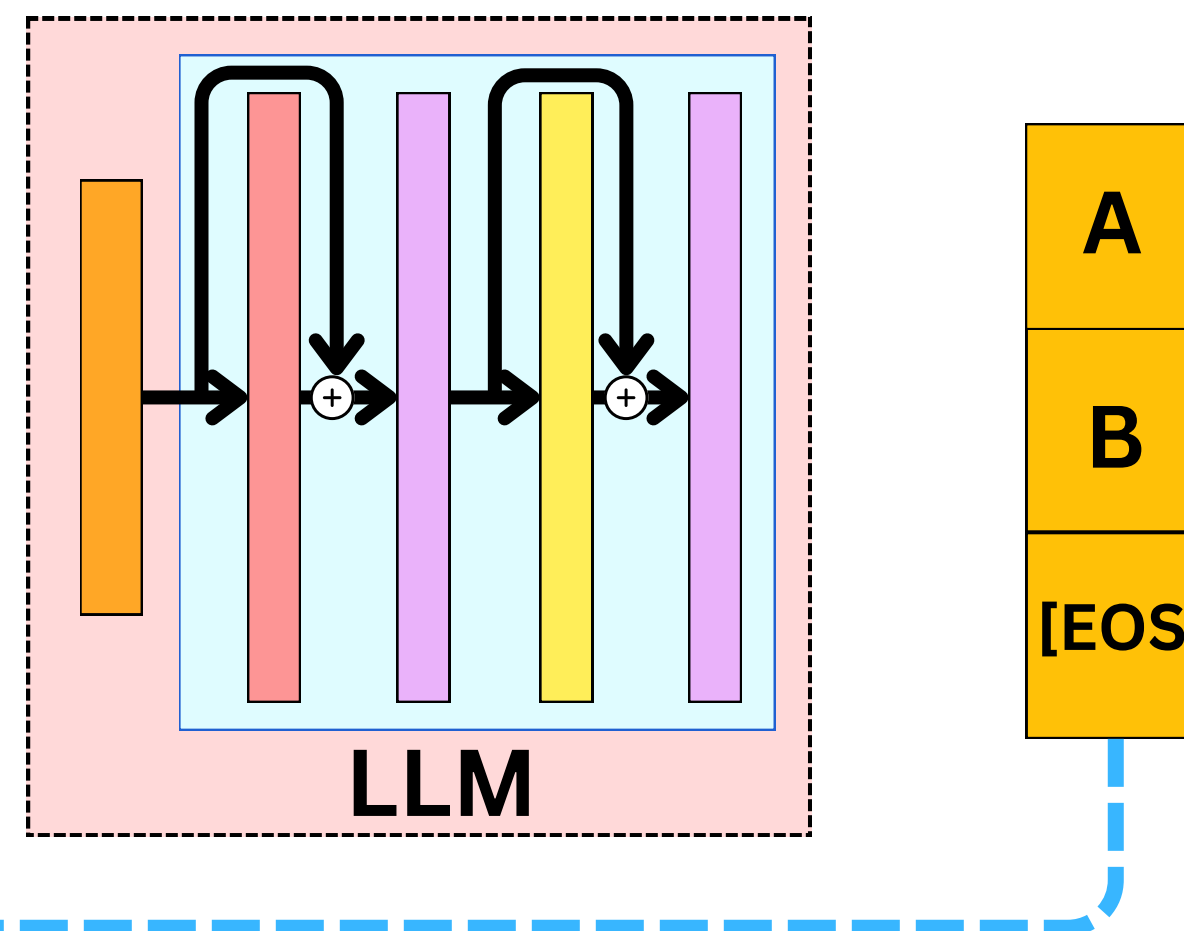
Send to client



Continuous Batching

Third iteration

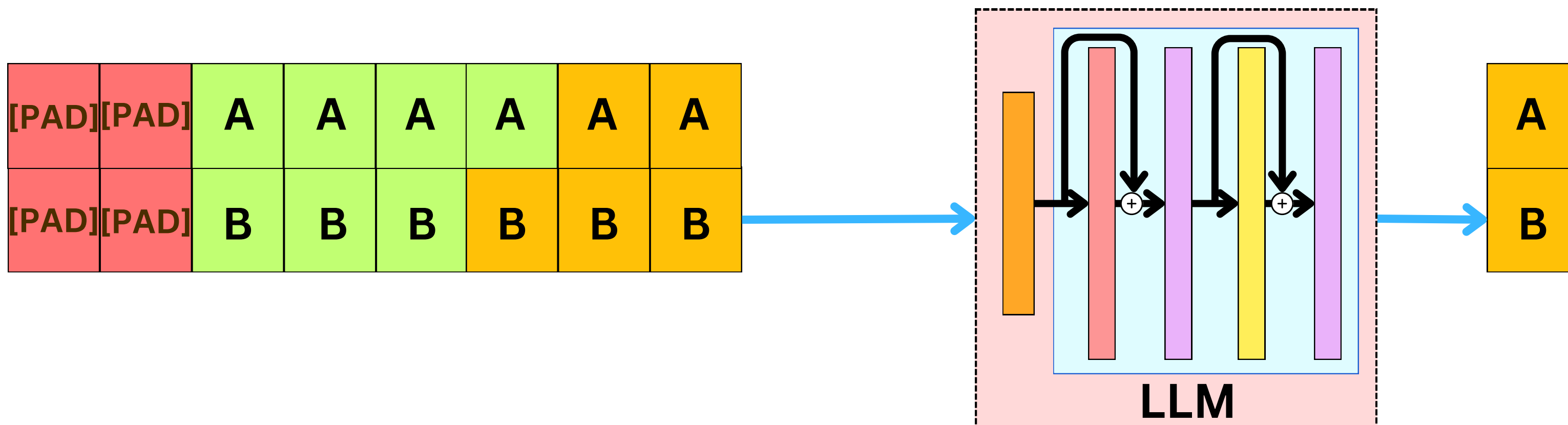
[PAD]	[PAD]	A	A	A	A	A	A
[PAD]	[PAD]	B	B	B	B	B	B





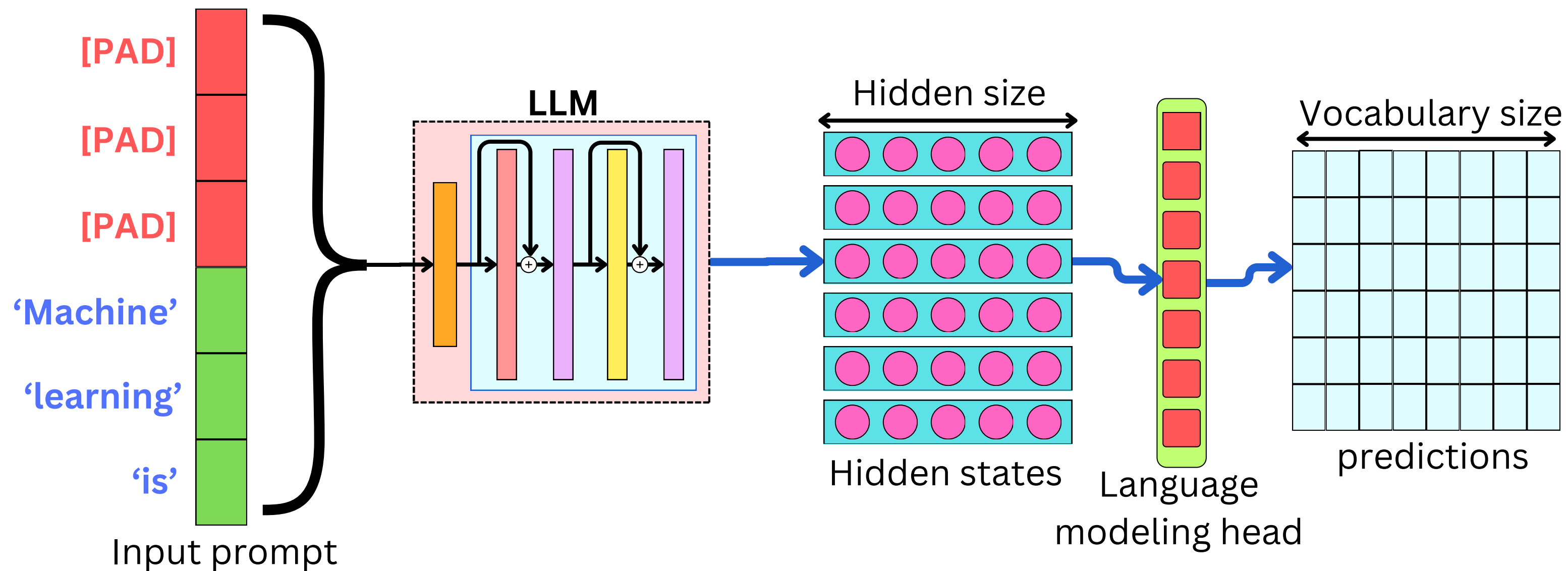
Continuous Batching

Forth iteration



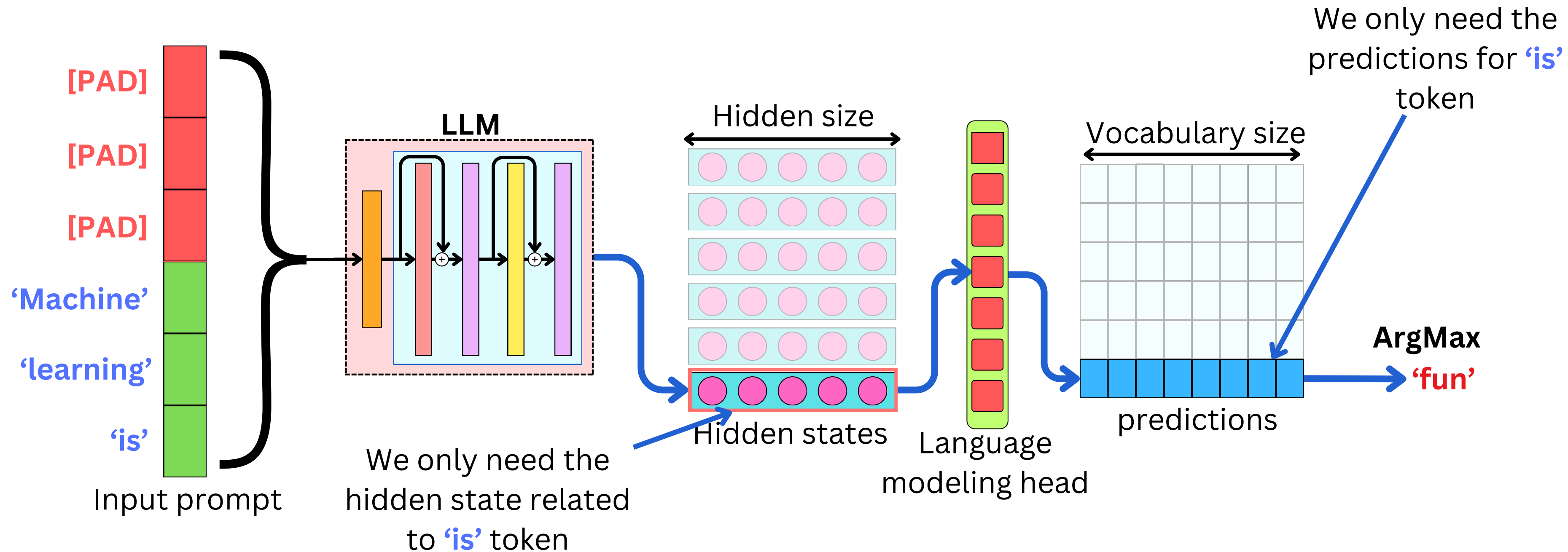


KV-Caching





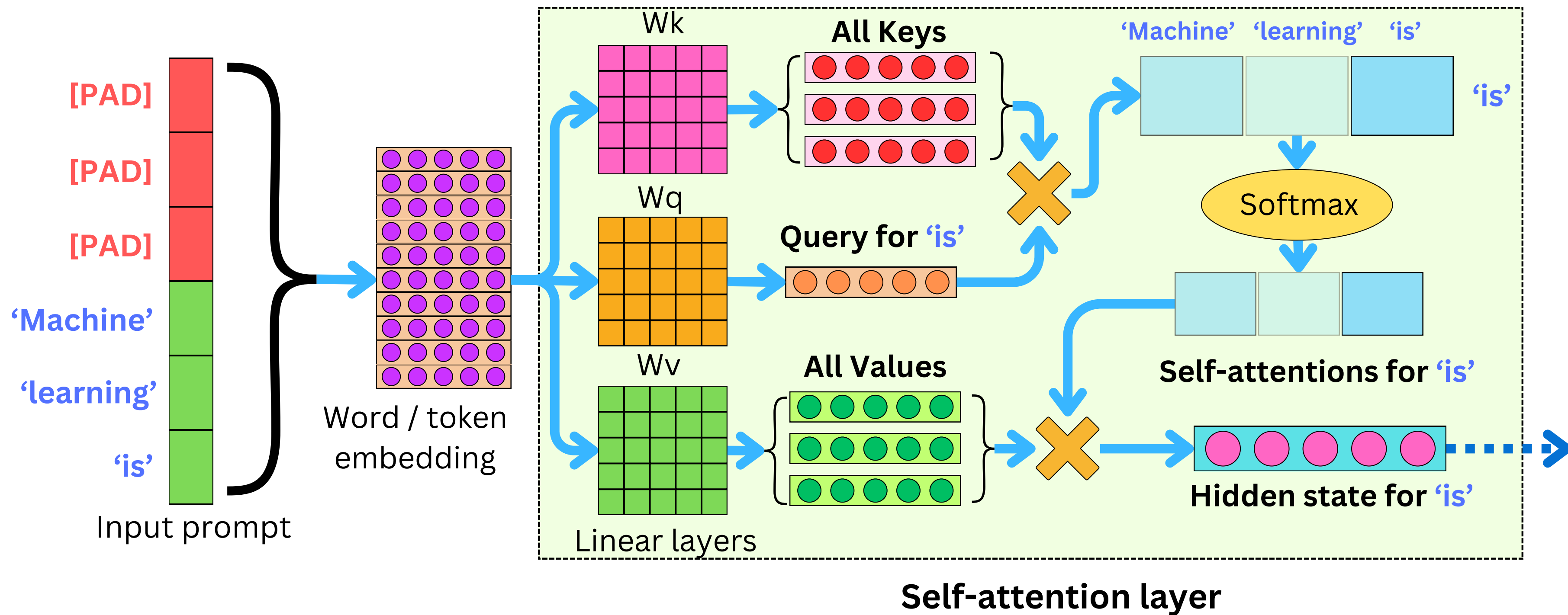
KV-Caching



When we generate text, we only need to predict the next token

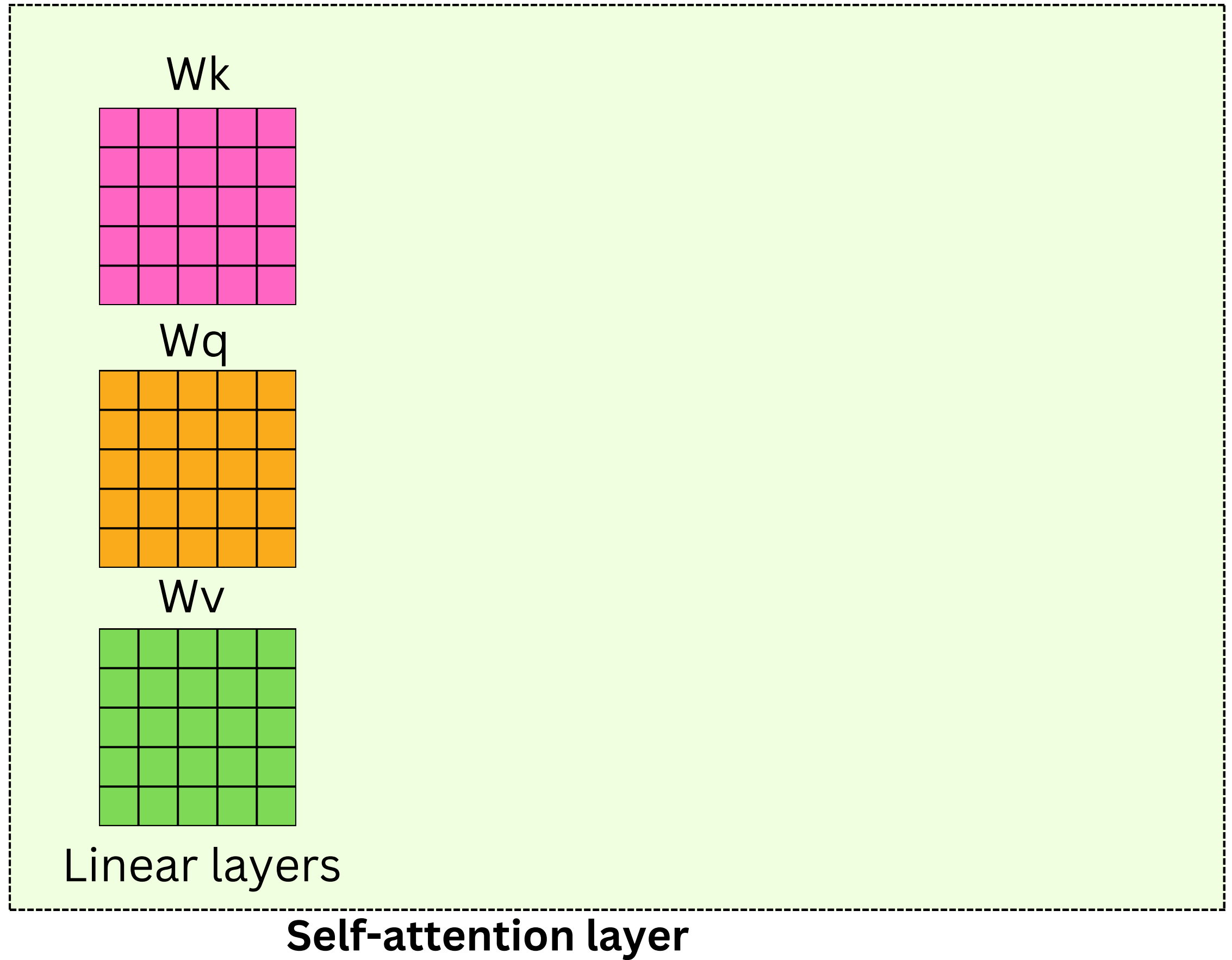
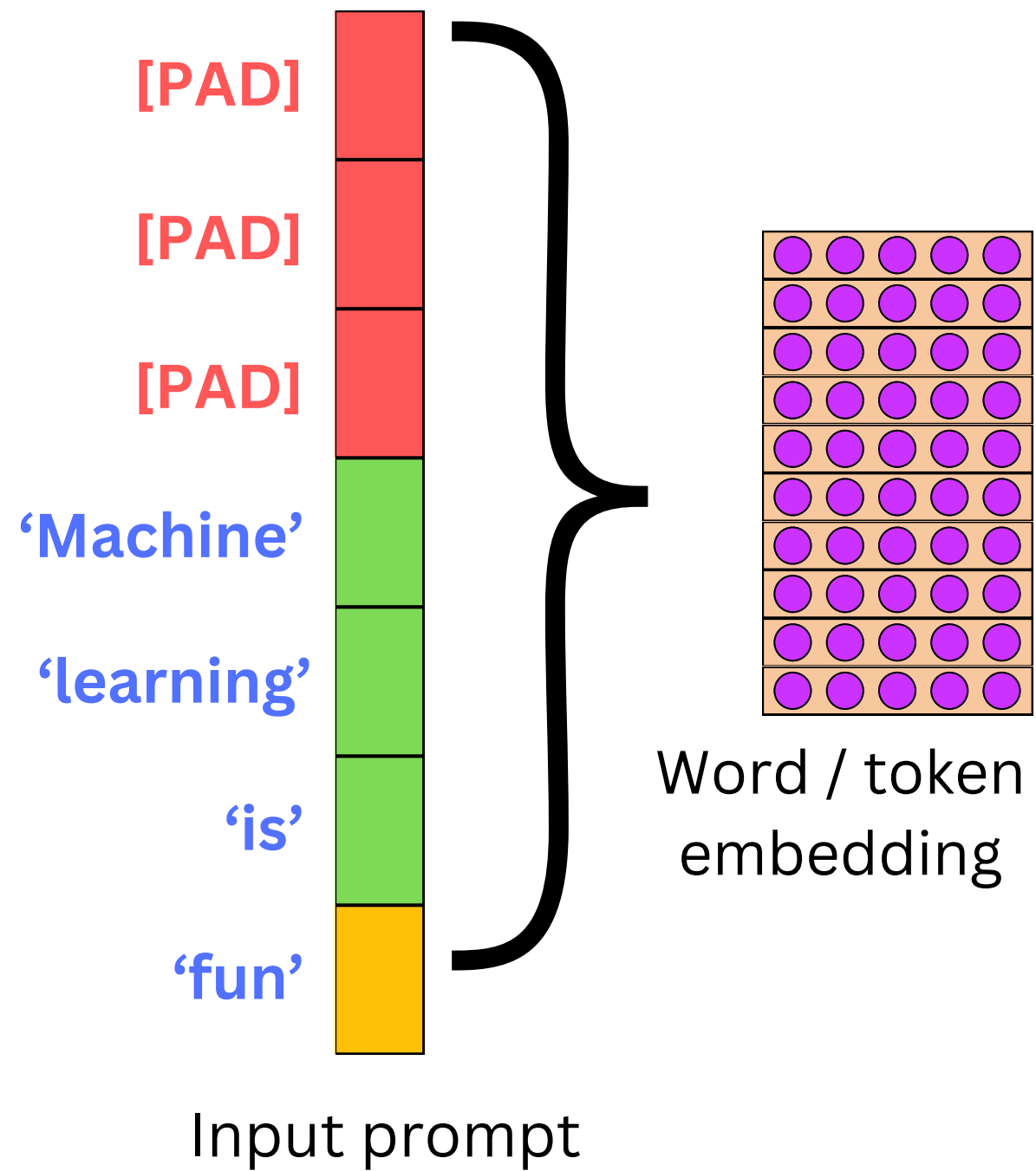


KV-Caching



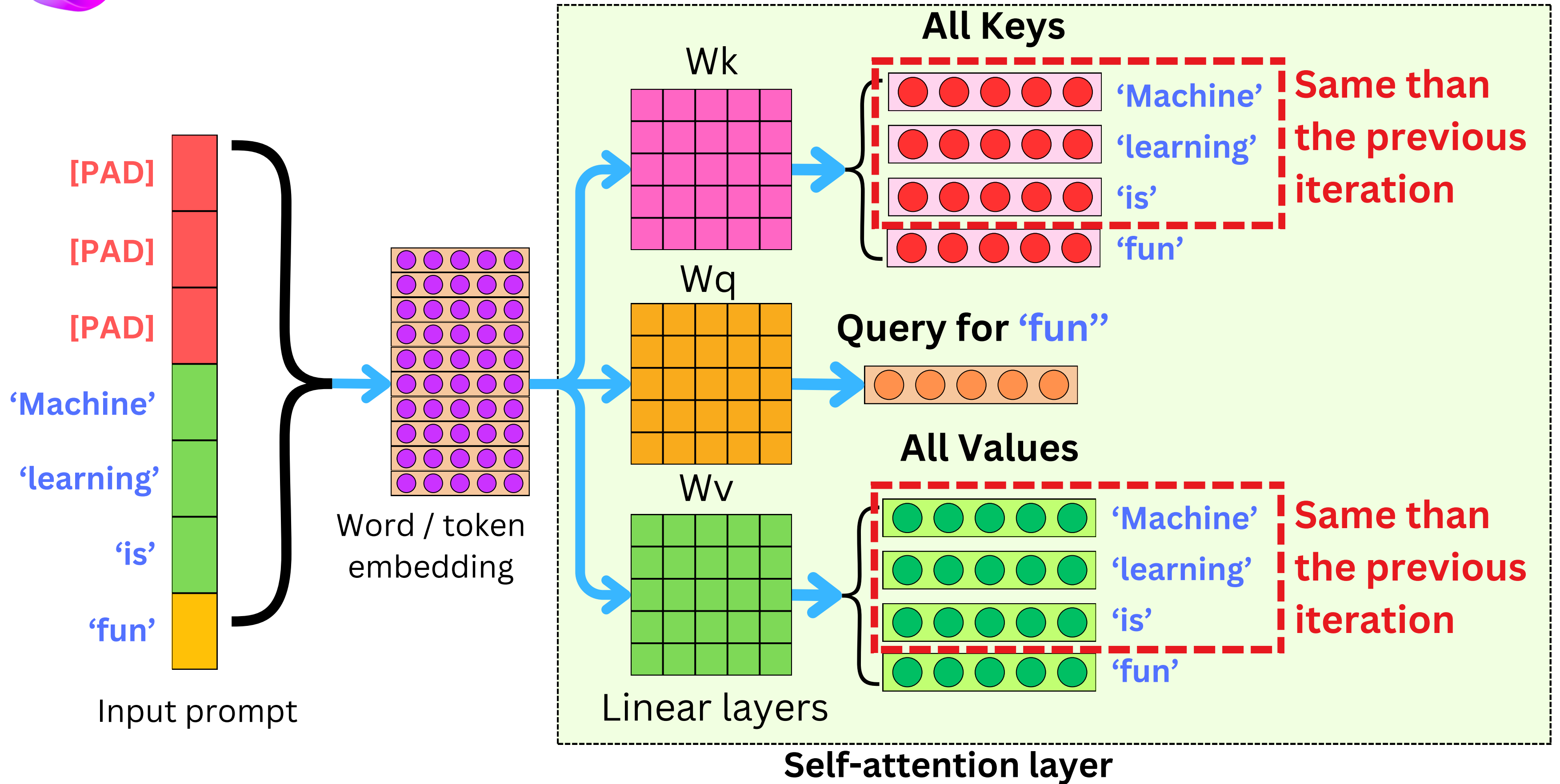


KV-Caching



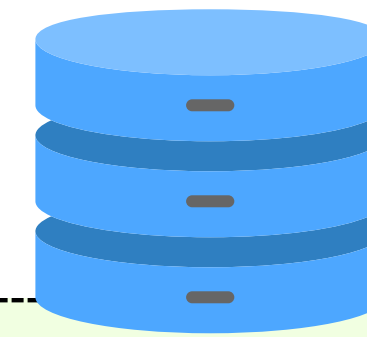


KV-Caching

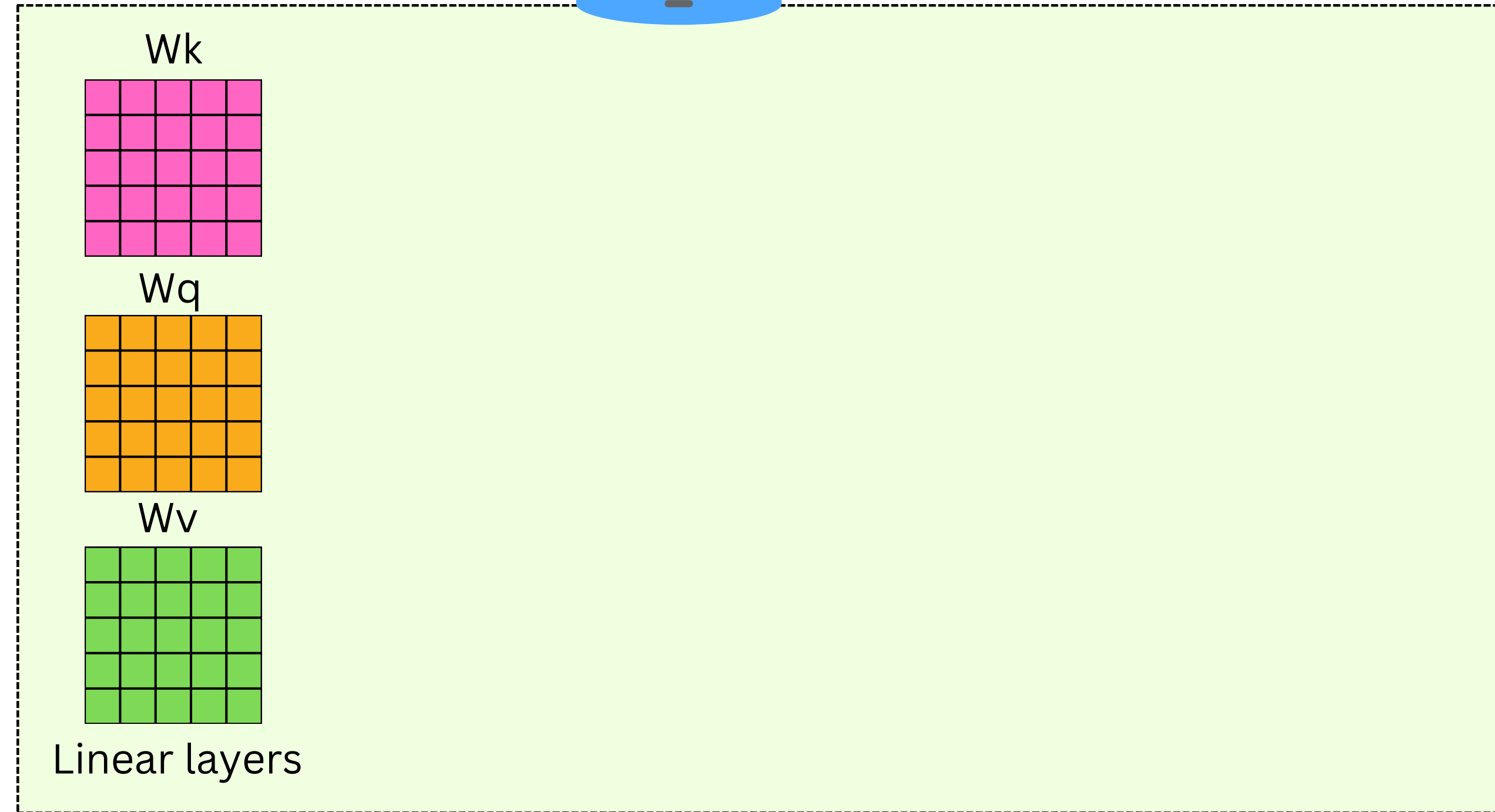
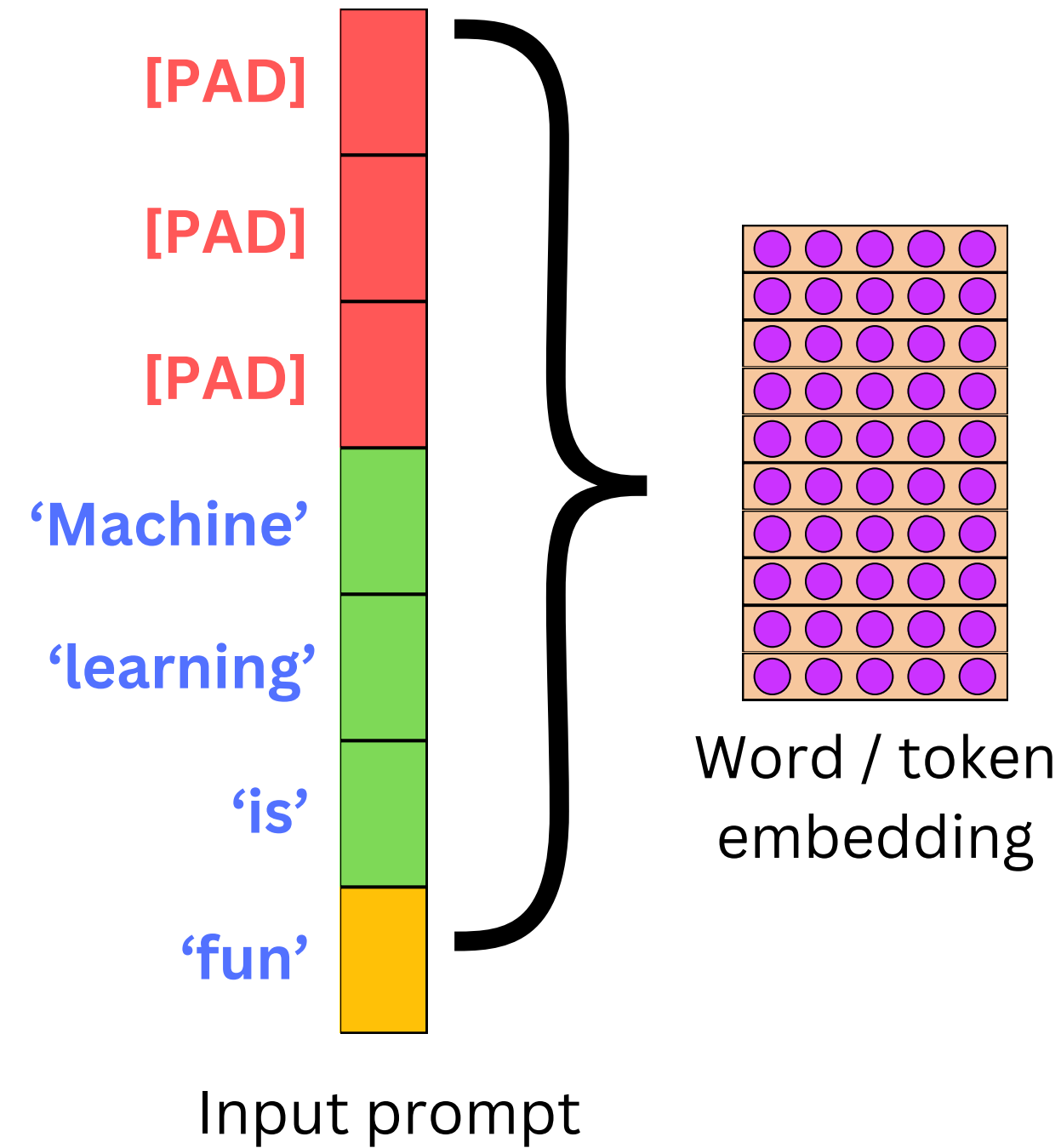




KV-Caching

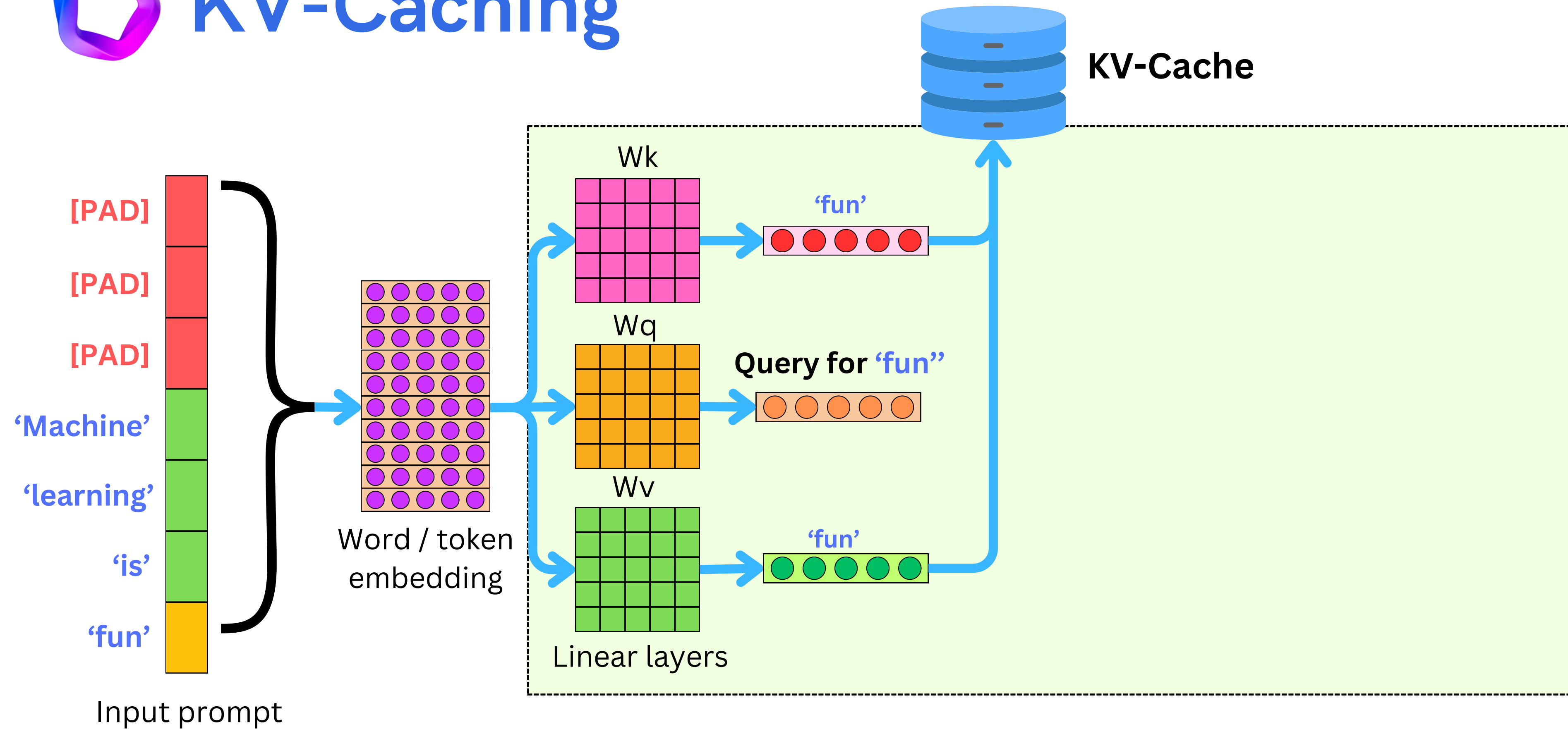


KV-Cache



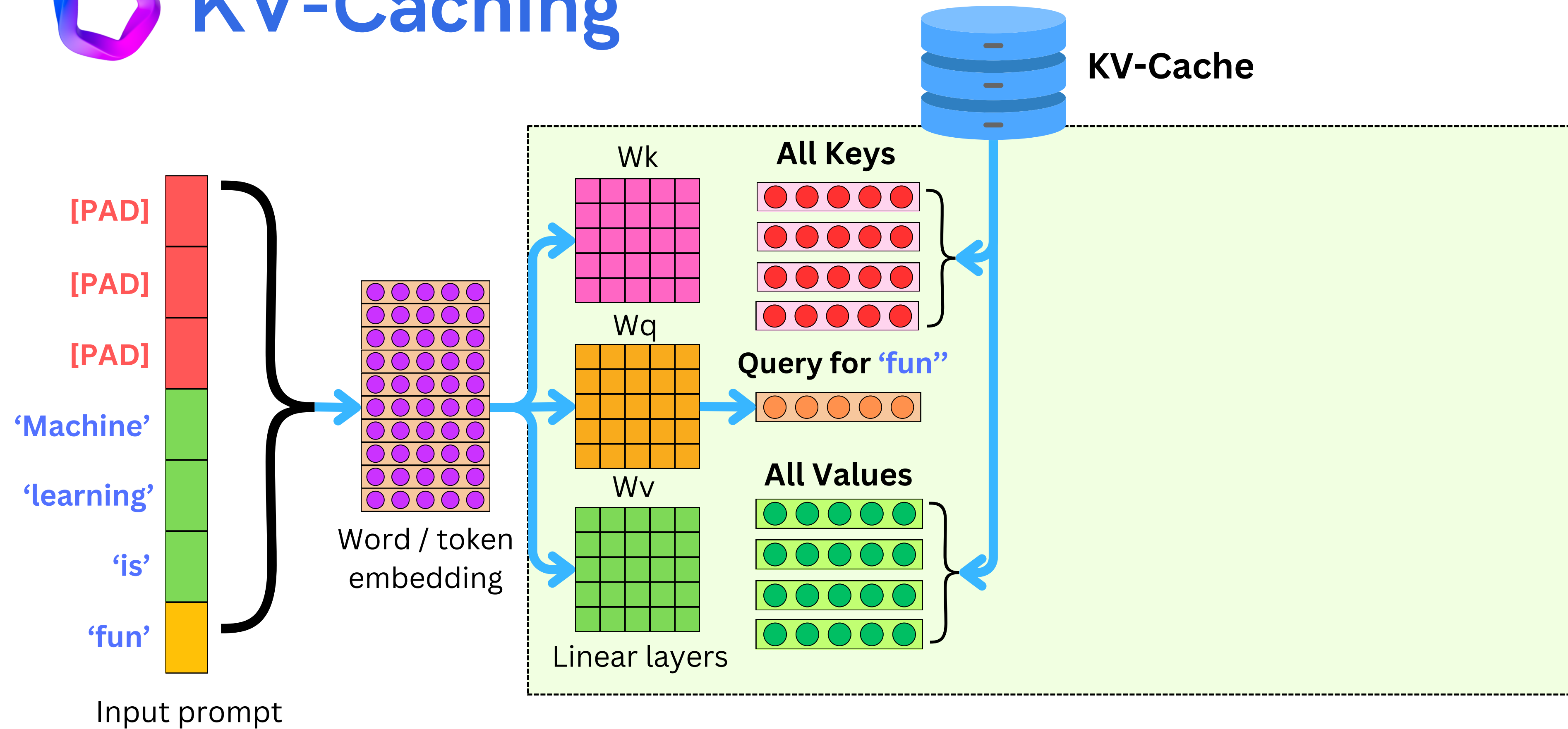


KV-Caching



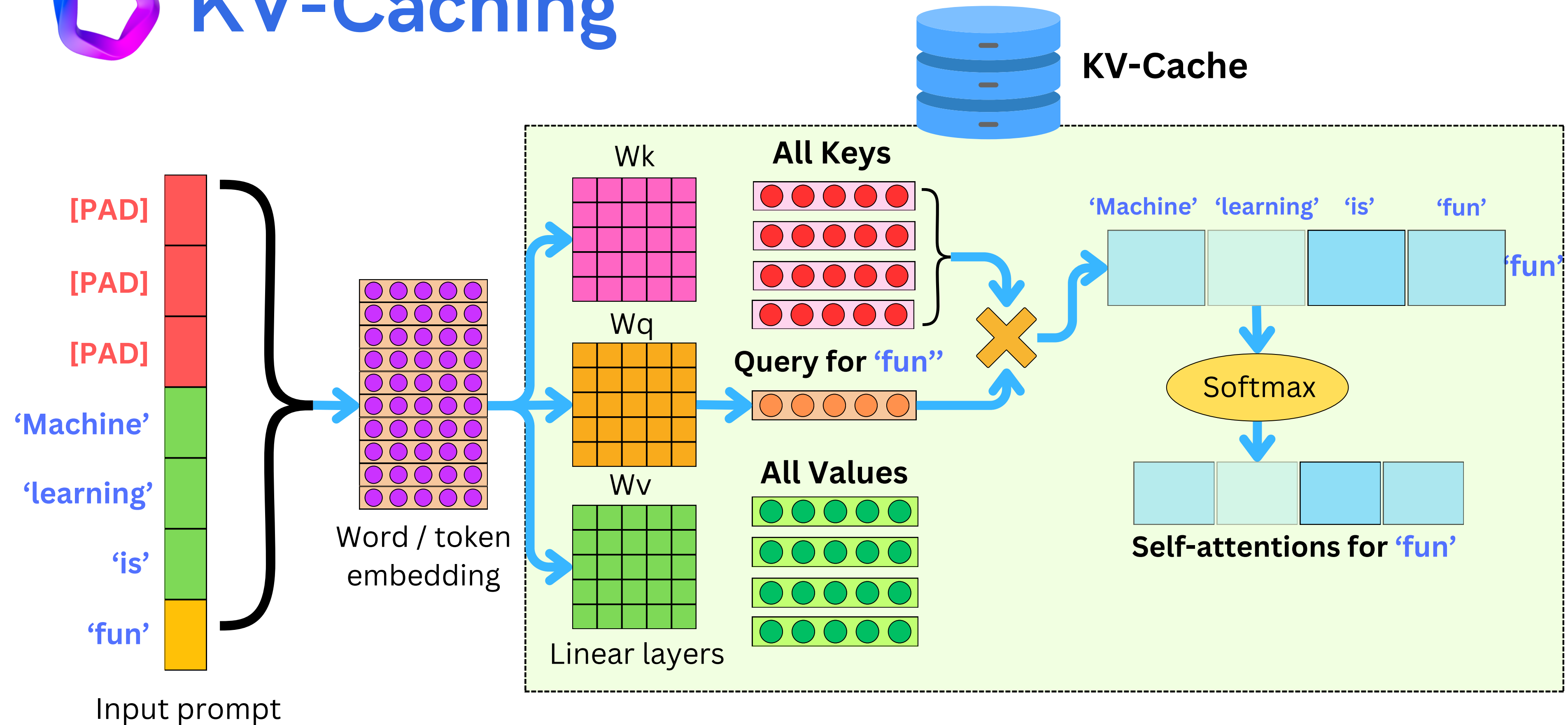


KV-Caching



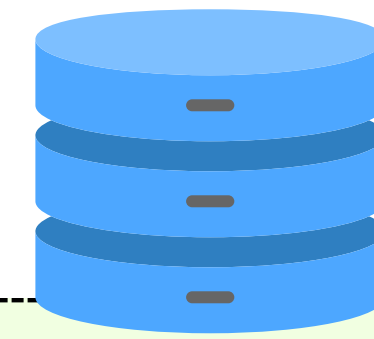


KV-Caching

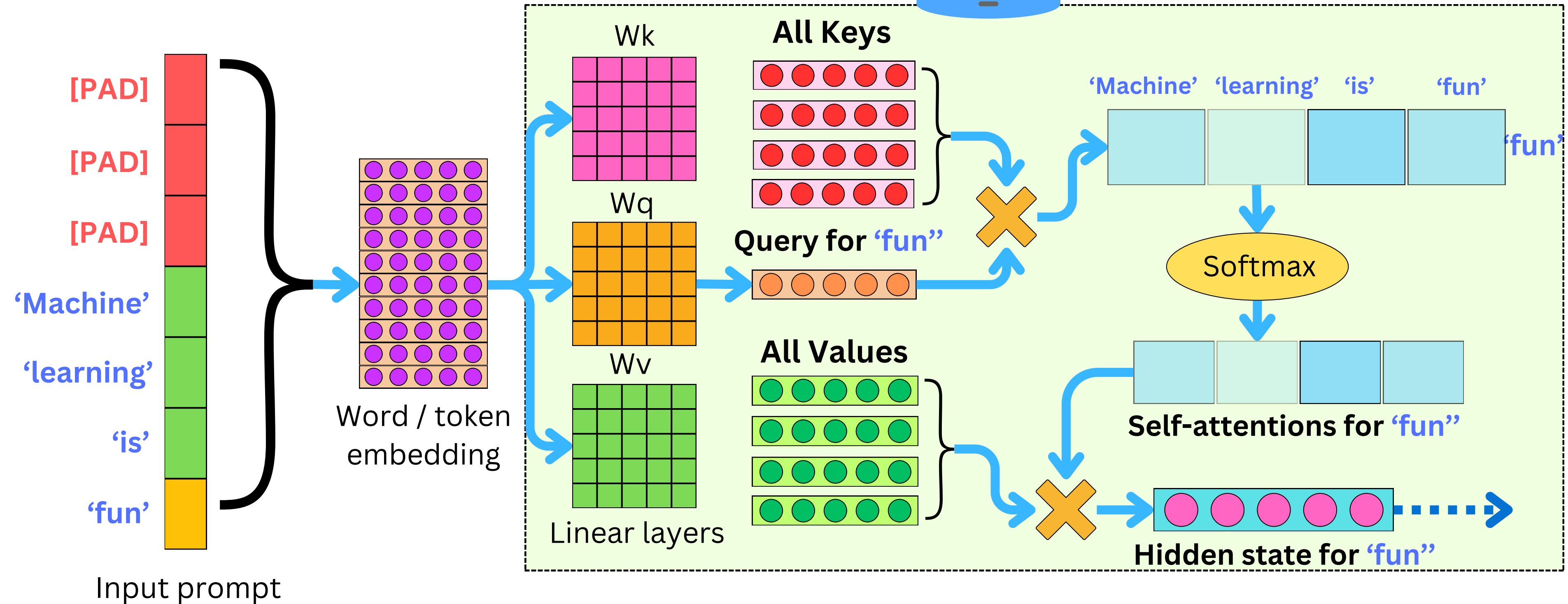




KV-Caching

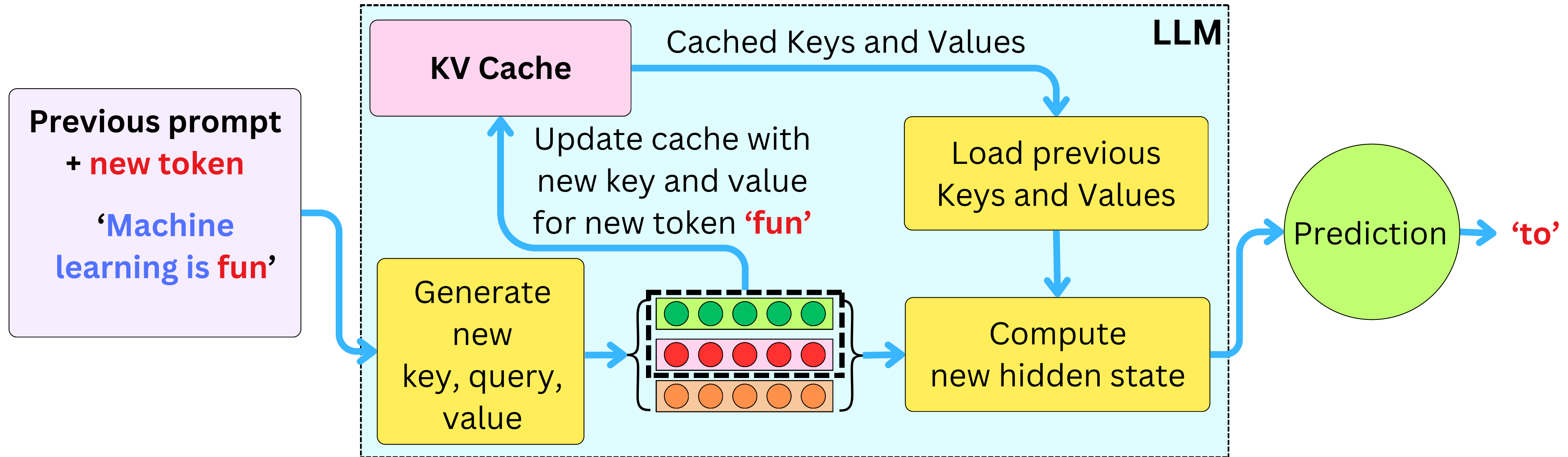


KV-Cache





KV-Caching





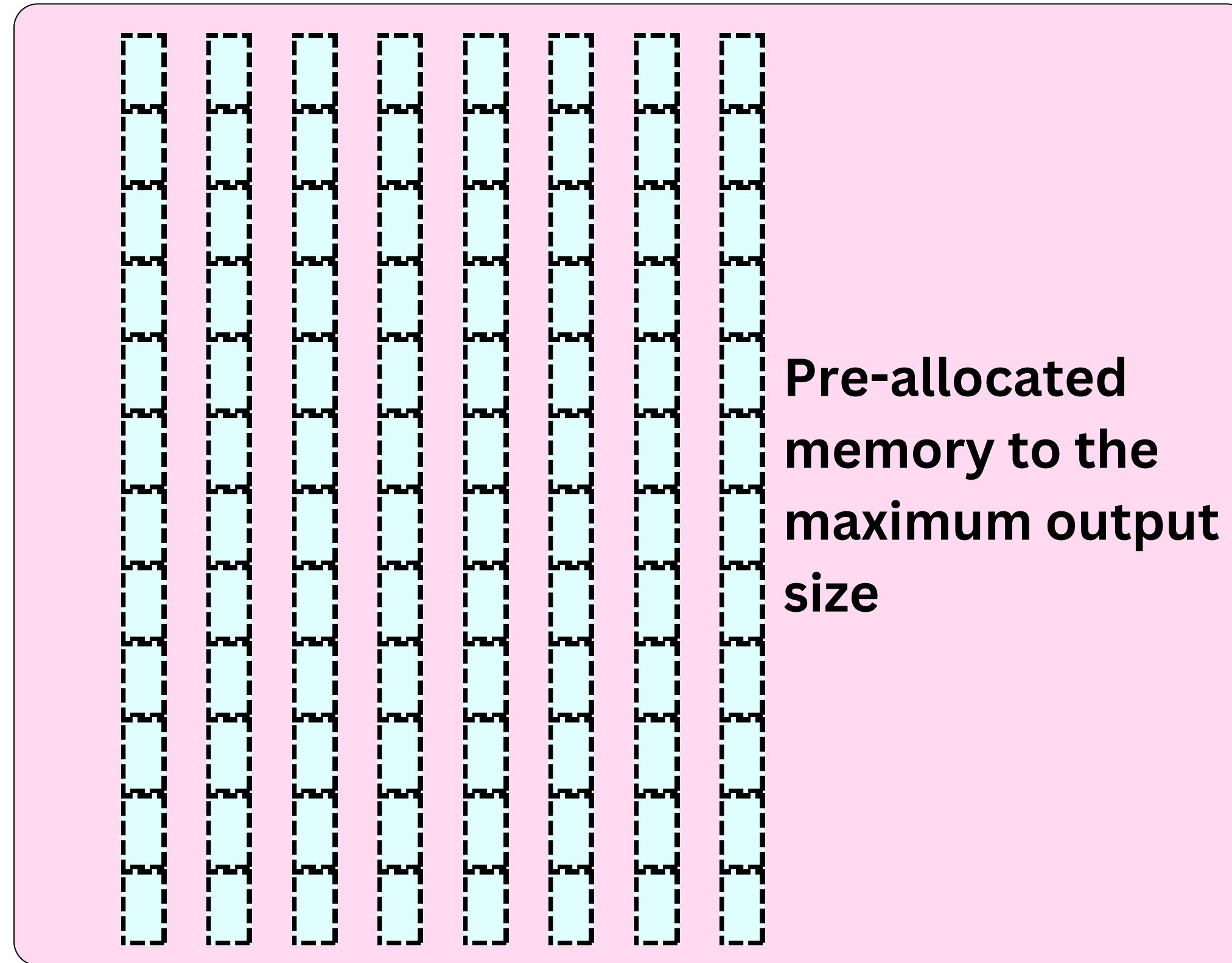
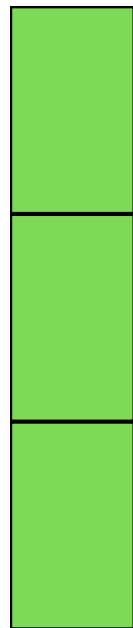
KV-Caching

‘Machine’

‘learning’

‘is’

Input prompt

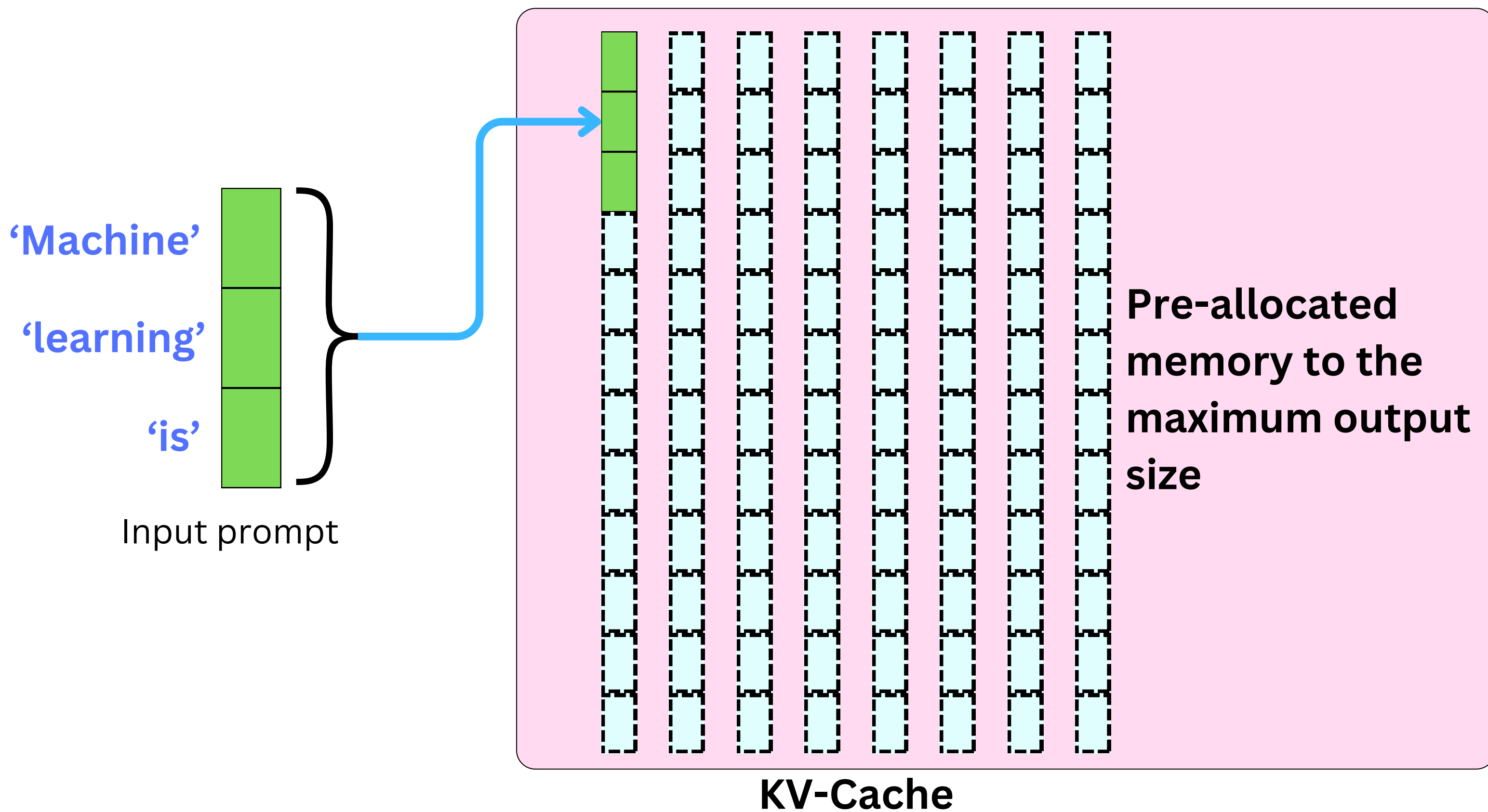


**Pre-allocated
memory to the
maximum output
size**

KV-Cache

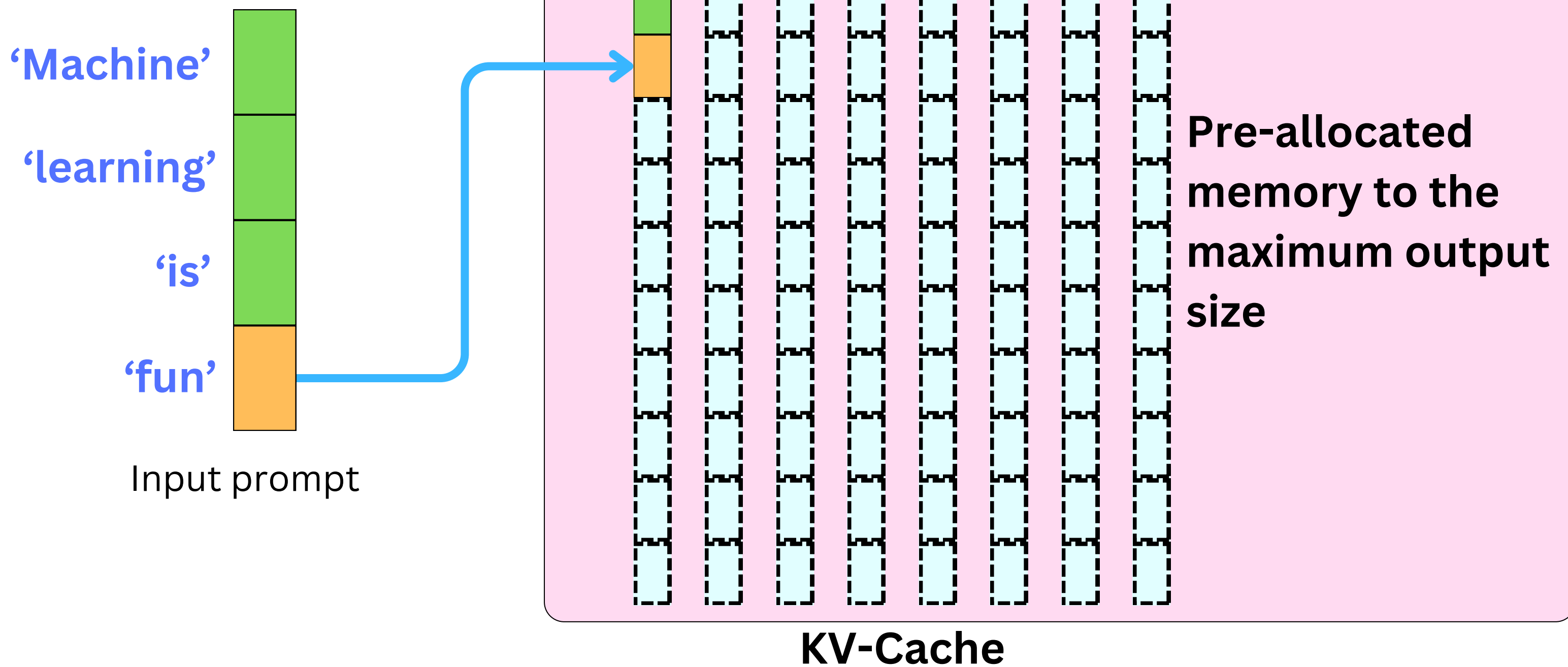


KV-Caching





KV-Caching





KV-Caching

For example, Llama 3 with 70B parameter:

- Hidden size: **8192**
- Number of layers: **80**
- Maximum number of output tokens: **4096**

For 1 token, we need

$$\boxed{2} \times \boxed{8192} \times \boxed{80} \times \boxed{2B} = \boxed{2.62MB}$$

Key and Value Number of elements per vector Number of attentions FP16 Amount of memory per token

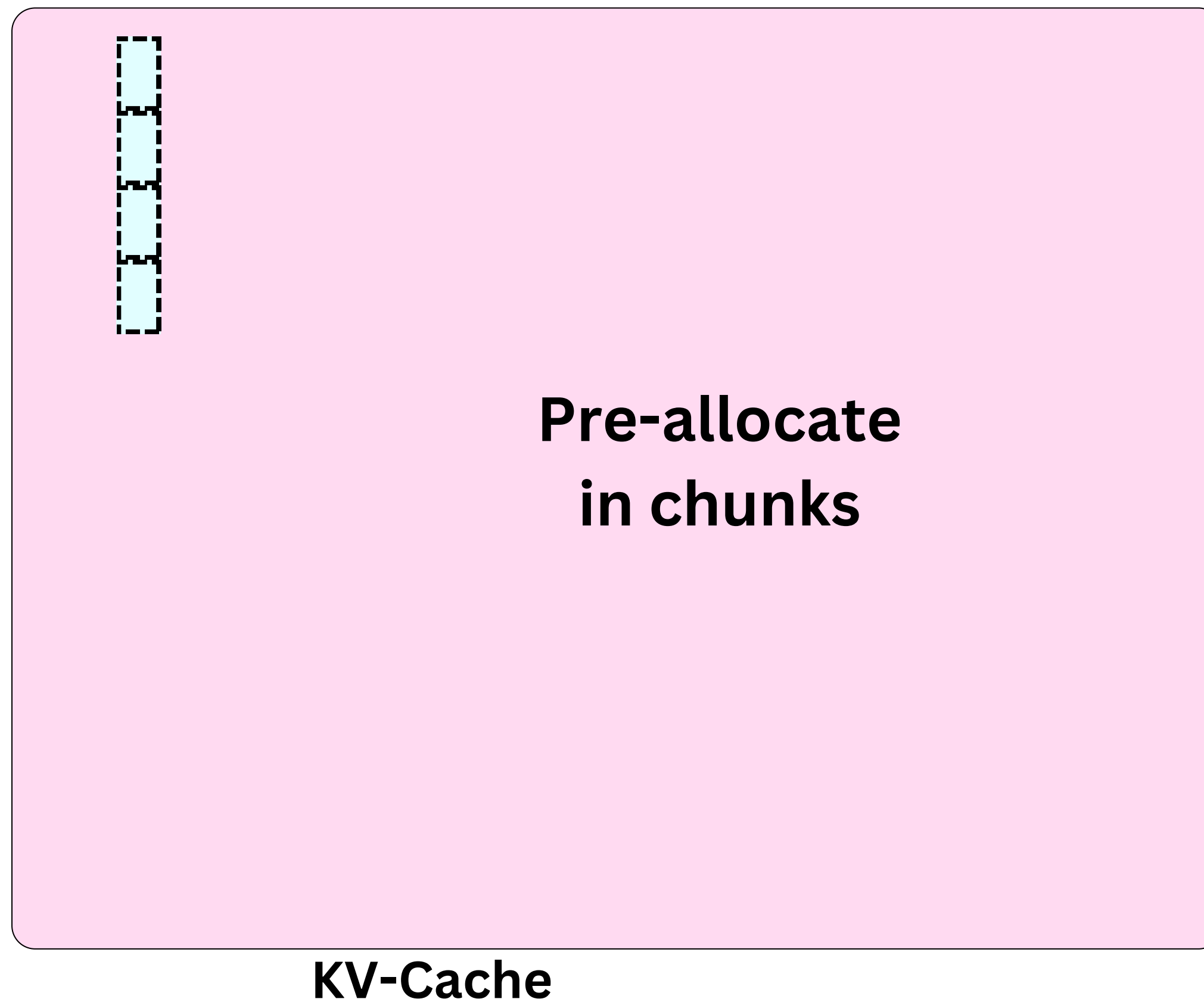
For 4096 token, we need

$$2.62MB \times 4096 = 10.73GB$$



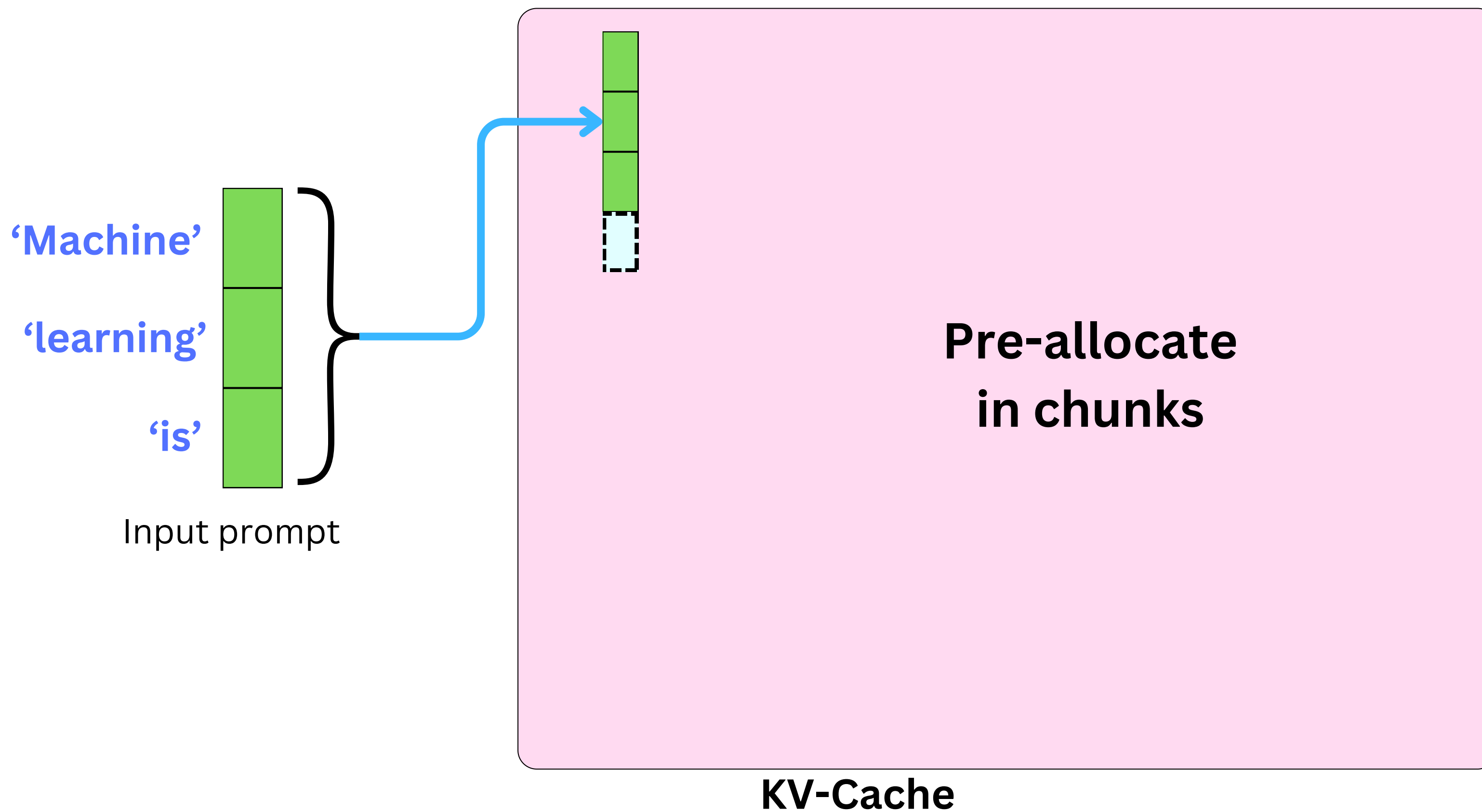
Paged-Attention

‘Machine’
‘learning’
‘is’
Input prompt



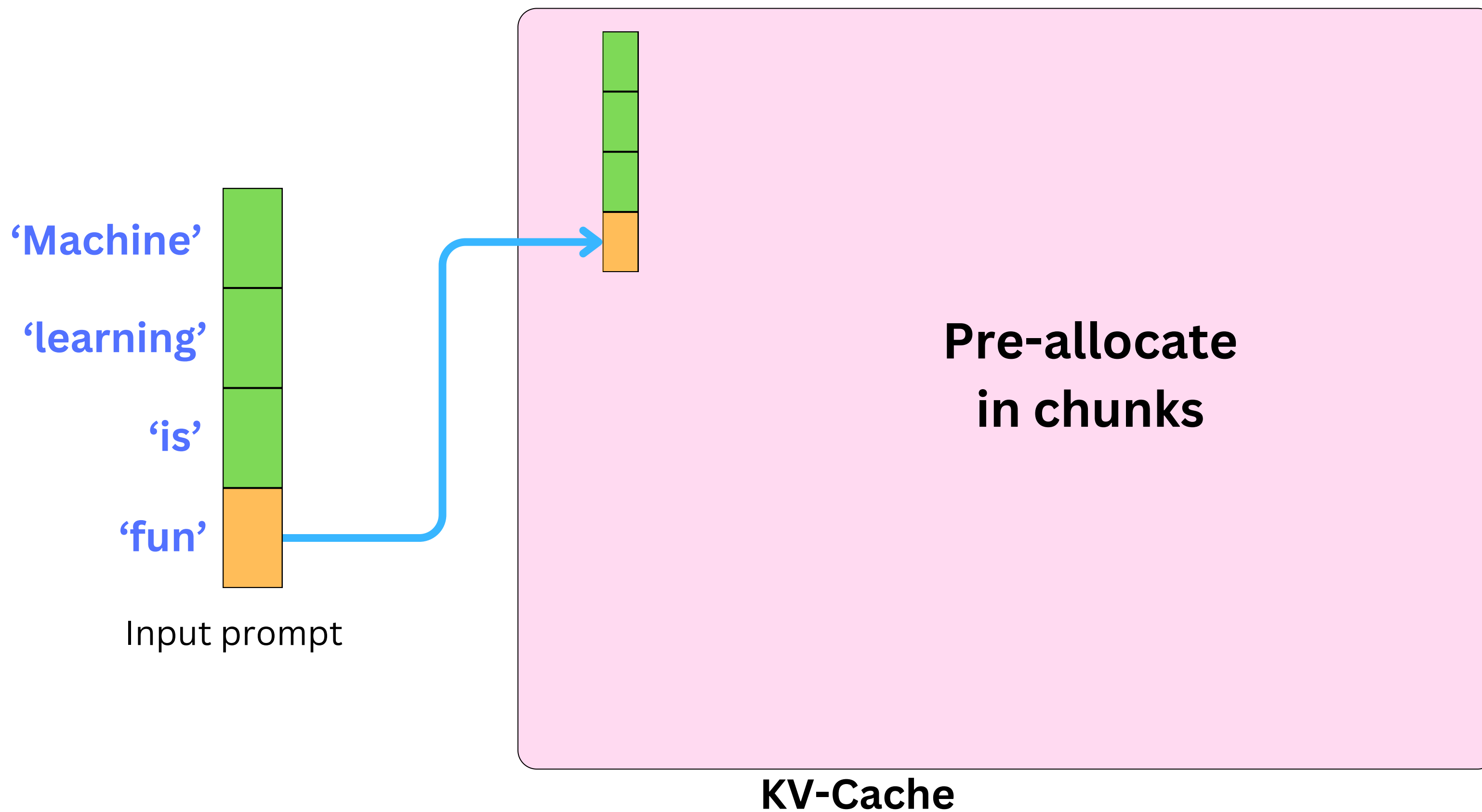


Paged-Attention





Paged-Attention





Paged-Attention

